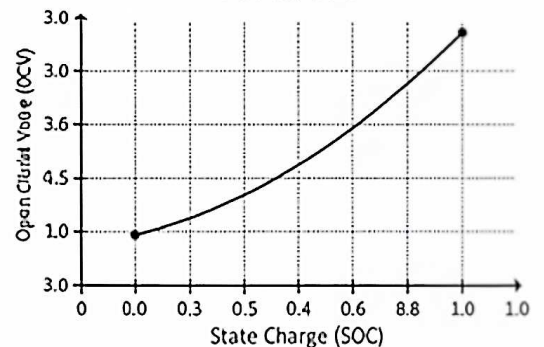


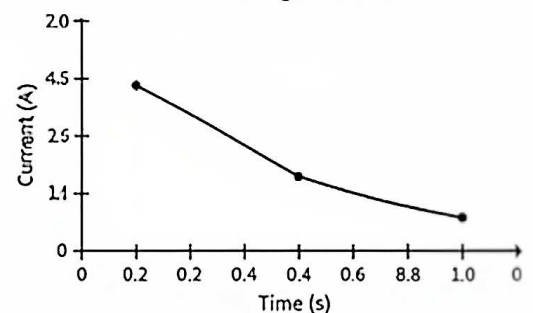
Modeling and Simulation Companion to Battery Technology for Electrical Engineers

```
class BatteryCell:
def [intiiH] = capacity, ocV-data);
def capacity = capacity, ocV-data);
def simulate, current, dt)
def simulate(( current, dt);
    pass
/ Iintilize battery model
ocv-curve = (0.0, 3.0) 0.4, 3.7, 0.6, 3.9)
    battery = BatteryCall(100, curre)
))
```

OCV vs. SOC



Discharge Profile



Copyright © 2026 Avirup

This work is licensed under the Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0).

You are free to share and adapt this material for noncommercial purposes, provided you give appropriate attribution and indicate if changes were made.

License text: <https://creativecommons.org/licenses/by-nc/4.0/>

Publisher: Independent publication

Published as part of the SodiumIonBatteryResearch project.

Contents

1	The Research Computing Environment	1
1.1	Chapter Opening	1
1.2	Prerequisites Check	2
1.3	Environment Setup	3
1.3.1	Choose conda or venv	3
1.3.2	Create the repository folder	3
1.3.3	Create the Python environment with conda	3
1.3.4	venv fallback	4
1.3.5	Verify the Python scientific stack	4
1.3.6	Register the Jupyter kernel	5
1.3.7	Install or verify MATLAB	5
1.3.8	Install and verify Git	6
1.3.9	Minimal environment hello world	6
1.3.10	Common Install Failures and Fixes	6
1.4	Conceptual Bridge: From Battery Theory to Reproducible Research Objects	7
1.5	Guided Walkthrough 1: Build and Freeze the Python Environment . .	9
1.5.1	Walkthrough 1 code	9
1.5.2	Walkthrough 1 code explanation	10
1.5.3	Walkthrough 1 expected output	11
1.5.4	Walkthrough 1 troubleshooting	11
1.5.5	Walkthrough 1 reflection	12
1.6	Guided Walkthrough 2: Create a Research Project Scaffold That Will Still Make Sense Later	12
1.6.1	Walkthrough 2 code	12
1.6.2	Walkthrough 2 code explanation	14
1.6.3	Walkthrough 2 expected output	14
1.6.4	What correct structure looks like	14
1.6.5	Walkthrough 2 troubleshooting	15
1.6.6	Walkthrough 2 reflection	15
1.7	Guided Walkthrough 3: Start a Research Log and Capture Decisions Before They Disappear	15
1.7.1	Walkthrough 3 code	15
1.7.2	Suggested first log entry content	17
1.7.3	Walkthrough 3 code explanation	18
1.7.4	Walkthrough 3 expected output	18
1.7.5	What belongs in the log and what does not	18
1.7.6	Walkthrough 3 troubleshooting	19

1.7.7	Walkthrough 3 reflection	19
1.8	Guided Walkthrough 4: Initialize Git and Build a Commit History Worth Keeping	19
1.8.1	Walkthrough 4 code	19
1.8.2	A meaningful 10-commit sequence	21
1.8.3	Walkthrough 4 code explanation	21
1.8.4	Walkthrough 4 expected output	21
1.8.5	What makes a commit “meaningful” in research	22
1.8.6	Walkthrough 4 troubleshooting	22
1.8.7	Walkthrough 4 reflection	22
1.9	Guided Walkthrough 5: Run a “Hello Battery” Sanity Check in Python and MATLAB	22
1.9.1	Part A: Python sanity check	23
1.9.2	Part B: MATLAB toolbox verification and sanity check	26
1.10	Open-Ended Exercises	30
1.10.1	Exercise 1	30
1.10.2	Exercise 2	30
1.10.3	Exercise 3	30
1.11	What Changes for Sodium-Ion?	30
1.12	Worked Solutions to the Open-Ended Exercises	31
1.12.1	Solution to Exercise 1	31
1.12.2	Solution to Exercise 2	32
1.12.3	Solution to Exercise 3	33
1.13	Chapter Summary and Skill Checklist	35
1.14	Deliverable	35
1.15	Further Practice and Reading	36
2	Scientific Python Refresher for Battery Work	37
2.1	Chapter Opening	37
2.2	Prerequisites Check	38
2.3	Environment Setup	39
2.3.1	Activate the Chapter 1 environment	39
2.3.2	Verify the scientific stack	39
2.3.3	Create a notebook and script location for this chapter	39
2.3.4	Minimal solve_ivp hello world	40
2.3.5	Launch JupyterLab and confirm the kernel	40
2.3.6	Common setup failures and fixes	40
2.4	Conceptual Bridge: From the Equivalent Circuit in the Textbook to a State Vector in Python	41
2.5	Guided Walkthrough 1: NumPy Patterns You Will Use Constantly . . .	43
2.5.1	Walkthrough 1 code	43
2.5.2	Walkthrough 1 code explanation	44
2.5.3	Walkthrough 1 expected output	45
2.5.4	Walkthrough 1 troubleshooting	45
2.5.5	Walkthrough 1 reflection	46
2.6	Guided Walkthrough 2: Solve a Second-Order RC Model with solve_ivp	46
2.6.1	Solver choice before code	46

2.6.2	Walkthrough 2 code	47
2.6.3	Walkthrough 2 code explanation	50
2.6.4	Walkthrough 2 expected output	51
2.6.5	Walkthrough 2 troubleshooting	51
2.6.6	Walkthrough 2 reflection	52
2.7	Dataset Integration: A Public Battery Time-Series File	52
2.8	Guided Walkthrough 3: Parse, Clean, and Plot a Public Battery Dataset with Pandas and Matplotlib	53
2.8.1	Walkthrough 3 code	53
2.8.2	Walkthrough 3 code explanation	56
2.8.3	Walkthrough 3 expected output	57
2.8.4	Walkthrough 3 troubleshooting	57
2.8.5	Walkthrough 3 reflection	58
2.9	Guided Walkthrough 4: Fit a Second-Order RC Model to Synthetic Noisy Data	58
2.9.1	Walkthrough 4 code	58
2.9.2	Walkthrough 4 code explanation	62
2.9.3	Walkthrough 4 expected output	62
2.9.4	Walkthrough 4 troubleshooting	63
2.9.5	Walkthrough 4 reflection	63
2.10	Guided Walkthrough 5: Reproduction Exercise - Recreate Figure 10 from Foles et al. (2022)	63
2.10.1	Our reproduction choices	64
2.10.2	Walkthrough 5 code	64
2.10.3	Walkthrough 5 code explanation	68
2.10.4	Walkthrough 5 expected output	68
2.10.5	Walkthrough 5 troubleshooting	69
2.10.6	Walkthrough 5 reflection	69
2.11	Open-Ended Exercises	69
2.11.1	Exercise 1: Make the ODE problem more stiff	69
2.11.2	Exercise 2: Fit the wrong model on purpose	70
2.11.3	Exercise 3: Flatten the OCV curve to mimic sodium-ion behavior	70
2.12	Worked Solutions to Open-Ended Exercises	70
2.12.1	Solution to Exercise 1	70
2.12.2	Solution to Exercise 2	72
2.12.3	Solution to Exercise 3	74
2.13	What Changes for Sodium-Ion?	75
2.14	Chapter Summary and Skill Checklist	76
2.15	Deliverable	76
2.16	Further Practice and Reading	77
3	Your First PyBaMM Simulation	79
3.1	Chapter Opening	79
3.2	Prerequisites Check	80
3.3	Environment Setup	81
3.3.1	Activate the Chapter 1 environment	81
3.3.2	Install PyBaMM with a pinned version	81

3.3.3	Verify the install with a minimal “hello battery”	81
3.3.4	Confirm Jupyter sees the correct environment	82
3.3.5	Common install failures and fixes	82
3.4	Conceptual Bridge: From Textbook DFN Equations to PyBaMM Objects	83
3.5	Guided Walkthrough 1: Meet the PyBaMM Object Hierarchy	85
3.5.1	Walkthrough 1 code	85
3.5.2	Walkthrough 1 explanation	86
3.5.3	Walkthrough 1 expected output	87
3.5.4	Walkthrough 1 troubleshooting	87
3.5.5	Walkthrough 1 reflection	87
3.6	Guided Walkthrough 2: Run a Default DFN Simulation and Interpret the Big Curves	88
3.6.1	Walkthrough 2 code	88
3.6.2	Walkthrough 2 explanation	89
3.6.3	Walkthrough 2 expected output	90
3.6.4	Walkthrough 2 troubleshooting	90
3.6.5	Walkthrough 2 reflection	90
3.7	Guided Walkthrough 3: Open the Cell and Inspect Internal Variables	91
3.7.1	Walkthrough 3 code	91
3.7.2	Walkthrough 3 explanation	94
3.7.3	Walkthrough 3 expected output	94
3.7.4	Walkthrough 3 troubleshooting	95
3.7.5	Walkthrough 3 reflection	95
3.8	Guided Walkthrough 4: Compare SPM, SPMe, and DFN on the Same Duty Cycle	96
3.8.1	Walkthrough 4 code	96
3.8.2	Walkthrough 4 explanation	98
3.8.3	Walkthrough 4 expected output	98
3.8.4	Walkthrough 4 interpreting the modeling choice	99
3.8.5	Walkthrough 4 troubleshooting	99
3.8.6	Walkthrough 4 reflection	99
3.9	Dataset Integration: Parsing a Public Battery Dataset Before Chapter 4	99
3.9.1	Dataset parsing code	100
3.9.2	How to read the parsed data	101
3.9.3	Dataset parsing expected output	102
3.10	Guided Walkthrough 5: Reproduce a Published Figure from Marquis et al. (2019)	102
3.10.1	The variable mapping we will use	103
3.10.2	Walkthrough 5 code	103
3.10.3	Walkthrough 5 explanation	105
3.10.4	Walkthrough 5 expected output	106
3.10.5	Walkthrough 5 reproduction tolerance	106
3.10.6	Walkthrough 5 troubleshooting	106
3.10.7	Walkthrough 5 reflection	107
3.11	Open-Ended Exercises	107
3.11.1	Exercise 1: How does model ranking change with current? . .	107
3.11.2	Exercise 2: How sensitive is the DFN to mesh resolution? . .	107

3.11.3	Exercise 3: What happens if you compare Marquis2019 and Chen2020 under the same model?	107
3.12	Worked Solutions to the Open-Ended Exercises	108
3.12.1	Solution to Exercise 1	108
3.12.2	Solution to Exercise 2	109
3.12.3	Solution to Exercise 3	110
3.13	What Changes for Sodium-Ion?	111
3.14	Chapter Summary and Skill Checklist	111
3.15	Deliverable	112
3.16	Further Practice and Reading	113
4	Parameters, Experiments, and Drive Cycles	114
4.1	Chapter Opening	114
4.2	Prerequisites Check	115
4.3	Environment Setup	116
4.3.1	Activate the research environment	116
4.3.2	Install the chapter dependencies	116
4.3.3	Verify the install with a minimal “hello protocol”	116
4.3.4	Confirm Jupyter is using the correct interpreter	116
4.3.5	Common install failures and their fixes	117
4.4	Conceptual Bridge: What the Tool Thinks a “Parameter” and an “Experiment” Are	117
4.5	Guided Walkthrough 1: Inspect, Compare, Modify, and Save a Parameter Set	119
4.5.1	Walkthrough 1 code	119
4.5.2	Walkthrough 1 explanation	120
4.5.3	Walkthrough 1 expected output	121
4.5.4	Walkthrough 2 troubleshooting	121
4.5.5	Reflection	122
4.6	Guided Walkthrough 2: Replace an OCV Curve and a Diffusivity Function	122
4.6.1	Walkthrough 2 code	122
4.6.2	Walkthrough 2 explanation	125
4.6.3	Walkthrough 2 expected output and interpretation	125
4.6.4	Walkthrough 3 troubleshooting	125
4.6.5	Walkthrough 2 reflection	126
4.7	Guided Walkthrough 3: Encode a Full Charge-Rest-Discharge Protocol with Experiment	126
4.7.1	Walkthrough 3 code	126
4.7.2	Walkthrough 3 explanation	128
4.7.3	Walkthrough 3 expected output and interpretation	128
4.7.4	Walkthrough 4 troubleshooting	128
4.7.5	Walkthrough 3 reflection	129
4.8	Dataset Integration: Public UDDS Drive-Cycle Data	129
4.9	Guided Walkthrough 4: Load a Real UDDS Current Profile and Run It in PyBaMM	130
4.9.1	Walkthrough 4 code	130
4.9.2	Walkthrough 4 explanation	133

4.9.3	Walkthrough 4 expected output and interpretation	133
4.9.4	Walkthrough 5 troubleshooting	134
4.9.5	Walkthrough 4 reflection	134
4.10	Guided Walkthrough 5: Temperature as an Input Versus Temperature as a Coupled State	134
4.10.1	Walkthrough 5 code	134
4.10.2	Walkthrough 5 explanation	137
4.10.3	Walkthrough 5 expected output and interpretation	137
4.10.4	What could go wrong	138
4.10.5	Walkthrough 5 reflection	138
4.11	Reproduction Exercise: Recreate the Six-Step EV Ageing Protocol from Pozzato, Allam, and Onori (2022)	138
4.11.1	What we can match exactly	139
4.11.2	What we cannot match exactly from the paper alone	139
4.11.3	Our declared approximations	139
4.11.4	Reproduction code	139
4.11.5	What the reproduced figure should look like	140
4.11.6	Where “close enough” ends	141
4.12	Open-Ended Exercises	141
4.12.1	Exercise 1: Design a GITT protocol and extract quasi-OCV . . .	141
4.12.2	Exercise 2: Swap UDDS for WLTC and compare transport stress	141
4.12.3	Exercise 3: Quantify cooling sensitivity	141
4.13	Worked Solutions	142
4.13.1	Solution to Exercise 1	142
4.13.2	Solution to Exercise 2	143
4.13.3	Solution to Exercise 3	144
4.14	What Changes for Sodium-Ion?	145
4.15	Chapter Summary and Skill Checklist	146
4.16	Deliverable	147
4.17	Further Practice and Reading	148

Chapter 1

The Research Computing Environment

1.1 Chapter Opening

This chapter is about a skill that sounds administrative until the day it saves your paper: building a research computing environment you can trust. By the end of the chapter, you will not merely have Python, Jupyter, MATLAB, and Git installed. You will have a working repository, an isolated environment, a repeatable project layout, a research log template, and a pair of sanity-check simulations that prove your tools are actually behaving the way you think they are. That combination is what separates “I got a plot once” from “I can still reproduce this result six weeks later, explain where it came from, and defend it in front of an advisor or reviewer.”

This matters because simulation research fails in boring ways long before it fails in interesting ways. A paper is not weakened only by a wrong diffusion coefficient or a bad thermal assumption. It is also weakened by a missing package version, a notebook that only runs on one laptop, an unlabeled CSV, a folder full of `final_results_v2_really_final`, or a figure whose exact generation path has been forgotten. Reviewers rarely see those internal failures directly, but they feel them in the form of irreproducible methods, vague parameter provenance, suspiciously convenient plots, and “we were unable to verify your claimed workflow” when someone in the group tries to build on your work later. The point of this chapter is to prevent those failures before they become habits.

This companion is a methods book, so we will treat environment setup as part of battery research rather than a preface to it. Later chapters will ask you to run PyBaMM models, identify equivalent-circuit parameters from public datasets, implement Kalman filters in MATLAB, and reproduce published figures. None of that work is credible if you cannot say exactly which software versions you used, which script generated which output, which assumptions were teaching shortcuts and which were field-standard practice, and where each intermediate result was stored. Research competence is not only the ability to derive equations from first principles. It is also the ability to preserve the chain from equation to executable model to stored artifact. Keep several theory-textbook chapters nearby while you work. Textbook Chapter 3, *Performance Metrics and Terminology*, is the one to keep most open during this

chapter, because every file name, axis label, unit annotation, and sanity-check output should use the vocabulary from that chapter consistently. When we record current, voltage, energy, capacity, SOC, and C-rate in files and plots, we are operationalizing that chapter. Textbook Chapter 7, *Degradation Mechanisms*, matters here too, because degradation claims are especially vulnerable to poor provenance: if you do not know which dataset, temperature window, and preprocessing code produced a trend, you do not know what physical mechanism you are actually looking at. Textbook Chapter 8, *Heat in Batteries*, will become relevant as soon as we start running electrothermal models; the same reproducibility discipline we build here will later protect you from silently mixing ambient temperature assumptions or heat-generation conventions. Textbook Chapter 10, *State Estimation*, is also worth keeping in mind because BMS algorithms look mathematically neat on paper and become fragile as soon as data conventions drift. This chapter is the infrastructure layer underneath all of them.

One more framing point matters for this manual in particular: you do not have wet-lab access. That is not a weakness if you are deliberate. It means your research identity will be built around careful simulation, disciplined use of public datasets, strong reproduction practice, and clean software methods. In other words, your credibility will come from the exact things we are building here. A simulation-based battery researcher can produce publishable work without touching an Arbin cycler, but not without knowing how to keep environments isolated, analyses versioned, and outputs traceable.

So we will move slowly and do the unglamorous work properly. We will decide when to use conda and when to use venv. We will pin package versions instead of trusting memory. We will create a project structure that still makes sense when the project doubles in size. We will verify that Jupyter sees the correct kernel. We will check MATLAB installation status for the toolboxes we need later. We will initialize Git before the repository is messy. We will create a research log template before there is anything exciting to log. And then we will end with a “hello battery” in both Python and MATLAB, because a battery methods workflow should prove itself on battery-flavored output, not just `print("hello world")`.

If you do this chapter carefully, later chapters become much easier. If you rush it, later chapters become haunted. Let’s build the boring foundation well enough that the interesting work can stand on it.

1.2 Prerequisites Check

- Required software: Miniforge or Anaconda for environment management; Python 3.11; Git 2.43+; JupyterLab 4.4+; MATLAB R2025b recommended, R2024b acceptable
- Python packages used in this chapter: `numpy==2.3.4`, `scipy==1.16.0`, `pandas==3.0.2`, `matplotlib==3.10.8`, `jupyterlab==4.4.10`, `ipykernel==7.2.0`
- MATLAB products to verify: Simulink, Simscape, Simscape Battery, Control System Toolbox, Optimization Toolbox
- Required textbook chapters: Textbook Chapter 3 is essential; Chapters 7, 8, and 10 should be familiar conceptually

- Required prior lab chapters: none
- Estimated time: 6 to 8 hours if installs go smoothly; 8 to 10 hours if you are also installing MATLAB or learning Git from scratch

If your Python fundamentals are shaky, pause and review array creation, scripts vs notebooks, and package imports before continuing. If Git is entirely new, that is fine; this chapter assumes no research-Git experience and builds only what we need. If MATLAB is not installed yet, do not skip the verification section. Later chapters in Part III will depend on it.

1.3 Environment Setup

We will set up the project in a way that is reproducible, boring, and strong. That is exactly the right combination.

1.3.1 Choose conda or venv

For this manual, conda is the default recommendation and venv is the fallback. The reason is practical rather than ideological. Battery research stacks often include scientific libraries with compiled dependencies, and conda-forge resolves those dependencies more smoothly across Windows, macOS, and Linux than plain pip does.

Use this rule:

- conda: use it when you want the least painful scientific-stack install. Tradeoff: a slightly heavier environment manager.
- venv: use it when you already have a clean system Python and want minimal tooling. Tradeoff: more likely to hit platform-specific wheel issues.

If you do not already have a strong preference, use conda.

1.3.2 Create the repository folder

Create a working folder anywhere you keep research projects. I will call it sib-research-companion, but you may choose another name.

```
mkdir sib-research-companion
cd sib-research-companion
```

The expected result is simple: your terminal prompt should now show that you are inside the new folder. If you run `pwd` on Linux or macOS, or `cd` on Windows PowerShell, you should see the new path.

1.3.3 Create the Python environment with conda

Create a file named `environment.yml` in the repository root with the following contents.

```
name: sib-research
channels:
  - conda-forge
dependencies:
  - python=3.11
  - numpy=2.3.4
  - scipy=1.16.0
```

```
- pandas=3.0.2
- matplotlib=3.10.8
- jupyterlab=4.4.10
- ipykernel=7.2.0
- pip
```

This file is intentionally small. It is the human-maintained description of the environment, not the machine-generated lockfile. We pin the major tools exactly because this manual is teaching research habits, not “install whatever was current on the day you happened to run the notebook.”

Now create and activate the environment:

```
conda env create -f environment.yml
conda activate sib-research
```

If `conda activate` fails because your shell has not been initialized, run the shell hook once:

```
conda init bash
```

Then close and reopen the terminal and activate again.

If you prefer `mamba`, the command is equivalent and often faster:

```
mamba env create -f environment.yml
mamba activate sib-research
```

1.3.4 venv fallback

If you are not using `conda`, create a `requirements-venv.txt` file:

```
numpy==2.3.4
scipy==1.16.0
pandas==3.0.2
matplotlib==3.10.8
jupyterlab==4.4.10
ipykernel==7.2.0
```

Then create and activate the environment.

On Linux or macOS:

```
python3.11 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements-venv.txt
```

On Windows PowerShell:

```
py -3.11 -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements-venv.txt
```

The `pip install --upgrade pip` line matters. Many beginner installation failures on Windows trace back to using an old `pip` against a modern wheel ecosystem.

1.3.5 Verify the Python scientific stack

Run this exact verification snippet from the repository root after activating the environment.

```
python - <<'PY'
import numpy as np
import scipy
import pandas as pd
import matplotlib

print("Scientific Python stack verified.")
print(f"NumPy      : {np.__version__}")
print(f"SciPy      : {scipy.__version__}")
print(f"Pandas     : {pd.__version__}")
print(f"Matplotlib : {matplotlib.__version__}")
PY
```

You should see terminal output of this form:

```
Scientific Python stack verified.
NumPy      : 2.3.4
SciPy      : 1.16.0
Pandas     : 3.0.2
Matplotlib : 3.10.8
```

The exact patch versions should match your pins. If they do not, you are not in the environment you think you are in.

1.3.6 Register the Jupyter kernel

A common mistake is installing packages into one environment and launching Jupyter from another. We prevent that now.

```
python -m ipykernel install --user --name sib-research --display-name "Python
↪ (sib-research)"
```

Launch JupyterLab:

```
jupyter lab
```

Expected behavior: a browser tab opens or Jupyter prints a local URL that you can paste into a browser. In the launcher, the kernel list should contain Python (sib-research). Create a new notebook with that kernel and run:

```
import sys
print(sys.executable)
```

The path should point into your sib-research environment, not system Python.

1.3.7 Install or verify MATLAB

For the MATLAB side of this manual, MATLAB R2025b is the recommended target version. R2024b is acceptable if that is what your license or trial provides. Later chapters will lean on products beyond base MATLAB, so install or verify these now:

- MATLAB
- Simulink
- Simscape
- Simscape Battery
- Control System Toolbox
- Optimization Toolbox

Use the MathWorks installer and choose a custom install if you have the option, so you can explicitly select toolboxes. If you are using a trial or campus license, availability of Simscape Battery is the item to check most carefully; base MATLAB alone will not be enough for Part III.

Open MATLAB and run:

```
ver
```

You should see a version table listing installed products. Do not worry yet if the list is long. In the guided walkthrough below, we will write a script that checks only the products we care about.

1.3.8 Install and verify Git

Check Git:

```
git --version
```

You should see something like:

```
git version 2.43.0
```

Now configure your identity once:

```
git config --global user.name "Your Name"
git config --global user.email "your_email@example.com"
git config --global init.defaultBranch main
```

These settings become part of your commit metadata. If the email is wrong, your GitHub history later becomes harder to trace correctly.

1.3.9 Minimal environment hello world

Before we build the battery-specific sanity check later, verify that the environment can run a tiny end-to-end Python snippet.

```
python - <<'PY'
import numpy as np
time_s = np.arange(0, 6)
voltage_v = 3.2 + 0.1 * np.cos(time_s)
print("time_s:", time_s.tolist())
print("voltage_v:", np.round(voltage_v, 3).tolist())
PY
```

Expected output:

```
time_s: [0, 1, 2, 3, 4, 5]
voltage_v: [3.3, 3.254, 3.158, 3.101, 3.135, 3.228]
```

The exact floating-point rounding may differ at the third decimal place, but the structure should be identical.

1.3.10 Common Install Failures and Fixes

The most common environment problems are not mysterious. They are usually one of four things.

1. conda: command not found
Symptom: your shell cannot see conda at all.

Fix: install Miniforge or Anaconda, then run `conda init bash` and restart the terminal.

2. `ModuleNotFoundError` after a successful install

Symptom: `pip` or `conda` said the package installed, but Python still cannot import it.

Fix: you are probably in the wrong interpreter. Run `which python` on Linux/macOS or `Get-Command python` on PowerShell, and confirm it points to your intended environment.

3. Jupyter launches but the wrong packages appear inside the notebook

Symptom: the notebook imports fail even though the terminal imports succeed.

Fix: register the kernel with `python -m ipykernel install --user --name sib-research ...` and then explicitly choose Python (`sib-research`) in JupyterLab.

4. MATLAB starts but needed toolboxes are missing

Symptom: `ver` does not list Simulink, Simscape, or Simscape Battery.

Fix: rerun the installer with toolbox selection enabled, or confirm your license includes them. If you only have base MATLAB today, you can still complete this chapter, but Part III will need either a trial or a fuller license.

1.4 Conceptual Bridge: From Battery Theory to Reproducible Research Objects

In the theory textbook, a battery model lives as mathematics, physics, and interpretation. You learned to think in terms of half-reactions, transport limitations, OCV curves, degradation mechanisms, thermal balances, and state estimators. That level is essential. But a research workflow needs one more translation layer: how those ideas become stable computational objects that can be rerun, compared, and defended.

The most important shift is this: a simulation result is not just a curve. It is a claim with dependencies.

Suppose later in this manual you run a PyBaMM DFN model and obtain a terminal-voltage trace that seems to match a published figure. In theory language, that trace depends on equations, boundary conditions, parameters, and a current profile. In research-computing language, it also depends on a specific package version, a specific parameter file, a specific script, and a specific output directory. If any one of those is ambiguous, the result is weaker than it looks. The curve may be visually correct and still be scientifically fragile.

That is why environment management belongs in a battery methods book. The environment is where the abstract model becomes executable. Textbook Chapter 3 taught you to distinguish capacity from energy, OCV from terminal voltage, and SOC from SOH. Here we add another distinction that matters just as much in practice: the difference between a result and a reproducible result. A result is “I ran something and got 3.27 V after 1800 s.” A reproducible result is “Script `src/hello_battery.py`, run under environment `sib-research`, using `numpy 2.3.4` and `matplotlib 3.10.8`, produced `results/hello_battery_python.csv` and `fig-`

ures/hello_battery_python.png from commit abc1234 on 2026-04-14.” The second form looks fussier, but it is the one you can build a paper or thesis on.

This is also where notebook culture often misleads beginners. Jupyter notebooks are excellent for exploration. They are poor as the sole home of a research workflow. Why? Because notebooks interleave code, output, and state in a way that encourages hidden dependencies. A cell may run only because an earlier cell created a variable yesterday. A figure may be regenerated from a stale in-memory DataFrame rather than from the raw data. For that reason, this manual will keep using notebooks, but we will pair them with scripts in `src/` and with saved outputs in `results/`. That separation is standard practice. Exploratory work can be notebook-first. Reproducible work should become script-backed.

The folder structure matters for the same reason. When you put raw downloads, cleaned datasets, notebooks, reusable code, generated figures, and paper notes into the same directory, you erase provenance. Later, when you revisit a model, you cannot tell whether a CSV is raw, processed, smoothed, manually edited, or exported from MATLAB. A clean project structure is not aesthetic minimalism. It is a physical embodiment of method.

Git plays an equally important role. Many people first meet Git as a collaboration tool, but in research it is just as valuable as a memory tool. A commit is a checkpoint in your reasoning. It lets you say, “This is the exact state of the repository before I changed the preprocessing,” or “This figure was generated before I changed the OCV smoothing routine.” That matters enormously once you start reproducing published papers. If a reproduction attempt diverges from the target figure, you want the freedom to compare commit states rather than guessing which invisible notebook change caused the drift.

Research logs are the human counterpart to Git history. Git can tell you which files changed. It cannot tell you why you distrusted a data column, why you chose one solver tolerance over another, or why you decided a mismatch with a paper was acceptable. Those decisions must live somewhere readable. A dated Markdown log is a remarkably effective choice because it is plain text, version-controllable, diffable, and searchable. It also trains you to think like a researcher rather than a code runner. You are not merely asking, “Did the script execute?” You are asking, “What assumption did I just make, and will future-me remember it?”

This is especially important for sodium-ion work. Textbook Chapter 13 emphasized that sodium-ion workflows cannot always inherit lithium-ion assumptions unchanged. OCV shapes differ. Dataset availability is worse. Figure digitization is more common because parameter sets and full data releases are less standardized. That means provenance matters even more. In a sparse-data setting, every preprocessing choice carries extra weight.

So the bridge from theory to tools is not only about learning commands. It is about learning how a scientific argument is preserved in software form. Equations become scripts and parameter files. Assumptions become log entries and config choices. Outputs become versioned artifacts. Reproducibility becomes the operating system of the whole project. With that frame in place, the next sections will feel less like computer setup and more like the beginning of actual battery research.

1.5 Guided Walkthrough 1: Build and Freeze the Python Environment

Learning objective: Create an isolated Python environment, verify that it is the one you are actually using, and generate a machine-readable record of its package versions.

Before we write any battery-flavored code, we need to establish one discipline that will repeat throughout this companion: every environment should be inspectable. It is not enough to say “I used Python with NumPy and SciPy.” That description is too vague for research. We want a human-readable environment specification and a machine-generated environment record.

1.5.1 Walkthrough 1 code

First, create `environment.yml` exactly as shown earlier if you have not already done so. Then create a script called `src/env_report.py`.

```
from __future__ import annotations

import json
import platform
import subprocess
import sys
from datetime import datetime, timezone
from importlib.metadata import PackageNotFoundError, version
from pathlib import Path

PACKAGES_TO_REPORT = [
    "numpy",
    "scipy",
    "pandas",
    "matplotlib",
    "jupyterlab",
    "ipykernel",
]

def package_version(package_name: str) -> str:
    try:
        return version(package_name)
    except PackageNotFoundError:
        return "NOT_INSTALLED"

def git_commit_hash() -> str:
    try:
        return subprocess.check_output(
            ["git", "rev-parse", "HEAD"],
            text=True,
        ).strip()
    except Exception:
        return "NOT_A_GIT_REPOSITORY"

def main() -> None:
    results_dir = Path("results")
    results_dir.mkdir(parents=True, exist_ok=True)
```

```

report = {
    "generated_utc": datetime.now(timezone.utc).isoformat(),
    "python_executable": sys.executable,
    "python_version": platform.python_version(),
    "platform": platform.platform(),
    "git_commit": git_commit_hash(),
    "packages": {
        package_name: package_version(package_name)
        for package_name in PACKAGES_TO_REPORT
    },
}

output_path = results_dir / "environment_report.json"
output_path.write_text(json.dumps(report, indent=2), encoding="utf-8")

print(f"Environment report written to: {output_path}")
print(json.dumps(report, indent=2))
if __name__ == "__main__":
    main()

```

Run it:

```

mkdir -p src results
python src/env_report.py

```

1.5.2 Walkthrough 1 code explanation

The `from __future__ import annotations` line is a small modern-Python convenience that keeps type annotations lightweight. It is not strictly necessary here, but it is a good habit and will appear again later.

We import `json`, `platform`, `subprocess`, and `sys` because we want the report to include not only package versions but also interpreter and operating-system information. That is important because a solver bug can be platform-dependent, and a kernel mismatch often shows up first in `sys.executable`.

`datetime.now(timezone.utc).isoformat()` deliberately uses UTC rather than local time. In collaborative or long-running projects, UTC timestamps reduce ambiguity. If you later compare logs from two machines or time zones, UTC keeps entries aligned. The `package_version` function wraps `importlib.metadata.version`. That call asks the installed environment for a package version without importing the whole package. This is slightly cleaner than importing every package just to read `__version__`, and it works for most Python distributions.

The `git_commit_hash` function tries to capture the current repository state. If Git has not been initialized yet, it returns `NOT_A_GIT_REPOSITORY` instead of crashing. That is intentional. A good environment-report script should degrade gracefully.

Inside `main`, we create `results/` if it does not already exist. This matters because scripts that assume preexisting folders are brittle. We are teaching robust patterns from the start.

The `report` dictionary contains five categories of information: generation time, Python executable path, Python version, operating-system platform, Git commit, and selected packages. This is not the maximum information you could capture, but it is

enough to answer the core reproducibility question: “What environment produced this output?”

The report is written to `results/environment_report.json`. JSON is a sensible choice because it is plain text, structured, easy to diff, and easy to read back later from Python or MATLAB if needed.

1.5.3 Walkthrough 1 expected output

In the terminal, you should see something like:

Environment report written to: `results/environment_report.json`

```
{
  "generated_utc": "2026-04-14T12:34:56.789012+00:00",
  "python_executable":
    ↪ "/home/yourname/miniforge3/envs/sib-research/bin/python",
  "python_version": "3.11.11",
  "platform": "Linux-6.8.0-57-generic-x86_64-with-glibc2.39",
  "git_commit": "NOT_A_GIT_REPOSITORY",
  "packages": {
    "numpy": "2.3.4",
    "scipy": "1.16.0",
    "pandas": "3.0.2",
    "matplotlib": "3.10.8",
    "jupyterlab": "4.4.10",
    "ipykernel": "7.2.0"
  }
}
```

Three details matter most.

First, `python_executable` should clearly point into your intended environment. If it points to system Python, stop and fix that before doing anything else.

Second, every package should show the pinned version you intended. If one shows `NOT_INSTALLED`, your environment is incomplete.

Third, `git_commit` will probably say `NOT_A_GIT_REPOSITORY` the first time. That is expected. Later, once Git is initialized, the same script will start recording commit hashes automatically. That change is the first small example of how good tooling compounds.

1.5.4 Walkthrough 1 troubleshooting

1. `ModuleNotFoundError: No module named 'importlib.metadata'`
This can happen only on older Python versions.
Fix: use Python 3.11 as specified.
2. The script writes a report, but the versions are not the pinned ones
Symptom: for example, `numpy` shows a different version from `environment.yml`.
Fix: confirm the activated environment with `python -c "import sys; print(sys.executable)"`.
3. `git_commit` always says `NOT_A_GIT_REPOSITORY`
Symptom: even after you think you initialized Git.

Fix: make sure you are running the script from the repository root, not from some parent or sibling folder.

1.5.5 Walkthrough 1 reflection

This exercise taught the first durable pattern of the manual: every meaningful output should be accompanied by enough environment metadata to explain where it came from. We will reuse this idea later when fitting parameters, exporting PyBaMM runs, and reproducing published figures.

1.6 Guided Walkthrough 2: Create a Research Project Scaffold That Will Still Make Sense Later

Learning objective: Build a folder structure that cleanly separates raw data, processed data, notebooks, reusable code, outputs, and paper materials.

A common beginner failure in computational battery work is building a project around a single notebook and a flat directory. That works for one evening. It fails for a semester. We are going to create a cross-platform project scaffold with Python so that anyone on any operating system can reproduce it exactly.

1.6.1 Walkthrough 2 code

Create `src/bootstrap_project.py`:

```
from __future__ import annotations
```

```
from pathlib import Path
```

```
PROJECT_DIRECTORIES = [
    "data/raw",
    "data/processed",
    "notebooks",
    "src",
    "results",
    "figures",
    "papers",
    "logs",
    "matlab",
]
```

```
README_TEXT = """# Sodium-Ion Battery Research Companion Workspace
```

```
This repository contains the hands-on exercises developed from the battery
↪ tools companion manual.
```

```
## Environment
```

```
Create the main Python environment with:
```

```
~~~bash
conda env create -f environment.yml
conda activate sib-research
```

```
python -m ipykernel install --user --name sib-research --display-name "Python
↳ (sib-research)"
~~~
```

Repository layout

- `data/raw/` stores downloaded datasets exactly as obtained.
- `data/processed/` stores cleaned or transformed datasets.
- `notebooks/` stores exploratory notebooks.
- `src/` stores reusable Python scripts and modules.
- `results/` stores machine-readable outputs such as CSV and JSON files.
- `figures/` stores generated figures ready for reports or papers.
- `papers/` stores paper notes, PDFs, and reproduction plans.
- `logs/` stores dated research-log entries.
- `matlab/` stores MATLAB scripts and Live Scripts.

Reproducibility rule

If an output matters, it should be reproducible from a script, recorded in the
↳ log, and traceable to a commit.
"""

```
GITKEEP_DIRECTORIES = [
    "data/raw",
    "data/processed",
    "notebooks",
    "results",
    "figures",
    "papers",
    "logs",
    "matlab",
]
def main() -> None:
    for directory in PROJECT_DIRECTORIES:
        Path(directory).mkdir(parents=True, exist_ok=True)

    for directory in GITKEEP_DIRECTORIES:
        gitkeep_path = Path(directory) / ".gitkeep"
        gitkeep_path.touch(exist_ok=True)

    readme_path = Path("README.md")
    if not readme_path.exists():
        readme_path.write_text(README_TEXT, encoding="utf-8")

    print("Project scaffold created successfully.")
    for directory in PROJECT_DIRECTORIES:
        print(f" - {directory}")
if __name__ == "__main__":
    main()
```

Run it:

```
python src/bootstrap_project.py
```

Then inspect the directory tree:

```
find . -maxdepth 2 -type d | sort
```

On Windows PowerShell, use:

```
Get-ChildItem -Directory -Recurse -Depth 2 | Select-Object FullName
```

1.6.2 Walkthrough 2 code explanation

The `PROJECT_DIRECTORIES` list is the whole design in one place. This is deliberate. If your project structure lives only in shell history, it becomes hard to audit and hard to reuse. By representing the scaffold as a Python list, we make the structure explicit and version-controllable.

Each folder has a methodological role. `data/raw/` is sacred: files there should be exact downloads from public sources and should not be edited in place. `data/processed/` is where parsed, renamed, resampled, or cleaned files go. `notebooks/` is exploratory. `src/` is reusable code. `results/` and `figures/` hold outputs, but they are separated because machine-readable tables and publication-ready figures are not the same artifact. `papers/` is where reading notes, PDFs, and reproduction plans live. `logs/` captures reasoning. `matlab/` isolates the second language ecosystem we will need later.

The `README_TEXT` string is not filler. A repository without a short explanation of its purpose, layout, and environment commands becomes opaque surprisingly fast. Think of the `README` as the front panel of the lab bench.

The `GITKEEP_DIRECTORIES` list exists because Git does not track empty directories. The `.gitkeep` files are placeholders that let you commit an intended structure before data and outputs arrive.

The script only writes `README.md` if it does not already exist. That protects later manual edits and is a small example of idempotent design: you should be able to rerun setup scripts without damaging work.

1.6.3 Walkthrough 2 expected output

The terminal should print:

```
Project scaffold created successfully.
```

- data/raw
- data/processed
- notebooks
- src
- results
- figures
- papers
- logs
- matlab

When you inspect the directory tree, you should see those folders under the repository root. The exact ordering may differ by platform, but nothing should be missing.

1.6.4 What correct structure looks like

A good structure after this step has three properties.

First, it is semantically meaningful. Someone joining your project should be able to guess where a downloaded dataset belongs and where a generated figure belongs without asking you.

Second, it separates source from artifact. Scripts go in `src/`; generated outputs go in `results/` or `figures/`. This keeps the repository conceptually clean.

Third, it is expandable. When we later add dataset loaders, PyBaMM experiments, MATLAB estimators, and reproduction notebooks, this scaffold will still hold.

1.6.5 Walkthrough 2 troubleshooting

1. The script runs but folders are created in the wrong place
Symptom: you accidentally ran it from your home directory or a parent folder.
Fix: `cd` into the repository root and run it again.
2. The `README.md` does not appear
Symptom: the script created folders but not the `README`.
Fix: check whether a `README` already existed. The script intentionally does not overwrite one.
3. `find` is not available
Symptom: the directory-inspection command fails on Windows or minimal shells.
Fix: use the platform-specific alternative shown above.

1.6.6 Walkthrough 2 reflection

This exercise taught that reproducibility begins with physical organization. Later, when we download CALCE or NASA datasets, or when we export model outputs for MATLAB, the distinction between raw, processed, and generated artifacts will save you from subtle mistakes.

1.7 Guided Walkthrough 3: Start a Research Log and Capture Decisions Before They Disappear

Learning objective: Create a log system that records decisions, failures, assumptions, and next steps in a form that Git can track and future-you can trust.

Most beginners think a research log is for major milestones. In practice, it is more valuable for the small decisions that would otherwise evaporate: why you distrusted a data column, why you clipped a current spike, why you changed a solver, why you think a reproduction mismatch is the paper's fault rather than yours. Git tracks file states. The research log tracks intent and interpretation.

1.7.1 Walkthrough 3 code

Create `src/create_log_entry.py`:

```
from __future__ import annotations

from datetime import datetime
from pathlib import Path
```

```
TEMPLATE = """# Research Log: {date}
```

```
## Objective
```

Write one sentence describing today's concrete objective.

```
## Environment
```

- Active Python environment:
- MATLAB version:
- Git commit:
- Dataset or paper in use:

```
## Work Performed
```

Describe what you actually did, not what you planned to do.

```
## Observations
```

Record numerical outputs, figure behavior, runtime notes, or anything
↪ surprising.

```
## Decisions and Rationale
```

Explain why you chose a solver, parameter bound, preprocessing rule, or file
↪ organization choice.

```
## Problems Encountered
```

Record failures honestly. Include exact error messages where useful.

```
## Next Steps
```

Write the next one to three actions you would take if you resumed tomorrow.
"""

```
def main() -> None:
    logs_dir = Path("logs")
    logs_dir.mkdir(parents=True, exist_ok=True)

    date_string = datetime.now().strftime("%Y-%m-%d")
    output_path = logs_dir / f"{date_string}_setup.md"

    if output_path.exists():
        print(f"Log entry already exists: {output_path}")
        return

    output_path.write_text(TEMPLATE.format(date=date_string), encoding="utf-8")
    print(f"Created log entry: {output_path}")
if __name__ == "__main__":
    main()
```

Run it:

```
python src/create_log_entry.py
```


Now open the generated file in your editor and replace the placeholders with your real setup notes. A first entry might include package installation success, MATLAB toolbox status, and whether GitHub remote setup is pending.

1.7.2 Suggested first log entry content

If you want a concrete model, the first real entry can look like this:

`# Research Log: 2026-04-14`

`## Objective`

Create a reproducible research environment for the battery simulation
 ↪ companion manual.

`## Environment`

- Active Python environment: ``sib-research``
- MATLAB version: ``R2025b``
- Git commit: not initialized yet
- Dataset or paper in use: none

`## Work Performed`

Installed the pinned Python stack from ``environment.yml``, registered the
 ↪ Jupyter kernel, and created the base repository scaffold using
 ↪ ``src/bootstrap_project.py``. Verified that imports for NumPy, SciPy,
 ↪ Pandas, and Matplotlib succeed. Opened MATLAB and confirmed that Simulink
 ↪ and Simscape are available, but still need to confirm Simscape Battery.

`## Observations`

Python executable points into the conda environment as expected. Jupyter sees
 ↪ the ``Python (sib-research)`` kernel. MATLAB launches normally. No datasets
 ↪ downloaded yet.

`## Decisions and Rationale`

Chose ``conda`` over ``venv`` because later chapters will use compiled scientific
 ↪ libraries and cross-platform reproducibility is more important than
 ↪ minimal tooling. Chose to keep raw and processed data in separate
 ↪ directories from day one to avoid later provenance confusion.

`## Problems Encountered`

``conda activate`` did not work until running ``conda init bash`` and restarting
 ↪ the terminal.

`## Next Steps`

1. Initialize Git and commit the scaffold.
2. Write Python and MATLAB hello-battery sanity checks.
3. Confirm Simscape Battery availability.

1.7.3 Walkthrough 3 code explanation

The script creates a dated Markdown file rather than a notebook, Word document, or proprietary note format. That choice is deliberate. Markdown is plain text, easy to diff, easy to search, and comfortable inside Git repositories.

The filename pattern YYYY-MM-DD_setup.md sorts naturally in chronological order. This sounds trivial until a project accumulates dozens of entries. Human-friendly ordering becomes a major quality-of-life feature.

The template sections are short because a research log should be sustainable. If the template is too long, you will stop using it. The fields here capture the essentials: objective, environment, work performed, observations, decisions, problems, and next steps.

The script refuses to overwrite an existing entry for the day. That is a good safety behavior. You can always edit the existing log, but you should not accidentally erase it.

1.7.4 Walkthrough 3 expected output

The terminal should show something like:

Created log entry: logs/2026-04-14_setup.md

The log file should exist and open as plain text. Once edited, it should read like a technical lab notebook rather than a diary. The difference matters. “Spent a while trying things” is not useful. “Tried conda activate before shell initialization; fixed with conda init bash and terminal restart” is useful.

1.7.5 What belongs in the log and what does not

A good rule is this: record anything that explains a result, a failure, or a future decision.

Record:

- environment changes
- dataset sources and licenses
- solver choices
- smoothing choices
- reasons for discarding or keeping data segments
- exact paper figures you are targeting in a reproduction
- uncertainties and ambiguities

Do not record:

- every command you typed if it had no consequence
- vague emotional summaries without technical content
- outputs that are already reproducible and fully captured elsewhere unless they matter interpretively

1.7.6 Walkthrough 3 troubleshooting

1. You create the template but never update it
Symptom: the repository has empty log files.
Fix: treat the generated template as a draft to be filled immediately, not later.
2. The log becomes a second README
Symptom: every entry repeats project background instead of recording decisions.
Fix: focus on what changed today.
3. You only log successes
Symptom: later you cannot reconstruct which dead ends you already tried.
Fix: record failed preprocessing or modeling attempts briefly and honestly.

1.7.7 Walkthrough 3 reflection

This exercise taught the human side of reproducibility. Later, when we compare model fidelities or troubleshoot estimation drift, your log will often be the only place where the rationale survives.

1.8 Guided Walkthrough 4: Initialize Git and Build a Commit History Worth Keeping

Learning objective: Turn the scaffold into a versioned research repository, add a battery-research-appropriate `.gitignore`, and make the first sequence of meaningful commits.

Git becomes helpful only when used before the repository is messy. Starting version control after a month of experiments is like deciding to label sample vials after mixing half of them together. We are going to initialize the repository now and establish a commit style that later supports actual research.

1.8.1 Walkthrough 4 code

Create `.gitignore` in the repository root:

```
# Python cache and notebook state
__pycache__/
*.py[cod]
.ipynb_checkpoints/

# Virtual environments
.venv/
env/
ENV/

# Tool caches
.pytest_cache/
.mypy_cache/

# OS clutter
.DS_Store
```

Thumbs.db

```
# Large local data: keep directory placeholders, ignore contents
data/raw/*
!data/raw/.gitkeep
data/processed/*
!data/processed/.gitkeep
```

```
# Temporary outputs
results/tmp/
figures/tmp/
```

```
# MATLAB generated files
*.asv
*.slxc
*.mexw64
*.mexa64
*.mexmaci64
```

Initialize the repository and make the first commits:

```
git init
git add environment.yml
git add requirements-venv.txt
git add src/bootstrap_project.py
git add src/env_report.py
git add src/create_log_entry.py
git add README.md
git add .gitignore
git add data/raw/.gitkeep
git add data/processed/.gitkeep
git add notebooks/.gitkeep
git add results/.gitkeep
git add figures/.gitkeep
git add papers/.gitkeep
git add logs/2026-04-14_setup.md
git add matlab/.gitkeep
git commit -m "Initialize reproducible battery research scaffold"
```

Now generate an environment report so the repository captures a first machine-readable environment artifact:

```
python src/env_report.py
git add results/environment_report.json
git commit -m "Add machine-readable environment report"
```

Register the current default branch explicitly:

```
git branch -M main
```

If you already created a GitHub repository in the browser, connect it:

```
git remote add origin
↪ https://github.com/YOUR_USERNAME/sib-research-companion.git
git push -u origin main
```

1.8.2 A meaningful 10-commit sequence

Your chapter deliverable asks for at least ten meaningful commits. Here is a clean sequence you can actually use over the rest of this chapter:

1. Initialize reproducible battery research scaffold
2. Add machine-readable environment report
3. Add MATLAB toolbox verification script
4. Add Python hello-battery sanity check
5. Add MATLAB hello-battery sanity check
6. Save Python sanity-check outputs
7. Save MATLAB sanity-check outputs
8. Refine README with setup and workflow notes
9. Add research-log entry for chapter 1
10. Document chapter 1 deliverable completion

These are meaningful because each one captures a coherent step, not because the messages sound formal.

1.8.3 Walkthrough 4 code explanation

The `.gitignore` is tuned for research. It does not blindly ignore `results/` and `figures/` wholesale, because some results are worth tracking. Instead, it ignores only explicitly temporary subdirectories if you create them later. That choice teaches an important distinction: some generated artifacts are disposable, while some are central to the research record.

We ignore contents of `data/raw/` and `data/processed/` by default because public battery datasets can be large, and many are better downloaded by script or documented URL than committed to Git. The `.gitkeep` exceptions preserve the folder structure in the repository. This is standard practice.

The first commit gathers the reproducibility scaffold into a single checkpoint. The second commit adds the environment report as an artifact. Notice the pattern: one commit establishes capability, the next records evidence that the capability works. We will repeat that pattern throughout the manual.

1.8.4 Walkthrough 4 expected output

After the first commit, `git status` should say:

```
On branch main
nothing to commit, working tree clean
```

If you run:

```
git log --online --decorate --graph -n 5
```

you should see a short history of your recent commits, for example:

```
* a1b2c3d (HEAD -> main) Add machine-readable environment report
* e4f5g6h Initialize reproducible battery research scaffold
```

Your commit hashes will differ, of course.

1.8.5 What makes a commit “meaningful” in research

A meaningful research commit is not necessarily a large one. It is one whose purpose you can summarize in one sentence.

Good:

- Add parser for CALCE HPPC current and voltage columns
- Fix sign convention in UDDS current import
- Compare DFN and SPMe runtime on same experiment
- Add EKF tuning sweep notebook

Poor:

- updates
- misc
- stuff
- fixes

The point is not style policing. It is traceability. Six weeks later, when a result changes, a meaningful commit history lets you find why.

1.8.6 Walkthrough 4 troubleshooting

1. `git commit` refuses because user name or email is not configured

Symptom: Git prompts you to set identity.

Fix: run the `git config --global user.name` and `user.email` commands from the setup section.

2. You accidentally commit a large dataset

Symptom: `git status` shows many files inside `data/raw/`.

Fix: check that `.gitignore` exists before `git add .`, and use `git rm --cached` on accidentally tracked data files.

3. Jupyter notebooks create noisy diffs

Symptom: commits are dominated by output-cell changes.

Fix: keep exploratory notebooks, but move stable logic into scripts under `src/`.

1.8.7 Walkthrough 4 reflection

This exercise taught that version control is part of scientific method, not just collaboration etiquette. Later, when reproducing papers or tuning estimators, your commit history will become one of the main tools for diagnosing divergence.

1.9 Guided Walkthrough 5: Run a “Hello Battery” Sanity Check in Python and MATLAB

Learning objective: Verify that both ecosystems can produce a simple, battery-flavored result with known behavior, known units, and stored outputs.

Now we do the first genuinely battery-shaped computation of the manual. It is still intentionally simple. We are not building a real cell model yet. We are building a sanity check that proves the toolchain can generate, save, and interpret a small discharge experiment in both Python and MATLAB.

We will use the following pedagogical model:

$$\text{SOC}(t) = \text{SOC}_0 - \frac{It}{3600Q_{\text{nom}}} \quad (1.1)$$

$$V_{\text{OCV}}(\text{SOC}) = 3.0 + 0.6 \text{ SOC} \quad (1.2)$$

$$V_{\text{term}}(t) = V_{\text{OCV}}(\text{SOC}(t)) - IR_0 \quad (1.3)$$

This is not a publishable model. It is a teaching model. The goal is to verify the workflow, not to represent a chemistry accurately. The model is intentionally transparent so that you can predict what the output should look like before you run it.

We choose:

- nominal capacity: $Q_{\text{nom}} = 2.0 \text{ Ah}$
- current: $I = 2.0 \text{ A}$, which is a 1C discharge
- ohmic resistance: $R_0 = 0.05 \Omega$
- initial SOC: $\text{SOC}_0 = 1.0$
- simulation time: $t_{\text{final}} = 3600 \text{ s}$

From Equation (1.1), a 1C discharge over 3600 s should take SOC from 1.0 to 0.0. From Equations (1.2) and (1.3), OCV should fall linearly from 3.6 V to 3.0 V, while terminal voltage should sit 0.1 V lower, from 3.5 V to 2.9 V. If your output does not look like that, the toolchain or the code is wrong.

1.9.1 Part A: Python sanity check

Create src/hello_battery.py:

```
from __future__ import annotations

from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
def main() -> None:
    # Create output directories if they do not already exist.
    Path("results").mkdir(parents=True, exist_ok=True)
    Path("figures").mkdir(parents=True, exist_ok=True)

    # Define a simple teaching model for a 2 Ah cell discharged at 1C.
    nominal_capacity_ah = 2.0
    discharge_current_a = 2.0
    ohmic_resistance_ohm = 0.05
    initial_soc = 1.0
    time_step_s = 1.0
    final_time_s = 3600.0

    # Build the simulation time vector.
    time_s = np.arange(0.0, final_time_s + time_step_s, time_step_s)
```

```

# Coulomb counting: SOC falls linearly under constant current discharge.
soc = initial_soc - (
    discharge_current_a * time_s / (3600.0 * nominal_capacity_ah)
)
soc = np.clip(soc, 0.0, 1.0)

# Simple linear OCV curve used only for a sanity check.
ocv_v = 3.0 + 0.6 * soc

# Terminal voltage includes an ohmic drop during discharge.
terminal_voltage_v = ocv_v - discharge_current_a * ohmic_resistance_ohm

# Instantaneous power delivered to the load.
power_w = terminal_voltage_v * discharge_current_a

# Compute delivered energy by integrating power over time.
delivered_energy_wh = np.trapz(power_w, time_s) / 3600.0

# Store results in a tidy table.
results_table = pd.DataFrame(
    {
        "time_s": time_s,
        "soc_fraction": soc,
        "ocv_v": ocv_v,
        "terminal_voltage_v": terminal_voltage_v,
        "current_a": discharge_current_a,
        "power_w": power_w,
    }
)

results_csv_path = Path("results") / "hello_battery_python.csv"
results_table.to_csv(results_csv_path, index=False)

# Create a two-panel figure.
fig, axes = plt.subplots(2, 1, figsize=(8, 8), sharex=True)

axes[0].plot(time_s, soc, linewidth=2, color="tab:blue")
axes[0].set_ylabel("SOC [-]")
axes[0].set_title("Hello Battery Sanity Check (Python)")
axes[0].grid(True, alpha=0.3)
axes[0].set_ylim(-0.05, 1.05)

axes[1].plot(time_s, ocv_v, linewidth=2, label="OCV", color="tab:green")
axes[1].plot(
    time_s,
    terminal_voltage_v,
    linewidth=2,
    label="Terminal voltage",
    color="tab:red",
)
axes[1].set_xlabel("Time [s]")
axes[1].set_ylabel("Voltage [V]")
axes[1].grid(True, alpha=0.3)

```



```

axes[1].legend()

figure_path = Path("figures") / "hello_battery_python.png"
fig.tight_layout()
fig.savefig(figure_path, dpi=200)
plt.close(fig)

print(f"Saved results table to: {results_csv_path}")
print(f"Saved figure to: {figure_path}")
print(f"Initial terminal voltage: {terminal_voltage_v[0]:.2f} V")
print(f"Final terminal voltage: {terminal_voltage_v[-1]:.2f} V")
print(f"Delivered capacity: {discharge_current_a * final_time_s /
    ↪ 3600.0:.2f} Ah")
print(f"Delivered energy: {delivered_energy_wh:.2f} Wh")
if __name__ == "__main__":
    main()

```

Run it:

```
python src/hello_battery.py
```

1.9.1.1 Python sanity check explanation

The first two `Path(...).mkdir(...)` calls make the script robust. A reproducible script should not assume that output directories already exist.

The parameter block is deliberately explicit. We are not hiding constants in cryptic variable names. This is good scientific style: a reader should be able to identify capacity, current, resistance, initial SOC, time step, and simulation horizon immediately.

`np.arange(0.0, final_time_s + time_step_s, time_step_s)` includes the final time point. If we omitted the `+ time_step_s`, the last sample at 3600 s would be missing. That sounds minor, but end-point consistency becomes important later when comparing outputs across tools.

The SOC calculation is just Equation (1.1). We then clip with `np.clip` so the value cannot drop below zero. For this chosen current and duration, the unclipped result lands exactly at zero, but clipping is still a healthy protective pattern.

The OCV line implements Equation (1.2). Again, this is not a chemistry model. It is an intentionally transparent surrogate so that the expected output is obvious.

The terminal voltage subtracts the constant ohmic drop, which is $IR_0 = 2.0 \times 0.05 = 0.1$ V. Therefore, the terminal curve should be parallel to the OCV curve and always 0.1 V lower. If it is not, the code or sign convention is wrong.

`np.trapz(power_w, time_s) / 3600.0` numerically integrates power in watts over time in seconds, then divides by 3600 to convert joules-per-second integrated over seconds into watt-hours. This is a good early reminder that unit discipline matters everywhere.

We store results as a tidy `pandas.DataFrame` because later chapters will rely heavily on tabular outputs. Even this toy script teaches the habit that plots and machine-readable tables should both be saved.

The plot is split into two panels because SOC and voltage live on different scales and tell different stories. Good plotting is part of research competence, not decoration.

1.9.1.2 Python sanity check expected output

In the terminal, you should see:

```
Saved results table to: results/hello_battery_python.csv
Saved figure to: figures/hello_battery_python.png
Initial terminal voltage: 3.50 V
Final terminal voltage: 2.90 V
Delivered capacity: 2.00 Ah
Delivered energy: 6.40 Wh
```

The CSV should contain 3601 rows, one for each second from 0 to 3600 inclusive.

The figure should have two panels.

The top panel, SOC vs time, should be a perfectly straight descending line from 1.0 at 0 s to 0.0 at 3600 s. If it curves, steps, or ends somewhere other than zero, something is off.

The bottom panel should show two straight lines. The green OCV line should descend from 3.6 V to 3.0 V. The red terminal-voltage line should descend from 3.5 V to 2.9 V and remain exactly 0.1 V below the OCV line for the entire discharge. That constant gap is the visual signature of the ohmic term in Equation (1.3).

A wrong result often looks like one of three things: the terminal voltage is above OCV, which indicates a sign error; the final SOC is negative, which indicates missing clipping or a duration mismatch; or the final voltage is not 2.9 V, which indicates incorrect current, capacity, or resistance.

1.9.1.3 Python sanity check troubleshooting

1. The script runs but no figure appears on screen
Symptom: nothing pops up visually.
Fix: this script saves the figure to disk and closes it intentionally. Check figures/hello_battery_python.png.
2. ImportError: No module named 'matplotlib' or pandas
Symptom: imports fail even though the environment exists.
Fix: confirm the environment is activated before running the script.
3. Delivered energy is not approximately 6.40 Wh
Symptom: printed energy is substantially different.
Fix: check the unit conversion in the integration step and confirm the current is 2.0 A.

1.9.1.4 Python sanity check reflection

This Python exercise taught a complete mini-workflow: define parameters, compute derived quantities, save a table, save a figure, and print interpretable summary metrics. That pattern will return in nearly every later chapter.

1.9.2 Part B: MATLAB toolbox verification and sanity check

Before we run the MATLAB battery check, verify the products we need later.

Create matlab/check_toolboxes.m:

```

requiredProducts = [
    "MATLAB"
    "Simulink"
    "Simscape"
    "Simscape Battery"
    "Control System Toolbox"
    "Optimization Toolbox"
];

installedProducts = ver;
installedNames = string({installedProducts.Name});

fprintf("MATLAB product check\n");
fprintf("-----\n");

for k = 1:numel(requiredProducts)
    isInstalled = any(installedNames == requiredProducts(k));
    if isInstalled
        statusText = "FOUND";
    else
        statusText = "MISSING";
    end
    fprintf("%-24s : %s\n", requiredProducts(k), statusText);
end

```

Run it in MATLAB:

```
check_toolboxes
```

Expected output is a short table of FOUND and MISSING statuses. If Simscape Battery is missing, note that explicitly in your research log.

Now create matlab/hello_battery.m:

```

clear;
clc;
close all;

if ~exist("results", "dir")
    mkdir("results");
end

if ~exist("figures", "dir")
    mkdir("figures");
end

nominalCapacityAh = 2.0;
dischargeCurrentA = 2.0;
ohmicResistanceOhm = 0.05;
initialSoc = 1.0;
timeStepS = 1.0;
finalTimeS = 3600.0;

timeS = (0:timeStepS:finalTimeS)';
soc = initialSoc - (dischargeCurrentA .* timeS) ./ (3600.0 .*
    ↪ nominalCapacityAh);

```

```

soc = max(0.0, min(1.0, soc));

ocvV = 3.0 + 0.6 .* soc;
terminalVoltageV = ocvV - dischargeCurrentA .* ohmicResistanceOhm;
powerW = terminalVoltageV .* dischargeCurrentA;

deliveredEnergyWh = trapz(timeS, powerW) / 3600.0;

resultsTable = table( ...
    timeS, ...
    soc, ...
    ocvV, ...
    terminalVoltageV, ...
    repmat(dischargeCurrentA, size(timeS)), ...
    powerW, ...
    'VariableNames', ...
    {'time_s', 'soc_fraction', 'ocv_v', 'terminal_voltage_v', 'current_a',
    ↪ 'power_w'} ...
);

writetable(resultsTable, fullfile("results", "hello_battery_matlab.csv"));

figureHandle = figure("Position", [100, 100, 800, 700]);

subplot(2, 1, 1);
plot(timeS, soc, "LineWidth", 2, "Color", [0.0, 0.45, 0.74]);
grid on;
ylabel("SOC [-]");
title("Hello Battery Sanity Check (MATLAB)");
ylim([-0.05, 1.05]);

subplot(2, 1, 2);
plot(timeS, ocvV, "LineWidth", 2, "Color", [0.47, 0.67, 0.19]);
hold on;
plot(timeS, terminalVoltageV, "LineWidth", 2, "Color", [0.85, 0.33, 0.10]);
grid on;
xlabel("Time [s]");
ylabel("Voltage [V]");
legend("OCV", "Terminal voltage", "Location", "best");

exportgraphics(figureHandle, fullfile("figures", "hello_battery_matlab.png"),
    ↪ "Resolution", 200);

fprintf("Saved results table to: %s\n", fullfile("results",
    ↪ "hello_battery_matlab.csv"));
fprintf("Saved figure to: %s\n", fullfile("figures",
    ↪ "hello_battery_matlab.png"));
fprintf("Initial terminal voltage: %.2f V\n", terminalVoltageV(1));
fprintf("Final terminal voltage: %.2f V\n", terminalVoltageV(end));
fprintf("Delivered capacity: %.2f Ah\n", dischargeCurrentA * finalTimeS /
    ↪ 3600.0);
fprintf("Delivered energy: %.2f Wh\n", deliveredEnergyWh);

```

Run it in MATLAB:

hello_battery

1.9.2.1 MATLAB sanity check explanation

The structure mirrors the Python version on purpose. That parallelism is pedagogically useful. Later, when we move data between PyBaMM and MATLAB, you want the two ecosystems to feel different in syntax but aligned in workflow.

The `mkdir` guards are the MATLAB equivalent of the Python directory-creation calls. Again, robust scripts should not assume a preexisting folder state.

The time vector is created as a column vector with `(0:timeStepS:finalTimeS)'`. The transpose matters because MATLAB tables and plotting functions often behave more predictably when vectors are column-oriented.

The SOC computation uses elementwise operators `.*` and `./`. This is one of the biggest MATLAB habits to internalize early. If you accidentally use matrix operators here, the script will either fail or behave incorrectly.

The clamp `max(0.0, min(1.0, soc))` is the MATLAB analogue of `np.clip`.

`table(...)` creates a labeled table rather than a bare matrix. As in Python, storing named columns is far better research practice than saving anonymous arrays and hoping you remember what each column meant.

`exportgraphics` is preferred over old screenshot-style save behavior because it gives cleaner, reproducible figure exports.

1.9.2.2 MATLAB sanity check expected output

The MATLAB command window should print:

```
Saved results table to: results\hello_battery_matlab.csv
Saved figure to: figures\hello_battery_matlab.png
Initial terminal voltage: 3.50 V
Final terminal voltage: 2.90 V
Delivered capacity: 2.00 Ah
Delivered energy: 6.40 Wh
```

The slash direction in file paths may differ by operating system.

The plot should match the Python result qualitatively and numerically. The top panel should be a straight-line SOC decline from 1 to 0. The bottom panel should show OCV and terminal-voltage lines with a constant 0.1 V gap.

If the MATLAB and Python outputs disagree materially, stop and resolve that now. Cross-tool agreement on simple cases is the first step toward trusting more complex models later.

1.9.2.3 MATLAB sanity check troubleshooting

1. MATLAB cannot find the script

Symptom: Undefined function or variable 'hello_battery'.

Fix: make sure the current MATLAB folder is the repository root or add the `matlab/` folder to the path.

2. Simscape Battery shows as missing
Symptom: the product check reports MISSING.
Fix: note it in your log. This does not block this chapter, but it will matter later.
3. The CSV is saved but the figure export fails
Symptom: exportgraphics errors.
Fix: ensure you are using R2024b or later; on older versions, use `saveas` as a fallback, but note the deviation in your log.

1.9.2.4 MATLAB sanity check reflection

This MATLAB exercise taught the same research pattern in a second ecosystem. That is exactly the point. Publishable battery work often spans tools. Consistent habits across tools matter more than memorizing one syntax perfectly.

1.10 Open-Ended Exercises

These exercises ask you to modify the guided work rather than simply rerun it. Try them before reading the worked solutions.

1.10.1 Exercise 1

Modify the Python `hello_battery.py` script so it runs three discharges at 0.5C, 1C, and 2C for the same 2 Ah cell, then saves one combined CSV with a `c_rate` column.

Hints:

At 0.5C, the current should be 1 A. At 2C, the current should be 4 A. You will need a loop over current values and a way to concatenate several tables together.

1.10.2 Exercise 2

Extend `src/env_report.py` so it also records the SHA-256 hash of `environment.yml`.

Hints:

Use Python's `hashlib` module. The point is to capture not only installed-package versions but also the exact environment-spec file that was intended.

1.10.3 Exercise 3

Create a second log-template generator that makes entries named `YYYY-MM-DD_experiment.md` instead of `YYYY-MM-DD_setup.md`, and add a section called Planned Figure or Table.

Hints:

Reuse `src/create_log_entry.py` rather than writing from scratch. Change only what actually differs.

1.11 What Changes for Sodium-Ion?

At this early stage, the main sodium-ion adaptation is not a different equation set. It is a different standard of bookkeeping.

Sodium-ion work often depends on sparser public data, less standardized parameter releases, and more figure digitization than mainstream lithium-ion work. That means provenance matters even more. In a lithium-ion project, you may be able to rely on a well-known public dataset and a widely reused parameter set. In a sodium-ion project, you may need to record exactly which review paper you extracted an OCV shape from, which chemistry family the cell belongs to, whether the hard-carbon behavior reflects the slope region or plateau region, and how you converted a published current density into an absolute current for your assumed cell area or capacity.

So, from the very first chapter, get into the habit of tagging artifacts with chemistry context. A filename like `ocv_curve.csv` is weak. A filename like `hard_carbon_p2_naion_ocv_digitized_2026-04-14.csv` is stronger. A log entry that says “used SIB data” is weak. One that says “used digitized hard-carbon half-cell OCV from [paper], Figure 2b, units converted from mAh g⁻¹ to SOC using author-reported reversible capacity” is much stronger.

That discipline will matter more and more as the manual progresses.

1.12 Worked Solutions to the Open-Ended Exercises

1.12.1 Solution to Exercise 1

A clean way to solve the multi-C-rate problem is to wrap the discharge logic in a function and concatenate results.

```
from __future__ import annotations

from pathlib import Path

import numpy as np
import pandas as pd

Path("results").mkdir(parents=True, exist_ok=True)

nominal_capacity_ah = 2.0
ohmic_resistance_ohm = 0.05
initial_soc = 1.0
time_step_s = 1.0

c_rates = [0.5, 1.0, 2.0]
all_results = []

for c_rate in c_rates:
    discharge_current_a = c_rate * nominal_capacity_ah
    final_time_s = 3600.0 / c_rate
    time_s = np.arange(0.0, final_time_s + time_step_s, time_step_s)

    soc = initial_soc - (
        discharge_current_a * time_s / (3600.0 * nominal_capacity_ah)
    )
    soc = np.clip(soc, 0.0, 1.0)

    ocv_v = 3.0 + 0.6 * soc
```

```

terminal_voltage_v = ocv_v - discharge_current_a * ohmic_resistance_ohm

one_case = pd.DataFrame(
    {
        "c_rate": c_rate,
        "time_s": time_s,
        "soc_fraction": soc,
        "ocv_v": ocv_v,
        "terminal_voltage_v": terminal_voltage_v,
        "current_a": discharge_current_a,
    }
)
all_results.append(one_case)

combined_results = pd.concat(all_results, ignore_index=True)
combined_results.to_csv("results/hello_battery_multi_rate.csv", index=False)

print(combined_results.groupby("c_rate")["terminal_voltage_v"].agg(["first",
↪ "last"]))

```

The key idea is that current scales with C-rate through $I = C_{\text{rate}}Q_{\text{nom}}$, and full-discharge time scales inversely. The expected pattern is that higher C-rate gives a larger ohmic drop and shorter discharge time. With this teaching model, the 2C case starts 0.2 V below OCV because $4 \text{ A} \times 0.05 \Omega = 0.2 \text{ V}$.

1.12.2 Solution to Exercise 2

Here is a version of the environment report with a SHA-256 hash for environment.yml.

```

from __future__ import annotations

import hashlib
import json
import platform
import subprocess
import sys
from datetime import datetime, timezone
from importlib.metadata import PackageNotFoundError, version
from pathlib import Path

PACKAGES_TO_REPORT = [
    "numpy",
    "scipy",
    "pandas",
    "matplotlib",
    "jupyterlab",
    "ipykernel",
]

def package_version(package_name: str) -> str:
    try:
        return version(package_name)
    except PackageNotFoundError:
        return "NOT_INSTALLED"

```



```

def sha256_of_file(file_path: Path) -> str:
    return hashlib.sha256(file_path.read_bytes()).hexdigest()
def git_commit_hash() -> str:
    try:
        return subprocess.check_output(
            ["git", "rev-parse", "HEAD"],
            text=True,
        ).strip()
    except Exception:
        return "NOT_A_GIT_REPOSITORY"
def main() -> None:
    results_dir = Path("results")
    results_dir.mkdir(parents=True, exist_ok=True)

    environment_file = Path("environment.yml")

    report = {
        "generated_utc": datetime.now(timezone.utc).isoformat(),
        "python_executable": sys.executable,
        "python_version": platform.python_version(),
        "platform": platform.platform(),
        "git_commit": git_commit_hash(),
        "environment_yaml_sha256": sha256_of_file(environment_file)
        if environment_file.exists()
        else "FILE_NOT_FOUND",
        "packages": {
            package_name: package_version(package_name)
            for package_name in PACKAGES_TO_REPORT
        },
    }

    output_path = results_dir / "environment_report.json"
    output_path.write_text(json.dumps(report, indent=2), encoding="utf-8")
    print(json.dumps(report, indent=2))
if __name__ == "__main__":
    main()

```

The research value here is subtle but real. Two environments can sometimes report similar installed packages while differing in the intended spec file. Hashing `environment.yml` gives you an additional integrity check.

1.12.3 Solution to Exercise 3

A minimal adaptation of the log generator is enough.

```

from __future__ import annotations

from datetime import datetime
from pathlib import Path

TEMPLATE = """# Research Log: {date}

## Objective

```

Write one sentence describing today's concrete objective.

Planned Figure or Table

Describe the exact figure, table, or output artifact you intend to produce.

Environment

- Active Python environment:
- MATLAB version:
- Git commit:
- Dataset or paper in use:

Work Performed

Describe what you actually did.

Observations

Record numerical outputs, runtime notes, and figure behavior.

Decisions and Rationale

Explain why you made methodological choices.

Problems Encountered

Include exact failure modes and error messages where useful.

Next Steps

Write the next one to three actions.

"""

```
def main() -> None:
    logs_dir = Path("logs")
    logs_dir.mkdir(parents=True, exist_ok=True)

    date_string = datetime.now().strftime("%Y-%m-%d")
    output_path = logs_dir / f"{date_string}_experiment.md"

    if output_path.exists():
        print(f"Log entry already exists: {output_path}")
        return

    output_path.write_text(TEMPLATE.format(date=date_string), encoding="utf-8")
    print(f"Created experiment log entry: {output_path}")
if __name__ == "__main__":
    main()
```

This version is useful once the project shifts from environment setup into actual model runs and figure generation. The new Planned Figure or Table section encourages intentional work rather than random exploration.

1.13 Chapter Summary and Skill Checklist

You should now have these skills in working memory:

- create and activate an isolated Python environment
- verify package versions instead of assuming them
- register and select the correct Jupyter kernel
- verify MATLAB product availability for later battery workflows
- initialize a Git repository early
- write a research-appropriate `.gitignore`
- maintain a dated Markdown research log
- separate raw data, processed data, notebooks, reusable code, and generated outputs
- run a small battery-flavored sanity check in both Python and MATLAB
- save both machine-readable outputs and figure files

You should now be able to answer “yes” to all of the following:

- I can create a reproducible Python environment from a file rather than from memory.
- I can prove which interpreter and package versions generated an output.
- I have a repository structure that distinguishes source code from artifacts.
- I can explain why `data/raw/` and `data/processed/` must remain separate.
- I have a research-log habit that records decisions, not just successes.
- I can initialize Git, make meaningful commits, and avoid committing bulky raw data by accident.
- I can run and interpret a simple battery sanity check in both Python and MATLAB.
- I know whether my MATLAB installation includes the toolboxes needed for later chapters.

If any of those are still shaky, revisit the corresponding walkthrough now. Later chapters assume them.

1.14 Deliverable

The deliverable from your plan is a GitHub repository with a working environment, a research log template, and a passing sanity check in both Python and MATLAB, with a commit history showing at least ten meaningful commits.

Approach it as a mini research artifact, not a homework submission. Your repository should contain `environment.yml`, the scaffold scripts, the log template script, the `.gitignore`, both sanity-check scripts, the resulting CSV and PNG outputs, and a short README explaining how to rerun everything. Your Git history should tell the story of the setup in coherent steps rather than one giant dump commit.

A good completion state looks like this:

- `README.md` explains the project purpose and setup commands.
- `environment.yml` is present and matches the actual working environment.
- `results/environment_report.json` exists.
- `logs/YYYY-MM-DD_setup.md` contains real content, not placeholders.

- `results/hello_battery_python.csv` and `figures/hello_battery_python.png` exist.
- `results/hello_battery_matlab.csv` and `figures/hello_battery_matlab.png` exist.
- `git log --oneline -n 10` shows a sensible progression of work.

If you want a good final commit message for this chapter, use something like:

Complete chapter 1 research environment deliverable

Then add a final log entry summarizing what is verified, what remains uncertain, and what Chapter 2 will require.

1.15 Further Practice and Reading

Start with one or two reproducibility papers, one or two official docs, and one or two community resources you will actually revisit.

- Sandve, Nekrutenko, Taylor, and Hovig, “Ten Simple Rules for Reproducible Computational Research.” A concise paper worth internalizing early.
- Wilson et al., “Best Practices for Scientific Computing.” Older, still foundational, and directly relevant to the habits in this chapter.
- JupyterLab documentation: bookmark the user guide and kernel-management sections.
- Conda or Miniforge documentation: especially environment creation and export commands.
- MathWorks documentation for `ver`, MATLAB Projects, and later Simscape Battery.
- Pro Git by Scott Chacon and Ben Straub: still the cleanest Git reference for researchers.
- Community resources: Jupyter Discourse, Scientific Python Forum, MATLAB Central.

Lab Chapter 2: *Scientific Python Refresher for Battery Work* is next.

Chapter 2

Scientific Python Refresher for Battery Work

2.1 Chapter Opening

This chapter is about turning general scientific Python familiarity into battery-modeling fluency. By the time you finish it, you should no longer think of NumPy, SciPy, Pandas, and Matplotlib as four separate libraries you learned once in a methods course. You should think of them as the basic hand tools with which you express battery equations, solve dynamic models, inspect time-series data, fit parameters, and generate figures that can survive the transition from notebook output to paper draft. The immediate skill goal is modest on purpose: solve a battery-flavored ODE in Python with confidence. But the research goal underneath it is larger. We are building the numerical habits you will reuse in PyBaMM, in parameter-estimation pipelines, in dataset cleaning, and in every simulation-validation loop that follows.

This chapter operationalizes material you already learned in the theory textbook, especially Textbook Chapter 10 on equivalent-circuit models and state estimation. Keep that chapter open while you work. When we write the second-order Thevenin model as a system of differential equations, we are translating the circuit intuition of that chapter into executable code. Textbook Chapter 3 should also remain close at hand, because we will be careful about units, current sign conventions, capacity normalization, and the distinction between open-circuit voltage and terminal voltage. Later, when we talk about fitting parameters from voltage data, you should also remember the warnings from the textbook chapters on cell behavior and degradation: a model can fit a curve and still be physically misleading if you ask the data to identify parameters it cannot actually distinguish.

This chapter also builds directly on Lab Chapter 1. Chapter 1 gave you the environment and reproducibility habits that prevent your numerical work from becoming a pile of semi-runnable notebook fragments. Here we use that environment for real modeling work. We will reuse the repository layout, the logging habit, and the idea that every meaningful output should be reproducible from code and tied to a specific environment. If Chapter 1 was the lab bench, Chapter 2 is the moment we start using the instruments.

The pacing is intentional. We are not jumping into PyBaMM yet. That will happen in Chapter 3. Before we ask PyBaMM to discretize a Doyle-Fuller-Newman model, you need to be comfortable with the simpler but crucial layer underneath: arrays, state vectors, ODE solvers, least-squares fitting, time-indexed data cleaning, and plot construction. Those are not beginner topics in disguise. They are the skeleton underneath almost every publishable battery workflow. If you can write and trust a clean second-order RC model today, you will understand PyBaMM objects better in Chapter 3, design cleaner experiments in Chapter 4, fit parameters more honestly in Chapter 5, and move into MATLAB ECM work in Chapter 6 without feeling as if the math and the code live in separate universes.

There is one more reason this chapter matters for your specific goal of publishable sodium-ion research. Sodium-ion work often begins in a data-sparse environment. You will frequently be adapting lithium-ion methods, public datasets, and model structures to sodium-ion chemistry. That means you need to distinguish the durable computational skill from the chemistry-specific assumption. Solving an ODE, resampling a time series, or fitting a parameter vector are durable skills. The exact OCV curve, voltage window, and identifiability behavior are chemistry-specific. This chapter helps you separate those layers cleanly, which is exactly what you need if your long-term target is simulation-based sodium-ion work built on partial public data.

So our pattern for the chapter is simple. First, we bridge the theory of equivalent-circuit models to their numerical representation. Then we refresh the NumPy patterns that battery work uses constantly. After that, we solve a second-order RC model with `scipy.integrate.solve_ivp`, compare solver behavior, and talk honestly about what solver choice means. Next, we shift from synthetic signals to a public experimental dataset so that Pandas and Matplotlib are grounded in real battery traces rather than toy CSV files. Then we fit a model back to noisy data using `scipy.optimize.least_squares`. Finally, because Part I of this companion must already train you in reproduction practice, we will reproduce a published time-series figure from an open paper using the paper's linked public dataset. That last exercise is where the chapter becomes more than a refresher. It becomes research practice.

2.2 Prerequisites Check

- Required software: the `sib-research` environment from Lab Chapter 1; Python 3.11; JupyterLab 4.4+; Git 2.43+
- Required Python packages: `numpy==2.3.4`, `scipy==1.16.0`, `pandas==3.0.2`, `matplotlib==3.10.8`, `jupyterlab==4.4.10`, `ipykernel==7.2.0`
- Optional but recommended: a code editor with notebook support, and a PDF viewer so you can keep the chapter and the cited paper open together
- Required textbook chapters: Textbook Chapter 3 and Textbook Chapter 10 are essential; the chapter becomes easier if Textbook Chapter 7 is also familiar conceptually
- Required prior lab chapters: Lab Chapter 1
- Estimated time: 10 to 14 hours, depending on how quickly you move through the fitting and reproduction sections

If any of these feel shaky, stop now instead of pushing forward half-prepared. If NumPy broadcasting still feels magical rather than understandable, the early sections of this chapter are especially important. If your Chapter 1 environment is not reproducible yet, fix that first. We will assume from this point onward that you can create a notebook, run a script from the repository root, and save outputs into `results/` and `figures/` without confusion.

2.3 Environment Setup

This chapter does not require a brand-new environment, but it does require confidence that the Chapter 1 environment is the one you are actually using. The most common failure mode in numerical methods chapters is not a wrong equation. It is running the right code in the wrong interpreter.

2.3.1 Activate the Chapter 1 environment

If you used `conda` in Chapter 1:

```
conda activate sib-research
```

If you used `venv` on Linux or macOS:

```
source .venv/bin/activate
```

If you used `venv` on Windows PowerShell:

```
.venv\Scripts\Activate.ps1
```

2.3.2 Verify the scientific stack

Run this from the repository root:

```
python - <<<'PY'
import numpy as np
import scipy
import pandas as pd
import matplotlib

print("NumPy      :", np.__version__)
print("SciPy      :", scipy.__version__)
print("Pandas     :", pd.__version__)
print("Matplotlib:", matplotlib.__version__)
PY
```

Expected output should look like:

```
NumPy      : 2.3.4
SciPy      : 1.16.0
Pandas     : 3.0.2
Matplotlib: 3.10.8
```

Patch-version drift is not automatically a disaster, but if your versions differ from the pinned Chapter 1 environment, record that in your research log before continuing.

2.3.3 Create a notebook and script location for this chapter

We will keep the usual pattern: exploratory work in `notebooks/`, reusable helpers in `src/`, and generated outputs in `results/` and `figures/`.

```
mkdir -p notebooks src results figures data/raw data/processed
```

Create a notebook called `notebooks/chapter_2_scientific_python_refresher.ipynb`. If you prefer to begin in a script, that is fine, but the notebook format is useful here because we will inspect intermediate arrays and plots frequently.

2.3.4 Minimal `solve_ivp` hello world

Before we solve a battery model, verify that SciPy's ODE solver is functioning.

```
import numpy as np
from scipy.integrate import solve_ivp

def exponential_decay_rhs(time_s, state):
    return [-0.5 * state[0]]

evaluation_times_s = np.linspace(0.0, 10.0, 6)

solution = solve_ivp(
    fun=exponential_decay_rhs,
    t_span=(0.0, 10.0),
    y0=[1.0],
    t_eval=evaluation_times_s,
)

print("status:", solution.status)
print("message:", solution.message)
print("y(t):", np.round(solution.y[0], 4))
```

Expected output:

```
status: 0
message: The solver successfully reached the end of the integration interval.
y(t): [1.      0.3679 0.1353 0.0498 0.0183 0.0067]
```

Those values follow the exact analytical solution $y(t) = e^{-0.5t}$. If your numbers are close to the ones above, your solver is working.

2.3.5 Launch JupyterLab and confirm the kernel

```
jupyter lab
```

Open the notebook you created and check the kernel name in the top-right corner. It should still be Python (sib-research) or the Chapter 1 equivalent. Then run:

```
import sys
print(sys.executable)
```

The printed path should point into your intended environment. If it points to a system interpreter, stop and fix the kernel mismatch now.

2.3.6 Common setup failures and fixes

1. `ModuleNotFoundError: No module named 'scipy'`

Symptom: imports fail inside Jupyter but succeed in the terminal, or vice versa.

Fix: you almost certainly have a kernel mismatch. Re-register the kernel from

the active environment with `python -m ipykernel install --user --name sib-research --display-name "Python (sib-research)"`.

2. `ImportError` mentioning a compiled library or wheel incompatibility
 Symptom: NumPy or SciPy imports fail even though installation appeared to work.
 Fix: this is usually an environment inconsistency. Recreate the environment from `environment.yml` rather than trying to patch it package-by-package.
3. Plots do not display in the notebook
 Symptom: code runs but no figure appears.
 Fix: ensure the notebook cell actually ends with a plotting command or `plt.show()`, and verify that you did not disable the inline backend accidentally.
4. `solve_ivp` works for the hello-world example but later battery code fails with shape errors
 Symptom: SciPy complains about incompatible dimensions or unpacking errors.
 Fix: the most common cause is forgetting that the state passed into the right-hand side is an array, even for scalar problems. We will be very explicit about state-vector shape in the next section.

2.4 Conceptual Bridge: From the Equivalent Circuit in the Textbook to a State Vector in Python

In Textbook Chapter 10, the equivalent-circuit model was valuable because it compressed a complicated electrochemical reality into a small number of states and parameters you could reason about directly. A resistor gave you the instantaneous ohmic drop. One or more RC branches gave you delayed polarization behavior. SOC evolved through charge balance. The model was not “true” in the same way a physics-based porous-electrode model aims to be true, but it was useful because it captured the right dynamic shape at the right computational cost.

The most important translation we make in this chapter is that from circuit picture to state-space representation. In the textbook, you probably drew the second-order Thevenin model as an OCV source in series with R_0 and two parallel RC branches. In Python, that same model becomes a state vector and a right-hand-side function. If discharge current is positive, one convenient form is

$$\frac{dz}{dt} = -\frac{I(t)}{3600Q_n}, \quad (1)$$

$$\frac{dv_1}{dt} = -\frac{v_1}{R_1C_1} + \frac{I(t)}{C_1}, \quad \frac{dv_2}{dt} = -\frac{v_2}{R_2C_2} + \frac{I(t)}{C_2}, \quad (2)$$

and

$$V_t(t) = \text{OCV}(z(t)) - I(t)R_0 - v_1(t) - v_2(t). \quad (3)$$

Equation (1) says SOC is just charge balance written in differential form. Equation (2) says each RC branch stores a transient overpotential with its own time constant

$\tau_i = R_i C_i$. Equation (3) says terminal voltage is open-circuit voltage minus the ohmic drop and minus the transient branch drops. Nothing here is new physically. What changes is the representation. Instead of solving the circuit by hand for one simple input, we hand the whole system to an ODE solver and let it march through time.

This is exactly the moment where many readers conceptually split in two. One group thinks, “Good, now we are doing code.” The other thinks, “Good, now we are still doing the same battery model, just in executable form.” The second attitude is the right one. The Python objects are not a separate subject. They are the mechanism by which the same model becomes testable, sweepable, fit-able, and eventually publishable.

NumPy enters first because state-space battery work is almost never scalar work. Even if the state vector is only three elements long, the surrounding workflow is array-shaped. You evaluate OCV at hundreds of SOC points. You compare multiple current levels on the same grid. You slice discharge windows out of a long record. You compute residual vectors between measured and simulated voltage samples. Vectorization is not a performance trick added at the end. It is the natural language of the problem.

SciPy’s `solve_ivp` is the next layer. In textbook notation, the model is a set of coupled first-order ODEs. In SciPy notation, it is a Python function with signature `rhs(t, y, ...)` that returns the time derivative of the state vector. The ODE solver repeatedly calls that function, chooses time steps, and constructs a numerical approximation to the trajectory. That sounds mechanical, but several research decisions hide inside it. Which solver should you choose? How tight should the tolerances be? Is the problem stiff? Do you want the solution only at a few evaluation times or on an adaptive grid? Those questions do not disappear just because the code runs. They become part of the modeling method.

`scipy.optimize.least_squares` extends the same logic into inverse problems. In forward simulation, we supply parameters and predict voltage. In fitting, we reverse the direction: we adjust parameters so that simulated voltage resembles measured voltage as closely as possible. Mathematically, we define a residual vector such as

$$r_i(\theta) = V_{\text{sim},i}(\theta) - V_{\text{meas},i}, \quad (4)$$

and we ask the optimizer to reduce the norm of that vector over parameter vector θ . Again, the software object and the battery method are the same idea seen from two angles. A parameter fit is just a repeated forward simulation wrapped in an optimizer.

Pandas then plays a complementary role. The textbook equivalent-circuit model is mathematically clean, but real battery datasets are not. Public files contain missing values, inconsistent headings, text legends, different sign conventions, and sometimes incomplete metadata. Pandas is the structure we use to make those data problems explicit rather than informal. A `DataFrame` is where units become column names, timestamps become indices, resampling becomes auditable, and segment labels become filter conditions. That matters because later, when you do publishable work, a large fraction of your time will not be spent solving equations. It will be spent making sure the data you feed into those equations are what you think they are.

Matplotlib is the last piece of the bridge. In theory, a plotted curve is evidence. In practice, plotting is part of analysis, not just presentation. A good plot tells you whether

a solver stepped over a sharp transient, whether a fit is matching only the slow drift and missing the pulse edges, whether your sign convention is backwards, and whether one current trace is actually being plotted on the wrong axis. Publication-quality plotting begins long before publication. It begins when your diagnostic plots are precise enough that you can trust them.

So the bridge from textbook theory to scientific Python is not a change in subject. It is a change in representation. The circuit becomes a state vector, the governing equations become a right-hand-side function, the dataset becomes a `DataFrame`, the calibration becomes a least-squares problem, and the interpretation becomes a figure you can read critically. Keep that translation in mind through every exercise in this chapter. If the code ever starts feeling like disconnected syntax, come back to Equations (1) through (4) and ask which mathematical object each line is implementing.

2.5 Guided Walkthrough 1: NumPy Patterns You Will Use Constantly

Learning objective: Use vectorization, broadcasting, and slicing to evaluate battery voltage over many SOC and current combinations without writing nested Python loops.

Before we solve any ODEs, we need to refresh the array operations that battery work uses all the time. The right way to think about vectorization here is not “faster code” but “clearer statement of the physics.” If you want terminal voltage for four current levels over an SOC grid, that is a 2D array problem. Writing it as a 2D array clarifies the structure of the question.

2.5.1 Walkthrough 1 code

```
import numpy as np
import matplotlib.pyplot as plt

def teaching_ocv_from_soc(soc_fraction):
    """Simple smooth OCV curve with low-SOC and high-SOC knees."""
    soc_fraction = np.clip(np.asarray(soc_fraction), 0.0, 1.0)
    return (
        2.85
        + 0.42 * soc_fraction
        + 0.06 * np.tanh((soc_fraction - 0.10) / 0.04)
        + 0.10 * np.tanh((soc_fraction - 0.88) / 0.05)
    )

soc_grid = np.linspace(0.02, 0.98, 241)
discharge_currents_a = np.array([1.0, 2.0, 4.0, 6.0])[:, None]
r0_ohm = 0.018

ocv_grid_v = teaching_ocv_from_soc(soc_grid)
terminal_voltage_grid_v = ocv_grid_v - discharge_currents_a * r0_ohm
```

```

mid_soc_slice = slice(80, 161)
mid_soc_soc = soc_grid[mid_soc_slice]
mid_soc_voltage_grid_v = terminal_voltage_grid_v[:, mid_soc_slice]

print("soc_grid shape:", soc_grid.shape)
print("discharge_currents_a shape:", discharge_currents_a.shape)
print("terminal_voltage_grid_v shape:", terminal_voltage_grid_v.shape)
print("Voltage at 50% SOC for each current level [V]:")
soc_50_index = np.argmin(np.abs(soc_grid - 0.50))
print(np.round(terminal_voltage_grid_v[:, soc_50_index], 4))

fig, ax = plt.subplots(figsize=(8, 5), constrained_layout=True)

for current_a, voltage_curve_v in zip(discharge_currents_a[:, 0],
    ↪ terminal_voltage_grid_v):
    ax.plot(
        soc_grid,
        voltage_curve_v,
        linewidth=2.0,
        label=f"{current_a:.0f} A discharge",
    )

ax.set_xlabel("State of charge, SOC [-]")
ax.set_ylabel("Terminal voltage [V]")
ax.set_title("Broadcasted voltage calculation over SOC and discharge current")
ax.grid(True, alpha=0.3)
ax.legend()

plt.show()

```

2.5.2 Walkthrough 1 code explanation

The `teaching_ocv_from_soc` function is not intended as a chemistry-specific OCV model. It is a smooth teaching function that mimics three features you expect in a real OCV curve: a broad central region, a low-SOC knee, and a high-SOC knee. The `np.tanh` terms are especially convenient for teaching because they create smooth transitions without any piecewise discontinuities. Later chapters will replace this kind of teaching function with tabulated or experimentally fitted OCV relations, but for now the smoothness is helpful because it keeps numerical behavior easy to interpret.

`soc_grid = np.linspace(0.02, 0.98, 241)` creates a one-dimensional array of SOC values. We deliberately stop short of exactly 0 and 1 because many battery functions behave awkwardly at the hard edges of the allowable range. In publishable work, you often clip away the extreme ends for similar reasons.

`discharge_currents_a = np.array([1.0, 2.0, 4.0, 6.0])[:, None]` is the line most worth pausing on. The values start as a one-dimensional array of shape `(4,)`. The `[:, None]` adds a new axis, turning it into a column vector of shape `(4, 1)`. That one extra axis is what allows broadcasting to work cleanly against the SOC grid of shape `(241,)`.

`ocv_grid_v = teaching_ocv_from_soc(soc_grid)` returns a one-dimensional voltage array of length 241. When we write `ocv_grid_v - discharge_currents_a *`

`r0_ohm`, NumPy broadcasts the (241,) OCV array against the (4, 1) current array and produces a (4, 241) terminal-voltage array. That is exactly what we want: one voltage curve for each current level, evaluated at every SOC point.

The `mid_soc_slice = slice(80, 161)` line is here because slicing is a daily battery-analysis operation, not a beginner curiosity. You will constantly isolate windows: the middle SOC range, the first 60 seconds of a pulse, the rest period after a current step, or the final 10% of a charge. Writing a slice object once and reusing it is clearer than sprinkling index literals throughout the code.

`soc_50_index = np.argmax(np.abs(soc_grid - 0.50))` finds the grid point closest to 50% SOC. In research code you should prefer explicit selection logic like this over assuming that a nice round SOC value is exactly present in a floating-point grid.

In the plot loop, the `zip(discharge_currents_a[:, 0], terminal_voltage_grid_v)` pairing works because `discharge_currents_a[:, 0]` turns the column vector back into a flat list of current values for labeling, while `terminal_voltage_grid_v` iterates row-by-row over the broadcast result.

2.5.3 Walkthrough 1 expected output

The terminal printout should look like this in structure:

```
soc_grid shape: (241,)
discharge_currents_a shape: (4, 1)
terminal_voltage_grid_v shape: (4, 241)
Voltage at 50% SOC for each current level [V]:
[3.1131 3.0951 3.0591 3.0231]
```

The exact voltages may differ by a few last digits depending on the SOC grid point closest to 0.50, but the pattern must be monotonic: higher discharge current produces lower terminal voltage at the same SOC because the ohmic drop grows as IR_0 .

The plot should show four smooth curves of terminal voltage versus SOC. All four curves should have the same shape because only the ohmic term changes in this simple example; the higher-current curves should be vertically shifted downward by a constant amount relative to the lower-current curves. The low-SOC and high-SOC knees should be visible. A wrong result usually looks like one of three things: curves crossing when they should not, a jagged line caused by accidental integer indexing, or only one curve appearing because broadcasting collapsed incorrectly.

2.5.4 Walkthrough 1 troubleshooting

1. `ValueError` about incompatible shapes

Symptom: operands could not be broadcast together.

Fix: make sure the current array has shape `(n_currents, 1)` rather than `(n_currents,)`. The `[:, None]` is the key.

2. All four curves lie on top of each other

Symptom: the plot shows only one apparent curve.

Fix: verify that `r0_ohm` is not accidentally zero and that you did not overwrite `discharge_currents_a` with a scalar.

3. The voltage increases with current

Symptom: higher discharge current produces a higher terminal voltage.

Fix: your sign convention is reversed somewhere. In this chapter, discharge current is positive and subtracts voltage.

2.5.5 Walkthrough 1 reflection

This exercise taught the NumPy shape habits we will keep reusing: column vectors for parameter sweeps, one-dimensional grids for state variables, slicing for physically meaningful windows, and explicit attention to units and sign convention. If this felt almost too simple, that is good. These are the habits that make later model code readable instead of mysterious.

2.6 Guided Walkthrough 2: Solve a Second-Order RC Model with `solve_ivp`

Learning objective: Implement the second-order Thevenin model from Textbook Chapter 10 as an ODE system, solve it under a pulse-rest protocol, and compare solver behavior.

This is the chapter’s core numerical exercise. We will move from an algebraic battery approximation to a dynamic model with internal states. The model is still small enough to understand line-by-line, which is exactly why it is such a good bridge to later chapters.

2.6.1 Solver choice before code

You will see three SciPy solvers in this chapter: RK45, BDF, and LSODA. Here is the practical distinction you should remember.

Solver	Numerical family	Good default use	Most common failure mode in battery work
RK45	explicit Runge-Kutta	nonstiff problems; first checks; teaching examples	many tiny steps when dynamics span very different time constants
BDF	implicit multistep	stiff or mildly stiff problems; later PyBaMM-like behavior	extra overhead when the problem is easy
LSODA	automatic stiffness switching	when you do not know if the problem is stiff	harder to diagnose solver behavior because the switching is internal

The two RC time constants in this exercise are intentionally separated enough that stiffness can begin to matter, but not so extreme that the model becomes numerically fragile.

2.6.2 Walkthrough 2 code

```

from dataclasses import dataclass
from time import perf_counter

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

@dataclass(frozen=True)
class TheveninParameters:
    capacity_ah: float
    r0_ohm: float
    r1_ohm: float
    c1_f: float
    r2_ohm: float
    c2_f: float

def teaching_ocv_from_soc(soc_fraction):
    soc_fraction = np.clip(np.asarray(soc_fraction), 0.0, 1.0)
    return (
        2.85
        + 0.42 * soc_fraction
        + 0.06 * np.tanh((soc_fraction - 0.10) / 0.04)
        + 0.10 * np.tanh((soc_fraction - 0.88) / 0.05)
    )

PARAMS = TheveninParameters(
    capacity_ah=3.0,
    r0_ohm=0.015,
    r1_ohm=0.010,
    c1_f=1800.0,
    r2_ohm=0.008,
    c2_f=18000.0,
)

segment_start_times_s = np.array([0, 300, 900, 1200, 1500, 1800, 2400, 3000],
    ↪ dtype=float)
segment_currents_a = np.array([0.0, 3.0, 0.0, 6.0, 0.0, 4.5, 0.0, 1.5],
    ↪ dtype=float)
simulation_end_time_s = 3600.0

def current_profile_a(time_s):
    segment_index = np.searchsorted(segment_start_times_s, time_s,
    ↪ side="right") - 1
    segment_index = np.clip(segment_index, 0, len(segment_currents_a) - 1)
    return float(segment_currents_a[segment_index])

```

```

def thevenin_rhs(time_s, state, params):
    soc_fraction, v_rc1_v, v_rc2_v = state
    current_a = current_profile_a(time_s)

    dsoc_dt = -current_a / (3600.0 * params.capacity_ah)
    dv_rc1_dt = -(v_rc1_v / (params.r1_ohm * params.c1_f)) + current_a /
↪ params.c1_f
    dv_rc2_dt = -(v_rc2_v / (params.r2_ohm * params.c2_f)) + current_a /
↪ params.c2_f

    return [dsoc_dt, dv_rc1_dt, dv_rc2_dt]

def simulate_model(method_name):
    evaluation_times_s = np.linspace(0.0, simulation_end_time_s, 1201)
    initial_state = [0.95, 0.0, 0.0]

    start_clock = perf_counter()

    solution = solve_ivp(
        fun=thevenin_rhs,
        t_span=(0.0, simulation_end_time_s),
        y0=initial_state,
        t_eval=evaluation_times_s,
        method=method_name,
        args=(PARAMS,),
        rtol=1e-8,
        atol=1e-10,
    )

    elapsed_wall_time_s = perf_counter() - start_clock

    current_trace_a = np.array([current_profile_a(t) for t in solution.t])
    soc_trace = solution.y[0]
    v_rc1_trace_v = solution.y[1]
    v_rc2_trace_v = solution.y[2]
    terminal_voltage_v = (
        teaching_ocv_from_soc(soc_trace)
        - current_trace_a * PARAMS.r0_ohm
        - v_rc1_trace_v
        - v_rc2_trace_v
    )

    output = pd.DataFrame(
        {
            "time_s": solution.t,
            "current_a": current_trace_a,
            "soc_fraction": soc_trace,
            "v_rc1_v": v_rc1_trace_v,
            "v_rc2_v": v_rc2_trace_v,
            "terminal_voltage_v": terminal_voltage_v,
        }
    )

```



```

)

diagnostics = {
    "method": method_name,
    "status": solution.status,
    "nfev": solution.nfev,
    "wall_time_s": elapsed_wall_time_s,
    "final_soc": soc_trace[-1],
    "final_voltage_v": terminal_voltage_v[-1],
}

return output, diagnostics

all_outputs = {}
all_diagnostics = []

for solver_name in ["RK45", "BDF", "LSODA"]:
    simulation_output, diagnostics = simulate_model(solver_name)
    all_outputs[solver_name] = simulation_output
    all_diagnostics.append(diagnostics)

diagnostics_table = pd.DataFrame(all_diagnostics)
print(diagnostics_table.round(6))

reference_voltage_v = all_outputs["BDF"]["terminal_voltage_v"].to_numpy()

for solver_name, simulation_output in all_outputs.items():
    max_abs_difference_v = np.max(
        np.abs(simulation_output["terminal_voltage_v"].to_numpy() -
        ↪ reference_voltage_v)
    )
    print(f"{solver_name:>5s} max |V - V_BDF| = {max_abs_difference_v:.6e} V")

fig, axes = plt.subplots(3, 1, figsize=(9, 9), sharex=True,
    ↪ constrained_layout=True)

for solver_name, simulation_output in all_outputs.items():
    axes[0].plot(
        simulation_output["time_s"] / 60.0,
        simulation_output["terminal_voltage_v"],
        linewidth=2.0,
        label=solver_name,
    )
    axes[1].plot(
        simulation_output["time_s"] / 60.0,
        simulation_output["soc_fraction"],
        linewidth=2.0,
        label=solver_name,
    )

```

```

bdf_output = all_outputs["BDF"]
axes[2].step(
    bdf_output["time_s"] / 60.0,
    bdf_output["current_a"],
    where="post",
    color="black",
    linewidth=1.8,
)

axes[0].set_ylabel("Terminal voltage [V]")
axes[1].set_ylabel("SOC [-]")
axes[2].set_ylabel("Current [A]")
axes[2].set_xlabel("Time [min]")

axes[0].set_title("Second-order Thevenin response under a pulse-rest protocol")
axes[0].grid(True, alpha=0.3)
axes[1].grid(True, alpha=0.3)
axes[2].grid(True, alpha=0.3)
axes[0].legend()

plt.show()

```

2.6.3 Walkthrough 2 code explanation

The `@dataclass(frozen=True)` definition is a clean way to package the model parameters. We use a dataclass here not because it is fashionable, but because it makes parameter names explicit and prevents accidental in-place mutation. In fitting work later, we will sometimes create modified parameter sets. Freezing the dataclass helps catch unintended edits.

The current profile is represented by `segment_start_times_s` and `segment_currents_a`. This is a standard pattern for piecewise-constant battery protocols. Each entry in `segment_currents_a` applies from its start time until the next listed start time. The sequence here is rest, 3 A pulse, rest, 6 A pulse, rest, 4.5 A pulse, rest, then a gentle 1.5 A discharge to the end. That protocol is pedagogically useful because it excites both fast and slow RC dynamics.

`current_profile_a` uses `np.searchsorted` to determine which time segment contains the current solver time. The crucial detail is `side="right"` combined with subtracting one. That means a time exactly equal to a segment boundary belongs to the new segment, which is what we want for a right-continuous step protocol.

Inside `thevenin_rhs`, the state vector is unpacked into `soc_fraction`, `v_rc1_v`, and `v_rc2_v`. This is the code representation of Equations (1) and (2). Notice how compact it becomes once the physics is already understood. The SOC equation divides current by `3600 * capacity_ah` because one amp-hour is 3600 coulombs. That factor is easy to forget, and forgetting it is one of the fastest ways to get obviously wrong SOC trajectories.

The RC branch equations each have a decay term and a forcing term. The decay term is $-v_{rc} / (R C)$, which tells the branch overpotential to relax to zero in the absence of current. The forcing term is I / C , which drives the branch state during current flow.

`simulate_model` exists so we can rerun the same model cleanly with different solver methods. Reusable wrappers like this are important. If the solver configuration is embedded inline in three separate notebook cells, comparison becomes much harder to trust.

The solver tolerances `rtol=1e-8` and `atol=1e-10` are tighter than you would need for a quick exploratory sketch, but that is intentional. We want solver-to-solver differences to reflect the method, not a loose tolerance choice. Later, when computational cost matters more, you will often loosen these tolerances.

We compute terminal voltage after solving rather than treating it as an additional differential state. That is correct because voltage here is an algebraic output of the state, not a state that needs its own ODE.

The `diagnostics_table` prints `status`, `nfev`, wall-clock time, final SOC, and final voltage. In model development, diagnostics like this should be routine. Otherwise you can switch solvers and feel productive without actually learning anything about the numerical behavior.

2.6.4 Walkthrough 2 expected output

The diagnostics table should have three rows, one per solver. The exact wall times depend on your machine, but the qualitative pattern should look like this:

	method	status	nfev	wall_time_s	final_soc	final_voltage_v
0	RK45	0	...	...	0.400...	2.98...
1	BDF	0	...	...	0.400...	2.98...
2	LSODA	0	...	...	0.400...	2.98...

All three solvers should finish successfully with `status = 0`. The final SOC should be about 0.40 because the total discharged ampere-hours in the profile are substantial but not enough to empty the 3 Ah cell. The final terminal voltage should be just under 3.0 V for the teaching OCV curve and parameter set used here.

The printed maximum voltage differences relative to BDF should usually be small, often in the microvolt-to-submillivolt range for this problem when using tight tolerances. If you see differences of several tens of millivolts, something is wrong.

The figure should have three stacked panels. The top panel shows terminal voltage, with visible downward steps at current pulses and smooth recovery during rest intervals. The middle panel shows SOC declining only during discharge segments and flattening during rest. The bottom panel shows the pulse-rest current schedule as a step plot. A correct voltage plot has both instantaneous drops and slower relaxations. If you see only sharp jumps with no recovery curvature, you have probably lost the RC dynamics. If you see SOC changing during rest, your current profile function is wrong.

2.6.5 Walkthrough 2 troubleshooting

1. SOC goes negative or above 1

Symptom: the SOC trace leaves the physically valid interval.

Fix: check the current sign convention and the capacity conversion factor of 3600.

2. Voltage rises during discharge pulses

Symptom: applying positive discharge current makes the terminal voltage jump upward.

Fix: the sign on the ohmic or RC terms is reversed.

3. The solver raises a dimension or unpacking error

Symptom: messages about too many or too few values to unpack.

Fix: verify that `y0` has three entries and the right-hand side returns three derivatives.

4. RK45 appears much slower than the others

Symptom: large `nfev` or noticeably longer runtime.

Fix: that can be real. The separated RC time constants make the problem mildly stiff. Record the observation rather than assuming it is a bug.

2.6.6 Walkthrough 2 reflection

This exercise taught the basic forward-model pattern we will reuse all through the manual: define parameters, define an input protocol, write a right-hand side, solve the ODE, compute outputs, and inspect diagnostics. Chapter 5 will wrap this same pattern inside an optimizer. Chapter 6 will rebuild a similar model in MATLAB. Chapter 3 will replace this hand-built ODE with a much larger PyBaMM model, but the logic will still feel familiar if you understand this exercise well.

2.7 Dataset Integration: A Public Battery Time-Series File

This chapter's real-data anchor is the public dataset released alongside Ana Foles and co-authors' open paper on lithium-ion battery pack modeling for stationary energy management. We will use it for Pandas practice and for the Part I reproduction exercise.

Item	Value
Dataset	Lithium_Ion_Battery_Testing_Data.csv
Landing page	https://doi.org/10.5281/zenodo.5196334
Direct download	https://zenodo.org/records/5196334/files/Lithium_Ion_Battery_Testing_Data.csv
Size	about 2.9 MB
Format	CSV
License	CC BY 4.0
Paper using the data	Foles et al., <i>Open Research Europe</i> (2022), DOI 10.12688/openreseurope.14301.2

The paper describes the file as containing charge and discharge acquisition data at constant power levels, including current, voltage, SOC, ambient temperature, and a final legend column named BI that explains the segment or operating condition. This is a good teaching dataset because it is small enough to inspect comfortably but real enough to remind you that public battery data never arrive in the exact shape your analysis wants.

Three common pitfalls matter before we touch the code.

First, current sign convention is never to be assumed. Some datasets treat discharge current as positive, some as negative, and some switch conventions between instruments. You should not infer the sign convention from wishful thinking. You should infer it from how voltage and SOC move together.

Second, time columns are often messy. Some files store elapsed seconds, some absolute timestamps, some MATLAB datenums, and some only preserve row order. Our parsing code will therefore detect time heuristically rather than assuming one exact header.

Third, text metadata matter. The BI legend column may look ancillary, but it is the column that lets us isolate constant-power segments for plotting and reproduction. Treat text columns in battery datasets as part of the method, not as decoration.

2.8 Guided Walkthrough 3: Parse, Clean, and Plot a Public Battery Dataset with Pandas and Matplotlib

Learning objective: Download a real battery CSV, normalize its column names, infer a usable time axis, resample a trace, and produce a publication-quality diagnostic plot.

We are now leaving the world of fully controlled synthetic signals. That means the code will get longer, but the length is doing real work. Every explicit cleaning step is a future argument you will not need to have with yourself about what the data actually mean.

2.8.1 Walkthrough 3 code

```
from pathlib import Path
from urllib.request import urlretrieve

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

DATA_URL = (
    "https://zenodo.org/records/5196334/files/"
    "Lithium_Ion_Battery_Testing_Data.csv?download=1"
)
RAW_PATH = Path("data/raw/Lithium_Ion_Battery_Testing_Data.csv")

def canonicalize_text(text):
    text = str(text).strip().lower()
    replacements = {
        " ": "_",
        "-": "_",
        "/": "_per_",
        "(": "",
        ")": "",
        "[": "",
```

```

        "]" : "",
        "%" : "pct",
    }
    for old, new in replacements.items():
        text = text.replace(old, new)
    while "__" in text:
        text = text.replace("__", "_")
    return text.strip("_")

def download_if_needed():
    RAW_PATH.parent.mkdir(parents=True, exist_ok=True)
    if not RAW_PATH.exists():
        print(f"Downloading dataset to {RAW_PATH}")
        urlretrieve(DATA_URL, RAW_PATH)
    else:
        print(f"Using existing file at {RAW_PATH}")

def normalize_columns(dataframe):
    dataframe = dataframe.copy()
    dataframe.columns = [canonicalize_text(column) for column in
↪ dataframe.columns]

    rename_map = {}
    for column in dataframe.columns:
        if column == "bi":
            rename_map[column] = "segment_label"
        elif "current" in column and "charge" not in column:
            rename_map[column] = "current_a"
        elif "voltage" in column and "charge" not in column:
            rename_map[column] = "voltage_v"
        elif "soc" in column:
            rename_map[column] = "soc_pct"
        elif column in {"ta", "ambient_temperature", "ambient_temperature_c"}:
            rename_map[column] = "ambient_temperature_c"
        elif "time" in column and "temperature" not in column:
            rename_map[column] = "time_raw"

    return dataframe.rename(columns=rename_map)

def build_elapsed_time_seconds(dataframe):
    dataframe = dataframe.copy()

    if "time_raw" not in dataframe.columns:
        dataframe["elapsed_s"] = np.arange(len(dataframe), dtype=float)
        time_origin_description = "No explicit time column detected; using row
↪ index as pseudo-time."
        return dataframe, time_origin_description

    numeric_time = pd.to_numeric(dataframe["time_raw"], errors="coerce")
    if numeric_time.notna().mean() > 0.95:

```

```

        dataframe["elapsed_s"] = numeric_time - numeric_time.iloc[0]
        time_origin_description = "Detected numeric time column and shifted it
↪ to start at zero."
        return dataframe, time_origin_description

    datetime_time = pd.to_datetime(dataframe["time_raw"], errors="coerce")
    if datetime_time.notna().mean() > 0.95:
        dataframe["elapsed_s"] = (datetime_time -
↪ datetime_time.iloc[0]).dt.total_seconds()
        time_origin_description = "Detected datetime-like time column and
↪ converted to elapsed seconds."
        return dataframe, time_origin_description

    dataframe["elapsed_s"] = np.arange(len(dataframe), dtype=float)
    time_origin_description = (
        "Time column could not be parsed reliably; using row index as
↪ pseudo-time."
    )
    return dataframe, time_origin_description

download_if_needed()
raw_df = pd.read_csv(RAW_PATH)
clean_df = normalize_columns(raw_df)
clean_df, time_note = build_elapsed_time_seconds(clean_df)

for column in ["current_a", "voltage_v", "soc_pct", "ambient_temperature_c",
↪ "elapsed_s"]:
    if column in clean_df.columns:
        clean_df[column] = pd.to_numeric(clean_df[column], errors="coerce")

clean_df = clean_df.dropna(subset=["current_a", "voltage_v",
↪ "elapsed_s"]).copy()
clean_df = clean_df.sort_values("elapsed_s")
clean_df = clean_df.drop_duplicates(subset=["elapsed_s"])

time_index = pd.to_timedelta(clean_df["elapsed_s"], unit="s")
time_indexed_df = clean_df.set_index(time_index)

resampled_df = (
    time_indexed_df[["current_a", "voltage_v"]]
    .resample("10s")
    .mean()
    .interpolate(method="time")
)

print("Normalized columns:")
print(clean_df.columns.tolist())
print()
print("Time parsing note:")
print(time_note)
print()
print("First five cleaned rows:")

```

```

print(clean_df.head())
print()
print("Missing-value fraction after cleaning:")
print(clean_df[["current_a", "voltage_v"]].isna().mean())

fig, axes = plt.subplots(2, 1, figsize=(10, 7), sharex=True,
    ↪ constrained_layout=True)

axes[0].plot(
    resampled_df.index.total_seconds() / 3600.0,
    resampled_df["voltage_v"],
    color="tab:blue",
    linewidth=1.5,
)
axes[0].set_ylabel("Voltage [V]")
axes[0].set_title("Resampled public battery dataset trace")
axes[0].grid(True, alpha=0.3)

axes[1].plot(
    resampled_df.index.total_seconds() / 3600.0,
    resampled_df["current_a"],
    color="tab:orange",
    linewidth=1.5,
)
axes[1].set_xlabel("Elapsed time [h]")
axes[1].set_ylabel("Current [A]")
axes[1].grid(True, alpha=0.3)

plt.show()

```

2.8.2 Walkthrough 3 code explanation

The `download_if_needed` helper exists because data acquisition should be explicit and idempotent. If the file is already present in `data/raw/`, the code reuses it. If not, it downloads it and stores it exactly as obtained. This is the same raw-versus-processed discipline introduced in Chapter 1.

`canonicalize_text` is a small but important utility. Public CSV files do not agree on whether a column will be named `Current`, `current`, `Current (A)`, `Current[A]`, or `current-a`. Normalizing once at the beginning is much safer than writing fragile downstream code that depends on one exact spelling.

`normalize_columns` then maps semantically similar names to a canonical set we actually want to work with: `current_a`, `voltage_v`, `soc_pct`, `ambient_temperature_c`, `segment_label`, and `time_raw`. This is one of the most reusable patterns in the whole manual. Later chapters will do the same thing for CALCE, NASA, Oxford, and other battery datasets that come with their own naming conventions.

`build_elapsed_time_seconds` is deliberately defensive. It first checks whether a time-like column exists. If not, it builds a pseudo-time from row order. If a time column exists, it tries numeric parsing first and datetime parsing second. This is realistic battery-data work. Very often you do not know ahead of time which representation a public file uses.

The numeric coercions with `errors="coerce"` convert malformed strings to NaN rather than raising an error immediately. This is usually the right first move in dataset cleaning because it lets you inspect how much of the file is affected before deciding what to drop.

`drop_duplicates(subset=["elapsed_s"])` prevents resampling from failing on repeated timestamps. Duplicate time values are common in exported instrument data and almost always need attention.

The resampling step uses 10s bins and then `interpolate(method="time")`. This is a standard teaching choice. It produces a visually smoother trace and gives you a uniformly sampled signal for downstream work. It is also a methodological choice you should record in a real study, because resampling can alter fast transients if done carelessly.

The plotting choices deserve comment too. We use a two-panel figure rather than a dual-axis figure because current and voltage deserve their own y-axes and it is easier to read alignment across time that way. The colors are conventional and high-contrast. Grid lines are faint because they should support reading, not dominate the plot.

2.8.3 Walkthrough 3 expected output

The first printed block should list normalized column names that include at least `current_a`, `voltage_v`, and `elapsed_s`. If the file headings match the paper description closely, you should also see `soc_pct`, `ambient_temperature_c`, and `segment_label`.

The time-parsing note will tell you which branch of the parser was used. If an explicit time column is present and parseable, you should see a message about shifting numeric time to start at zero or converting datetimes to elapsed seconds. If not, the parser will tell you it fell back to pseudo-time based on row order. That fallback is acceptable for quick visualization but should be logged as a limitation.

The `head()` output should show a tidy table rather than a messy raw import. Current and voltage should be numeric. If the dataset includes SOC, it should look like a percentage-like quantity rather than random text. Missing-value fractions for current and voltage should be zero after cleaning.

The figure should show a long voltage trace in the upper panel and a corresponding current trace in the lower panel. The current trace should have blocks corresponding to constant-power charging or discharging segments and near-zero intervals if rest periods are present. A wrong result usually reveals itself immediately: flat zero current everywhere, voltage values clearly outside plausible pack ranges, or a plot compressed into a vertical line because the time axis was parsed incorrectly.

2.8.4 Walkthrough 3 troubleshooting

1. The file downloads but `pd.read_csv` fails with an encoding or delimiter issue
Symptom: a single giant column or unreadable characters.
Fix: inspect the raw file manually. If needed, pass an explicit delimiter or encoding. For this dataset the default comma-separated parse should work.
2. The time axis is nonsense
Symptom: elapsed time jumps backward or spans an absurd range.

Fix: print the raw time column before conversion. The file may store absolute time, elapsed time, or a format you need to parse differently.

3. Current and voltage columns are empty after coercion

Symptom: NaN fills the supposedly numeric columns.

Fix: the header mapping likely missed the correct source columns. Print `raw_df.columns.tolist()` and update the mapping logic.

4. Resampling smooths away important transitions

Symptom: sharp steps look suspiciously rounded.

Fix: reduce the resample interval or plot the raw cleaned signal alongside the resampled one. Resampling is a choice, not a law.

2.8.5 Walkthrough 3 reflection

This exercise taught the battery-data workflow that documentation often skips: download, normalize, coerce, time-align, resample, and only then plot. We will keep this exact posture throughout the manual. In publishable work, careful parsing is not “preprocessing before the real research.” It is part of the research.

2.9 Guided Walkthrough 4: Fit a Second-Order RC Model to Synthetic Noisy Data

Learning objective: Generate synthetic voltage data from a known RC model, add realistic measurement noise, and recover the model parameters with `scipy.optimize.least_squares`.

This is the moment where forward simulation turns into inverse modeling. Parameter fitting is where many otherwise clean numerical workflows become fragile, because the optimizer will happily chase numerical artifacts, bad initial guesses, or structurally unidentifiable parameter combinations if you let it. We will therefore keep the problem controlled: known OCV function, known current profile, moderate noise, bounded parameters, and a model class that matches the data-generating model.

2.9.1 Walkthrough 4 code

```
from dataclasses import dataclass

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
from scipy.optimize import least_squares

@dataclass(frozen=True)
class TheveninParameters:
    capacity_ah: float
    r0_ohm: float
    r1_ohm: float
    c1_f: float
    r2_ohm: float
    c2_f: float
```

```

def teaching_ocv_from_soc(soc_fraction):
    soc_fraction = np.clip(np.asarray(soc_fraction), 0.0, 1.0)
    return (
        2.85
        + 0.42 * soc_fraction
        + 0.06 * np.tanh((soc_fraction - 0.10) / 0.04)
        + 0.10 * np.tanh((soc_fraction - 0.88) / 0.05)
    )

```

```

TRUE_PARAMS = TheveninParameters(
    capacity_ah=3.2,
    r0_ohm=0.012,
    r1_ohm=0.009,
    c1_f=2200.0,
    r2_ohm=0.006,
    c2_f=22000.0,
)

```

```

profile_start_times_s = np.array([0, 200, 500, 800, 1100, 1400, 1800, 2200],
    ↪ dtype=float)
profile_currents_a = np.array([0.0, 2.5, 0.0, 5.0, 0.0, 3.5, 0.0, 1.5],
    ↪ dtype=float)
end_time_s = 2600.0
evaluation_times_s = np.linspace(0.0, end_time_s, 900)
initial_soc_fraction = 0.92

```

```

def current_profile_a(time_s):
    index = np.searchsorted(profile_start_times_s, time_s, side="right") - 1
    index = np.clip(index, 0, len(profile_currents_a) - 1)
    return float(profile_currents_a[index])

```

```

def simulate_thevenin(parameters):
    def rhs(time_s, state):
        soc_fraction, v_rc1_v, v_rc2_v = state
        current_a = current_profile_a(time_s)

        dsoc_dt = -current_a / (3600.0 * parameters.capacity_ah)
        dv_rc1_dt = -(v_rc1_v / (parameters.r1_ohm * parameters.c1_f)) +
    ↪ current_a / parameters.c1_f
        dv_rc2_dt = -(v_rc2_v / (parameters.r2_ohm * parameters.c2_f)) +
    ↪ current_a / parameters.c2_f
        return [dsoc_dt, dv_rc1_dt, dv_rc2_dt]

    solution = solve_ivp(
        fun=rhs,
        t_span=(0.0, end_time_s),
        y0=[initial_soc_fraction, 0.0, 0.0],
    )

```

```

        t_eval=evaluation_times_s,
        method="BDF",
        rtol=1e-8,
        atol=1e-10,
    )

    current_trace_a = np.array([current_profile_a(t) for t in solution.t])
    voltage_trace_v = (
        teaching_ocv_from_soc(solution.y[0])
        - current_trace_a * parameters.r0_ohm
        - solution.y[1]
        - solution.y[2]
    )

    return solution.t, current_trace_a, solution.y[0], voltage_trace_v

rng = np.random.default_rng(seed=42)
time_s, current_a, soc_fraction, clean_voltage_v =
    ↪ simulate_thevenin(TRUE_PARAMS)
measured_voltage_v = clean_voltage_v + rng.normal(loc=0.0, scale=0.003,
    ↪ size=clean_voltage_v.shape)

def parameter_vector_to_dataclass(parameter_vector):
    return TheveninParameters(
        capacity_ah=3.2,
        r0_ohm=parameter_vector[0],
        r1_ohm=parameter_vector[1],
        c1_f=parameter_vector[2],
        r2_ohm=parameter_vector[3],
        c2_f=parameter_vector[4],
    )

def residuals(parameter_vector):
    parameters = parameter_vector_to_dataclass(parameter_vector)
    _, _, _, simulated_voltage_v = simulate_thevenin(parameters)
    return simulated_voltage_v - measured_voltage_v

initial_guess = np.array([0.020, 0.015, 1200.0, 0.012, 12000.0])
lower_bounds = np.array([0.001, 0.001, 200.0, 0.001, 2000.0])
upper_bounds = np.array([0.050, 0.050, 20000.0, 0.050, 80000.0])

fit_result = least_squares(
    fun=residuals,
    x0=initial_guess,
    bounds=(lower_bounds, upper_bounds),
    method="trf",
    verbose=1,
)

```

```

fitted_params = parameter_vector_to_dataclass(fit_result.x)
_, _, _, fitted_voltage_v = simulate_thevenin(fitted_params)

comparison_table = pd.DataFrame(
    {
        "parameter": ["R0 [ohm]", "R1 [ohm]", "C1 [F]", "R2 [ohm]", "C2 [F]"],
        "true_value": [
            TRUE_PARAMS.r0_ohm,
            TRUE_PARAMS.r1_ohm,
            TRUE_PARAMS.c1_f,
            TRUE_PARAMS.r2_ohm,
            TRUE_PARAMS.c2_f,
        ],
        "initial_guess": initial_guess,
        "fitted_value": fit_result.x,
    }
)
comparison_table["relative_error_pct"] = (
    100.0
    * (comparison_table["fitted_value"] - comparison_table["true_value"])
    / comparison_table["true_value"]
)

rmse_v = np.sqrt(np.mean((fitted_voltage_v - measured_voltage_v) ** 2))

print(comparison_table.round(4))
print()
print(f"Optimization success: {fit_result.success}")
print(f"Optimizer message      : {fit_result.message}")
print(f"Voltage RMSE            : {rmse_v:.6f} V")

fig, axes = plt.subplots(2, 1, figsize=(10, 7), sharex=True,
    ↪ constrained_layout=True)

axes[0].plot(time_s / 60.0, measured_voltage_v, label="Synthetic measurement",
    ↪ linewidth=1.2)
axes[0].plot(time_s / 60.0, fitted_voltage_v, label="Fitted model",
    ↪ linewidth=2.0)
axes[0].set_ylabel("Voltage [V]")
axes[0].set_title("Parameter identification for a second-order RC model")
axes[0].grid(True, alpha=0.3)
axes[0].legend()

axes[1].plot(time_s / 60.0, measured_voltage_v - fitted_voltage_v,
    ↪ color="tab:red", linewidth=1.0)
axes[1].axhline(0.0, color="black", linestyle="--", linewidth=1.0)
axes[1].set_xlabel("Time [min]")
axes[1].set_ylabel("Residual [V]")
axes[1].grid(True, alpha=0.3)

plt.show()

```

2.9.2 Walkthrough 4 code explanation

The first half of the script is a forward simulator very similar to Walkthrough 2. That is deliberate. Parameter estimation should not require an entirely separate mental model. It should feel like “the same simulator, now embedded in an optimizer.”

TRUE_PARAMS defines the hidden ground truth we want the optimizer to recover. We keep capacity fixed during fitting because one of the key lessons of parameter estimation is restraint. The more parameters you let float, the easier it becomes for the optimizer to trade one physical effect against another. In real experiments, fitting Q_n , R_0 , R_1 , C_1 , R_2 , and C_2 all at once from one voltage trace is rarely a good first move.

The current profile includes multiple pulse amplitudes and rest intervals because identifiability improves when the excitation contains both fast and slow dynamic content. A constant-current trace alone often does not give enough information to separate two RC branches cleanly.

`rng = np.random.default_rng(seed=42)` makes the noisy data reproducible. That matters because otherwise every run would create a slightly different estimation problem, which makes learning harder and logging results messier.

The residual function returns `simulated_voltage_v - measured_voltage_v`, not the squared residuals. This is an important SciPy convention. `least_squares` wants the residual vector itself and takes care of the objective function internally.

The parameter bounds are not arbitrary. They encode physical plausibility and also stabilize the optimization numerically. Without bounds, the optimizer can temporarily explore absurd capacitance values or nearly zero resistances that make the problem harder to solve robustly. In real research, every bound should be justified in your notes or methods section.

The optimizer method `trf` is SciPy’s trust-region reflective algorithm, a sensible default for bounded least-squares problems. The `verbose=1` option is pedagogically useful because it reminds you that fitting is an iterative process, not magic.

The residual plot is as important as the parameter table. A fit can have a low RMSE and still miss the pulse edges systematically. Looking at the residual as a function of time is how you catch that.

2.9.3 Walkthrough 4 expected output

The comparison table should list the true values, the deliberately imperfect initial guesses, and the fitted values. Because the data are generated by the same model class being fitted, the recovered parameters should usually land close to the truth. With the noise level used here, you should expect R_0 to be recovered very accurately and the RC branch parameters to come back within a few to perhaps ten percent, depending on the random draw and solver details.

The script should print `Optimization success: True` and an optimizer message indicating termination because the gradient or cost reduction criterion was met. The voltage RMSE should be on the order of the noise level, roughly a few millivolts.

The upper plot should show the noisy synthetic measurement and the fitted model lying almost on top of each other. You should still be able to see small differences near the sharpest transitions because noise and parameter coupling make exact over-

lap unnecessary. The lower residual plot should oscillate around zero without long obvious drifts. A wrong result often looks like a good match in the flat parts and a bad mismatch at every pulse edge, which is a sign the branch time constants are wrong even if the global RMSE seems acceptable.

2.9.4 Walkthrough 4 troubleshooting

1. The optimizer converges but the parameters are unphysical
Symptom: one resistance hits the upper bound and one capacitance hits the lower bound.
Fix: your initial guess or excitation profile may not be informative enough. Try a richer pulse sequence or fewer free parameters.
2. The fit fails with a solver error inside `least_squares`
Symptom: SciPy complains during some candidate evaluations.
Fix: tighten bounds so the optimizer does not explore numerically awkward regions.
3. RMSE is low but the residual plot has a visible structure
Symptom: the residual swings systematically at each pulse.
Fix: do not trust RMSE alone. This usually means the model class or fitted time constants are still wrong.
4. Parameter estimates vary every time you rerun the script
Symptom: different results each run.
Fix: set the random seed, as we did with `default_rng(seed=42)`.

2.9.5 Walkthrough 4 reflection

This exercise taught the core idea of model calibration: a forward simulator wrapped in an optimizer. In Chapter 5 we will bring this idea into PyBaMM and discuss identifiability more seriously. For now, the important lesson is that fitting is not only about obtaining numbers. It is about checking whether the experiment, the model structure, and the residual pattern justify believing those numbers.

2.10 Guided Walkthrough 5: Reproduction Exercise - Recreate Figure 10 from Foles et al. (2022)

Learning objective: Reproduce a published time-series figure from a public battery dataset, document the ambiguities, and decide what counts as “close enough.”

This is the highest-value exercise in Part I because it forces all the earlier pieces to work together: parsing a public dataset, interpreting metadata, choosing a plotting strategy, and being explicit about what the paper does and does not specify. The target is Figure 10 from:

Ana Foles, Luís Fialho, Pedro Horta, and Manuel Collares-Pereira, “Validation of a lithium-ion commercial battery pack model using experimental data for stationary energy management application,” *Open Research Europe*, 2022, DOI 10.12688/open-research.14301.2.

The paper describes Figure 10 as “Lithium-ion battery voltage and current data from the experimental test plan, for complete charge-discharge cycles, at different constant power levels (due to readiness, only few power levels are represented).” The underlying dataset is the Zenodo CSV we used in Walkthrough 3. The figure caption does not list the exact selected power levels, which is the main ambiguity we need to handle honestly.

2.10.1 Our reproduction choices

We need to make three explicit choices.

First, because the caption says only “a few power levels are represented,” we will select three representative power levels directly from the `segment_label` text in the dataset: a low, medium, and high power level among those that appear frequently enough to plot cleanly.

Second, because public CSV exports can segment cycles unevenly, we will detect contiguous groups of identical `segment_label` values and then keep one representative contiguous segment for each selected power level.

Third, because exact figure aesthetics are rarely fully specified, we will aim to reproduce the figure’s informational content rather than every cosmetic choice. “Close enough” here means the resulting figure should show complete charge-discharge voltage and current traces for multiple power levels, with the expected ordering in duration and magnitude: lower power produces longer cycles, higher power produces shorter cycles and larger current magnitude.

2.10.2 Walkthrough 5 code

```
from pathlib import Path
from urllib.request import urlretrieve

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

DATA_URL = (
    "https://zenodo.org/records/5196334/files/"
    "Lithium_Ion_Battery_Testing_Data.csv?download=1"
)
RAW_PATH = Path("data/raw/Lithium_Ion_Battery_Testing_Data.csv")

def canonicalize_text(text):
    text = str(text).strip().lower()
    for old, new in {
        " ": "_",
        "-": "_",
        "/": "_per_",
        "(": "(",
        ")": ")",
        "[": "[",
    }.items():
        text = text.replace(old, new)
```



```

        "]" : "",
        "%" : "pct",
    }.items():
        text = text.replace(old, new)
    while "__" in text:
        text = text.replace("__", "_")
    return text.strip("_")

def load_foles_dataset():
    RAW_PATH.parent.mkdir(parents=True, exist_ok=True)
    if not RAW_PATH.exists():
        urlretrieve(DATA_URL, RAW_PATH)

    dataframe = pd.read_csv(RAW_PATH)
    dataframe.columns = [canonicalize_text(column) for column in
↪ dataframe.columns]

    rename_map = {}
    for column in dataframe.columns:
        if column == "bi":
            rename_map[column] = "segment_label"
        elif "current" in column and "charge" not in column:
            rename_map[column] = "current_a"
        elif "voltage" in column and "charge" not in column:
            rename_map[column] = "voltage_v"
        elif "soc" in column:
            rename_map[column] = "soc_pct"
        elif column in {"ta", "ambient_temperature", "ambient_temperature_c"}:
            rename_map[column] = "ambient_temperature_c"
        elif "time" in column and "temperature" not in column:
            rename_map[column] = "time_raw"

    dataframe = dataframe.rename(columns=rename_map)

    for column in ["current_a", "voltage_v", "soc_pct"]:
        if column in dataframe.columns:
            dataframe[column] = pd.to_numeric(dataframe[column],
↪ errors="coerce")

    if "time_raw" in dataframe.columns:
        numeric_time = pd.to_numeric(dataframe["time_raw"], errors="coerce")
        if numeric_time.notna().mean() > 0.95:
            dataframe["elapsed_s"] = numeric_time - numeric_time.iloc[0]
        else:
            datetime_time = pd.to_datetime(dataframe["time_raw"],
↪ errors="coerce")
            if datetime_time.notna().mean() > 0.95:
                dataframe["elapsed_s"] = (datetime_time -
↪ datetime_time.iloc[0]).dt.total_seconds()
            else:
                dataframe["elapsed_s"] = np.arange(len(dataframe), dtype=float)
    else:

```

```

        dataframe["elapsed_s"] = np.arange(len(dataframe), dtype=float)

    dataframe["segment_label"] = dataframe.get("segment_label", pd.Series("",
↪ index=dataframe.index)).fillna("")
    dataframe = dataframe.dropna(subset=["current_a", "voltage_v",
↪ "elapsed_s"]).copy()
    dataframe = dataframe.sort_values("elapsed_s").reset_index(drop=True)

    return dataframe

df = load_foles_dataset()

df["segment_id"] = (df["segment_label"] !=
↪ df["segment_label"].shift(fill_value="")).cumsum()
df["power_level_w"] = (
    df["segment_label"]
    .astype(str)
    .str.extract(r"(\d{3,4})", expand=False)
)
df["power_level_w"] = pd.to_numeric(df["power_level_w"], errors="coerce")

segment_summary = (
    df.groupby("segment_id")
    .agg(
        segment_label=("segment_label", "first"),
        power_level_w=("power_level_w", "first"),
        start_time_s=("elapsed_s", "first"),
        end_time_s=("elapsed_s", "last"),
        n_rows=("elapsed_s", "size"),
        mean_abs_current_a=("current_a", lambda x: np.mean(np.abs(x))),
    )
    .reset_index()
)
segment_summary["duration_s"] = segment_summary["end_time_s"] -
↪ segment_summary["start_time_s"]

usable_segments = segment_summary.dropna(subset=["power_level_w"]).copy()
usable_segments = usable_segments[usable_segments["duration_s"] > 0]
usable_segments = usable_segments.sort_values(["power_level_w", "duration_s"],
↪ ascending=[True, False])

available_levels = np.sort(usable_segments["power_level_w"].unique())
if len(available_levels) < 3:
    raise RuntimeError("Need at least three distinguishable power levels for
↪ the reproduction.")

selected_levels = np.array([
    available_levels[0],
    available_levels[len(available_levels) // 2],
    available_levels[-1],
])

```

```

selected_segment_rows = []
for level in selected_levels:
    representative_segment = usable_segments[usable_segments["power_level_w"]
    == level].iloc[0]
    selected_segment_rows.append(representative_segment)

selected_segments = pd.DataFrame(selected_segment_rows)
print("Selected representative power levels [W]:", selected_levels.tolist())
print(selected_segments[["segment_label", "power_level_w", "duration_s",
    "n_rows"]])

plot_frames = []
for _, row in selected_segments.iterrows():
    segment_df = df[df["segment_id"] == row["segment_id"]].copy()
    segment_df["segment_elapsed_h"] = (segment_df["elapsed_s"] -
    segment_df["elapsed_s"].iloc[0]) / 3600.0
    segment_df["power_level_w"] = row["power_level_w"]
    plot_frames.append(segment_df)

plot_df = pd.concat(plot_frames, ignore_index=True)

fig, axes = plt.subplots(2, 1, figsize=(10, 8), sharex=True,
    constrained_layout=True)

for level, one_level_df in plot_df.groupby("power_level_w", sort=True):
    axes[0].plot(
        one_level_df["segment_elapsed_h"],
        one_level_df["voltage_v"],
        linewidth=2.0,
        label=f"{int(level)} W",
    )
    axes[1].plot(
        one_level_df["segment_elapsed_h"],
        one_level_df["current_a"],
        linewidth=2.0,
        label=f"{int(level)} W",
    )

axes[0].set_ylabel("Voltage [V]")
axes[0].set_title("Reproduction of Foles et al. (2022) Figure 10 using the
    public CSV")
axes[0].grid(True, alpha=0.3)
axes[0].legend(title="Power level")

axes[1].set_xlabel("Elapsed time within representative segment [h]")
axes[1].set_ylabel("Current [A]")
axes[1].grid(True, alpha=0.3)

figure_path = Path("figures/chapter_2_reproduced_figure_10_foles.png")
figure_path.parent.mkdir(parents=True, exist_ok=True)
fig.savefig(figure_path, dpi=300)
print(f"Saved reproduction figure to: {figure_path}")

```

```
plt.show()
```

2.10.3 Walkthrough 5 code explanation

The data-loading function repeats logic from Walkthrough 3 because every code block in this companion should be runnable independently. In your own repository, you would probably factor the shared loader into `src/data_loading.py`. In a teaching chapter, repetition is preferable to hidden dependencies.

`segment_id = (segment_label != segment_label.shift()).cumsum()` is a very useful Pandas idiom. It turns changes in a label column into contiguous segment identifiers. This lets us separate one block of constant-power operation from the next even if the same power level appears again later in the file.

`power_level_w` is extracted from text with a regular expression. This is a realistic compromise. The paper tells us the file contains legend text in BI, but it does not give us a perfectly normalized numeric power column. Instead of complaining, we mine the text responsibly.

The segment summary table computes duration, row count, and mean absolute current. These diagnostics help us filter out trivial or degenerate segments. We then sort by power level and duration and choose the longest representative segment for each selected power level.

The most important methodological line is the selection of `selected_levels` as low, middle, and high values from the available set. This is where we handle the paper's ambiguity explicitly. We are not pretending to know the exact hidden choice behind the published figure. We are making a principled, reproducible choice from the public data and stating it.

We plot voltage and current in stacked axes because Figure 10 is about both together. The x-axis is elapsed time within each representative segment, not global experiment time, because that makes multi-level comparison much clearer and is consistent with the paper's figure intent.

2.10.4 Walkthrough 5 expected output

The printed table should list three selected representative segments with their power levels, durations, and row counts. The exact labels depend on the dataset, but you should see one relatively low power level, one midrange value, and one high value.

The reproduced figure should have two panels. In the upper panel, the three voltage traces should occupy similar voltage ranges but differ in duration and slope; higher power generally produces a faster traversal of the cycle. In the lower panel, current magnitudes should clearly differ by power level. Depending on the dataset's sign convention, discharge may appear as positive or negative current. What matters is consistency with the voltage evolution and the paper's narrative.

"Close enough" for this reproduction does not mean pixel-perfect overlay with the paper figure. It means:

1. The figure displays complete representative charge-discharge behavior for multiple constant power levels.

2. The relative ordering makes physical sense: lower power corresponds to longer duration, higher power to larger current magnitude.
3. The general visual story matches the published caption and surrounding text.

If your result satisfies those three conditions, you have performed a defensible reproduction. If it does not, the first place to investigate is not Matplotlib styling. It is segment selection and sign convention.

2.10.5 Walkthrough 5 troubleshooting

1. No usable power levels are extracted
Symptom: `power_level_w` is all NaN.
Fix: inspect the raw `segment_label` strings. The numeric power value may be formatted differently from the regular expression we used.
2. The selected segment is too short or obviously incomplete
Symptom: the plotted curve looks like only a fragment of a cycle.
Fix: refine the segment filter using duration or row count and inspect the summary table manually.
3. Current sign appears “wrong” relative to your expectation
Symptom: discharge segments are negative when you expected positive, or vice versa.
Fix: do not force a sign change unless you can justify it. The goal is faithful reproduction, not aesthetic conformity.
4. Your reproduced figure does not visually resemble the published one
Symptom: wrong ordering, strange durations, or implausible voltage range.
Fix: revisit the ambiguity discussion. Figure reproduction is often mostly about data segmentation, not plotting syntax.

2.10.6 Walkthrough 5 reflection

This exercise taught a publishable-research habit that is more important than people admit: when a paper is ambiguous, do not hide the ambiguity. State your interpretation, make it reproducible, and define what “close enough” means. That is the difference between a serious reproduction attempt and a screenshot imitation.

2.11 Open-Ended Exercises

These are the first points in the chapter where you should stop copying and start adapting. Work them before reading the solutions.

2.11.1 Exercise 1: Make the ODE problem more stiff

Modify Walkthrough 2 so that the fast RC branch becomes much faster and the slow branch becomes much slower. A good starting point is to reduce `C1` by a factor of 10 and increase `C2` by a factor of 5. Then compare RK45, BDF, and LSODA again.

Hints:

- Keep the current profile unchanged so the solver comparison is meaningful.
- Compare `nfev`, wall-clock time, and maximum voltage differences.

- Pay attention to whether one solver begins to take dramatically more function evaluations.

2.11.2 Exercise 2: Fit the wrong model on purpose

Repeat Walkthrough 4, but fit a first-order RC model to data generated by the second-order model. In other words, allow only R0, R1, and C1 to vary and remove the second RC branch from the fitted model.

Hints:

- Keep the same noisy synthetic measurements.
- Compare RMSE and the residual plot against the second-order fit.
- Ask which features of the residual reveal model inadequacy.

2.11.3 Exercise 3: Flatten the OCV curve to mimic sodium-ion behavior

Replace `teaching_ocv_from_soc` with a flatter plateau-like function and rerun the synthetic fitting problem from Walkthrough 4. Keep the same current profile and noise level.

Hints:

- A plateau means $dOCV/dz$ becomes small over a wide SOC range.
- Think about why that makes SOC-related effects harder to separate from RC transients.
- Compare how stable the fitted parameters look relative to the earlier case.

2.12 Worked Solutions to Open-Ended Exercises

2.12.1 Solution to Exercise 1

The code change is small, but the interpretation matters.

```
import numpy as np
import pandas as pd
from scipy.integrate import solve_ivp
```

```
class Params:
    capacity_ah = 3.0
    r0_ohm = 0.015
    r1_ohm = 0.010
    c1_f = 180.0
    r2_ohm = 0.008
    c2_f = 90000.0
```

```
segment_start_times_s = np.array([0, 300, 900, 1200, 1500, 1800, 2400, 3000],
    ↪ dtype=float)
segment_currents_a = np.array([0.0, 3.0, 0.0, 6.0, 0.0, 4.5, 0.0, 1.5],
    ↪ dtype=float)
```

```

def ocv_from_soc(soc):
    soc = np.clip(np.asarray(soc), 0.0, 1.0)
    return 2.85 + 0.42 * soc + 0.06 * np.tanh((soc - 0.10) / 0.04) + 0.10 *
        ↪ np.tanh((soc - 0.88) / 0.05)

def current_profile_a(time_s):
    index = np.searchsorted(segment_start_times_s, time_s, side="right") - 1
    index = np.clip(index, 0, len(segment_currents_a) - 1)
    return float(segment_currents_a[index])

def rhs(time_s, state):
    soc, v1, v2 = state
    current_a = current_profile_a(time_s)
    return [
        -current_a / (3600.0 * Params.capacity_ah),
        -(v1 / (Params.r1_ohm * Params.c1_f)) + current_a / Params.c1_f,
        -(v2 / (Params.r2_ohm * Params.c2_f)) + current_a / Params.c2_f,
    ]

summary_rows = []
evaluation_times_s = np.linspace(0.0, 3600.0, 1201)

for method_name in ["RK45", "BDF", "LSODA"]:
    solution = solve_ivp(
        fun=rhs,
        t_span=(0.0, 3600.0),
        y0=[0.95, 0.0, 0.0],
        t_eval=evaluation_times_s,
        method=method_name,
        rtol=1e-8,
        atol=1e-10,
    )
    current_trace_a = np.array([current_profile_a(t) for t in solution.t])
    voltage_trace_v = ocv_from_soc(solution.y[0]) - current_trace_a *
    ↪ Params.r0_ohm - solution.y[1] - solution.y[2]
    summary_rows.append(
        {
            "method": method_name,
            "nfev": solution.nfev,
            "final_voltage_v": voltage_trace_v[-1],
        }
    )

print(pd.DataFrame(summary_rows))

```

You should find that RK45 now takes noticeably more function evaluations than before, while BDF and often LSODA remain comparatively comfortable. That is the practical signal of increased stiffness. The model class did not change, only the separation in time scales.

2.12.2 Solution to Exercise 2

Here we intentionally underfit the data with a model that is too simple.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
from scipy.optimize import least_squares

def ocv_from_soc(soc):
    soc = np.clip(np.asarray(soc), 0.0, 1.0)
    return 2.85 + 0.42 * soc + 0.06 * np.tanh((soc - 0.10) / 0.04) + 0.10 *
    ↪ np.tanh((soc - 0.88) / 0.05)

profile_start_times_s = np.array([0, 200, 500, 800, 1100, 1400, 1800, 2200],
    ↪ dtype=float)
profile_currents_a = np.array([0.0, 2.5, 0.0, 5.0, 0.0, 3.5, 0.0, 1.5],
    ↪ dtype=float)
evaluation_times_s = np.linspace(0.0, 2600.0, 900)

def current_profile_a(time_s):
    index = np.searchsorted(profile_start_times_s, time_s, side="right") - 1
    index = np.clip(index, 0, len(profile_currents_a) - 1)
    return float(profile_currents_a[index])

def generate_second_order_truth():
    true_params = {"capacity_ah": 3.2, "r0": 0.012, "r1": 0.009, "c1": 2200.0,
    ↪ "r2": 0.006, "c2": 22000.0}

    def rhs(time_s, state):
        soc, v1, v2 = state
        current_a = current_profile_a(time_s)
        return [
            -current_a / (3600.0 * true_params["capacity_ah"]),
            -(v1 / (true_params["r1"] * true_params["c1"])) + current_a /
            ↪ true_params["c1"],
            -(v2 / (true_params["r2"] * true_params["c2"])) + current_a /
            ↪ true_params["c2"],
        ]

    solution = solve_ivp(rhs, (0.0, 2600.0), [0.92, 0.0, 0.0],
    ↪ t_eval=evaluation_times_s, method="BDF")
    current_trace_a = np.array([current_profile_a(t) for t in solution.t])
    voltage_trace_v = ocv_from_soc(solution.y[0]) - current_trace_a *
    ↪ true_params["r0"] - solution.y[1] - solution.y[2]
    return voltage_trace_v

rng = np.random.default_rng(42)
```



```
measured_voltage_v = generate_second_order_truth() + rng.normal(0.0, 0.003,
↪ size=evaluation_times_s.shape)
```

```
def simulate_first_order(parameter_vector):
    r0_ohm, r1_ohm, c1_f = parameter_vector
```

```
    def rhs(time_s, state):
        soc, v1 = state
        current_a = current_profile_a(time_s)
        return [
            -current_a / (3600.0 * 3.2),
            -(v1 / (r1_ohm * c1_f)) + current_a / c1_f,
        ]
```

```
    solution = solve_ivp(rhs, (0.0, 2600.0), [0.92, 0.0],
↪ t_eval=evaluation_times_s, method="BDF")
    current_trace_a = np.array([current_profile_a(t) for t in solution.t])
    voltage_trace_v = ocv_from_soc(solution.y[0]) - current_trace_a * r0_ohm -
↪ solution.y[1]
    return voltage_trace_v
```

```
def residuals(parameter_vector):
    return simulate_first_order(parameter_vector) - measured_voltage_v
```

```
fit_result = least_squares(
    residuals,
    x0=np.array([0.020, 0.012, 2000.0]),
    bounds=(np.array([0.001, 0.001, 100.0]), np.array([0.050, 0.050,
↪ 40000.0])),
)
```

```
fitted_voltage_v = simulate_first_order(fit_result.x)
residual_v = measured_voltage_v - fitted_voltage_v
```

```
print("Fitted parameters:", fit_result.x)
print("RMSE [V]:", np.sqrt(np.mean(residual_v ** 2)))
```

```
plt.figure(figsize=(9, 4.5))
plt.plot(evaluation_times_s / 60.0, residual_v, linewidth=1.0)
plt.axhline(0.0, color="black", linestyle="--", linewidth=1.0)
plt.xlabel("Time [min]")
plt.ylabel("Residual [V]")
plt.title("Residual from fitting a first-order model to second-order data")
plt.grid(True, alpha=0.3)
plt.show()
```

The important outcome is not just a higher RMSE. It is the residual structure. You should see systematic pulse-edge mismatch that the first-order model cannot eliminate. That is exactly the kind of evidence you need before claiming a model order is inadequate.

2.12.3 Solution to Exercise 3

Here we flatten the OCV curve to mimic one of the practical difficulties of sodium-ion work.

```
import numpy as np
from scipy.integrate import solve_ivp
from scipy.optimize import least_squares

def flat_plateau_ocv_from_soc(soc):
    soc = np.clip(np.asarray(soc), 0.0, 1.0)
    return 2.95 + 0.05 * soc + 0.025 * np.tanh((soc - 0.08) / 0.03) + 0.030 *
    ↪ np.tanh((soc - 0.92) / 0.03)

profile_start_times_s = np.array([0, 200, 500, 800, 1100, 1400, 1800, 2200],
    ↪ dtype=float)
profile_currents_a = np.array([0.0, 2.5, 0.0, 5.0, 0.0, 3.5, 0.0, 1.5],
    ↪ dtype=float)
evaluation_times_s = np.linspace(0.0, 2600.0, 900)

def current_profile_a(time_s):
    index = np.searchsorted(profile_start_times_s, time_s, side="right") - 1
    index = np.clip(index, 0, len(profile_currents_a) - 1)
    return float(profile_currents_a[index])

def simulate(parameter_vector):
    r0_ohm, r1_ohm, c1_f, r2_ohm, c2_f = parameter_vector

    def rhs(time_s, state):
        soc, v1, v2 = state
        current_a = current_profile_a(time_s)
        return [
            -current_a / (3600.0 * 3.2),
            -(v1 / (r1_ohm * c1_f)) + current_a / c1_f,
            -(v2 / (r2_ohm * c2_f)) + current_a / c2_f,
        ]

    solution = solve_ivp(rhs, (0.0, 2600.0), [0.92, 0.0, 0.0],
    ↪ t_eval=evaluation_times_s, method="BDF")
    current_trace_a = np.array([current_profile_a(t) for t in solution.t])
    return flat_plateau_ocv_from_soc(solution.y[0]) - current_trace_a * r0_ohm
    ↪ - solution.y[1] - solution.y[2]

true_parameters = np.array([0.012, 0.009, 2200.0, 0.006, 22000.0])
rng = np.random.default_rng(42)
measured_voltage_v = simulate(true_parameters) + rng.normal(0.0, 0.003,
    ↪ size=evaluation_times_s.shape)
```

```

fit_result = least_squares(
    fun=lambda p: simulate(p) - measured_voltage_v,
    x0=np.array([0.020, 0.015, 1200.0, 0.012, 12000.0]),
    bounds=(np.array([0.001, 0.001, 200.0, 0.001, 2000.0]), np.array([0.050,
↪ 0.050, 20000.0, 0.050, 80000.0])),
)

print("True parameters :", true_parameters)
print("Fitted parameters:", np.round(fit_result.x, 6))
print("RMSE [V]          :", np.sqrt(np.mean((simulate(fit_result.x) -
↪ measured_voltage_v) ** 2)))

```

The typical outcome is that the optimizer still fits voltage reasonably well, but the recovered parameters become less stable or less interpretable. That is the qualitative lesson you should take forward into sodium-ion work: flatter OCV regions reduce the observability you get “for free” from voltage.

2.13 What Changes for Sodium-Ion?

Up to this point, almost every skill in the chapter is chemistry-agnostic. Arrays, ODE solvers, least-squares fitting, Pandas cleaning, and figure construction do not care whether the active ion is lithium or sodium. But the interpretation of the outputs does change, and it changes in ways you should start internalizing now.

The first change is the OCV curve. Many sodium-ion chemistries, especially hard-carbon-based systems, exhibit flatter voltage plateaus over important SOC ranges than the textbook lithium-ion examples you may be used to. In numerical terms, that means $d\text{OCV}/dz$ can be small over a broad interval. The practical consequence is that terminal voltage becomes less informative about SOC in those regions, and RC transients or sensor noise can dominate what you see. When you reach the fitting and state-estimation chapters later in the manual, that one chemistry fact will explain a large fraction of the extra difficulty.

The second change is parameter transferability. In this chapter we used a smooth teaching OCV function and synthetic RC parameters. In sodium-ion work, you should be much less willing to borrow lithium-ion OCV or resistance assumptions casually. The workflow of the chapter still applies, but the parameter values need stronger justification, and the sensitivity of the results to those values should be reported more explicitly.

The third change is dataset availability. Lithium-ion public datasets are abundant enough that we can afford to learn parsing on them. Sodium-ion public datasets are much sparser and often less standardized. That makes the data-cleaning habits from Walkthrough 3 even more important. In sodium-ion projects, you will often combine sparse public measurements, digitized figures, and simulation-generated synthetic data. The computational workflow stays the same; the provenance burden gets heavier.

The fourth change is validation strategy. For lithium-ion, it is often possible to find a public benchmark dataset that closely matches your modeling setup. For sodium-ion, you may need to validate at the level of qualitative behaviors, voltage-window consistency, rate sensitivity, or agreement with a published figure rather than against

a rich open raw dataset. That does not lower the standard. It changes what honest validation looks like.

So the short version is this: the software patterns from this chapter transfer directly to sodium-ion, but the chemistry makes the inverse problems harder, the datasets thinner, and the need for explicit assumptions greater. Keep that in mind from the beginning rather than treating sodium-ion as “lithium-ion with renamed variables.”

2.14 Chapter Summary and Skill Checklist

The key skills from this chapter are the ones you should start to feel in your hands rather than merely remember abstractly.

- evaluate battery equations over entire grids with NumPy vectorization and broadcasting
- slice arrays deliberately to isolate physically meaningful windows
- write a state-vector ODE for a second-order Thevenin model
- choose among RK45, BDF, and LSODA with a reason rather than a guess
- inspect solver diagnostics instead of trusting a successful run blindly
- download and normalize a public battery dataset with Pandas
- infer a usable time axis and resample a battery time series
- fit model parameters with `scipy.optimize.least_squares`
- diagnose a fit with residual structure, not RMSE alone
- reproduce a published figure while documenting the ambiguities honestly

You should now be able to check all of the following:

- I can represent a battery equivalent-circuit model as a system of first-order ODEs.
- I can explain why SOC, RC overpotentials, and terminal voltage play different roles in the code.
- I can use broadcasting to evaluate terminal voltage for multiple current levels without nested loops.
- I can run `solve_ivp` and explain why one solver may be more appropriate than another.
- I can build a residual function and fit parameters with bounded least squares.
- I can clean a real battery CSV rather than only work with synthetic arrays.
- I can state my current sign convention explicitly instead of assuming it.
- I can reproduce a published time-series figure in a way that is transparent about interpretation choices.

If any box above still feels uncertain, revisit the corresponding walkthrough now. Later chapters assume these skills are already in muscle memory.

2.15 Deliverable

The deliverable from your plan is a notebook that simulates a second-order RC equivalent-circuit discharge, adds noise, and fits the parameters back. The bonus deliverable is to wrap it as a reusable function.

Approach the deliverable in two layers.

First, create the notebook `notebooks/chapter_2_scientific_python_refresher.ipynb` and make sure it contains:

- a clean implementation of the second-order Thevenin forward model
- at least one pulse-rest current protocol
- a synthetic noisy voltage trace generated from known parameters
- a bounded least-squares fit recovering those parameters
- at least one residual plot and one parameter-comparison table

Second, factor the forward model into a reusable helper in `src/thevenin_model.py` or similar. A minimal but good reusable interface would be a function that accepts parameters, a current-profile function, initial SOC, and evaluation times and returns a `DataFrame` of time, current, SOC, branch voltages, and terminal voltage.

A strong submission for this chapter will also save:

- a CSV of the synthetic measurements in `results/`
- a CSV of fitted parameters in `results/`
- at least one figure in `figures/`
- a short log entry describing solver choice, fit behavior, and what you learned from the residuals

If you want a worked partial structure for the reusable function, this is a good starting signature:

```
def simulate_thevenin(
    parameters,
    current_profile_a,
    evaluation_times_s,
    initial_soc_fraction,
    method="BDF",
):
    ...
```

The important design idea is to separate the model definition from the specific experiment. That habit will pay off immediately in Chapter 3 when PyBaMM starts making the same separation explicit at a much larger scale.

2.16 Further Practice and Reading

- Ana Foles, Luís Fialho, Pedro Horta, and Manuel Collares-Pereira, “Validation of a lithium-ion commercial battery pack model using experimental data for stationary energy management application,” *Open Research Europe* (2022). This is the paper used in the reproduction exercise and worth reading alongside the public CSV.
- G. L. Plett, *Battery Management Systems*, especially the equivalent-circuit modeling material. The mathematics there will feel different after doing the coding work in this chapter.
- SciPy official documentation for `scipy.integrate.solve_ivp` and `scipy.optimize.least_squares`. Bookmark both now; they will reappear throughout the manual.
- NumPy official documentation on broadcasting. This is one of the few docs pages that repays careful rereading.

- Matplotlib official tutorials on labeling, styles, and multi-axis layouts. Publication-quality plotting is learned by iteration, not by memorizing one magic function.
- Community resources: Scientific Python Forum and the PyData community spaces. Even before PyBaMM enters, these are useful homes for the numerical questions that battery research raises.

Lab Chapter 3: *Your First PyBaMM Simulation* is next.

Chapter 3

Your First PyBaMM Simulation

3.1 Chapter Opening

This chapter is the point where the theory textbook stops being something you understand and starts becoming something you can execute. Up to now, the Doyle-Fuller-Newman model has lived mostly in the language of conservation laws, porous-electrode assumptions, constitutive relations, and boundary conditions. By the end of this chapter, you will have turned that theory into a working PyBaMM workflow that you can run, modify, inspect, and defend. You will know how to build a lithium-ion battery model from PyBaMM's core objects, how to solve a default DFN simulation with a real parameter set, how to extract internal states instead of staring only at terminal voltage, and how to compare the DFN against reduced-order models in a way that is honest about both computational cost and modeling error.

This matters for publishable research because physics-based battery modeling is not only about deriving equations correctly. It is about being able to move from the equations to a trustworthy software implementation, ask the model targeted questions, and explain why a particular model fidelity is justified for a particular study. Reviewers do not care that you know the DFN model exists. They care whether your parameter set is traceable, whether your solver choices are sensible, whether your plotted internal states mean what you say they mean, and whether you are using a reduced model because it is appropriate rather than because you never learned how to run the full one.

Keep Textbook Chapter 8 open while you work through this lab. This chapter operationalizes the DFN model you learned there, so you want the derivation fresh in your mind. When we inspect electrolyte concentration gradients, solid-particle diffusion, interfacial overpotentials, and open-circuit voltage contributions, we are not learning new battery physics from scratch. We are learning how PyBaMM represents the same physics in software. Textbook Chapter 3 should also be nearby, because the distinction between open-circuit voltage, terminal voltage, overpotential, discharge capacity, C-rate, and state of charge needs to remain exact when we start reading variables out of a simulation object. Textbook Chapter 10 is worth keeping in the background as well, because one of the most important skills you will develop here is choosing an appropriate model fidelity. That same judgment call sits underneath every later BMS approximation, virtual-cell workflow, and parameter-estimation loop.

This chapter also depends directly on Labs 1 and 2. Lab Chapter 1 gave you the reproducible environment and project habits we now rely on. Lab Chapter 2 refreshed the scientific Python tools that sit underneath every modeling workflow in this companion. PyBaMM is not a replacement for those skills. It is a higher-level expression of them. If NumPy arrays, SciPy-style problem structure, and plotting logic still feel foggy, this chapter will become much harder than it needs to be.

There is one more reason this chapter matters for your long-term goal of sodium-ion research. Today we will use mostly lithium-ion parameter sets because PyBaMM's built-in ecosystem is much richer there. That is not a detour. It is training. Your future sodium-ion work will reuse the same software patterns: model construction, parameter management, solver choice, internal-variable inspection, fidelity comparison, and figure reproduction. What will change is the chemistry-specific content: different OCV shapes, different diffusivity functions, different parameter availability, and weaker public-data coverage. The software method is durable. We are learning that durable layer here.

The pace of the chapter is deliberate. First, we will install PyBaMM properly and verify that it really works. Next, we will build a conceptual bridge from the DFN derivation in the textbook to PyBaMM's object model. Then we will run a first DFN simulation with the Chen2020 parameter set and interpret every major curve. After that, we will open the model up and inspect internal variables, because serious battery modeling begins when you stop treating voltage as the only truth. Then we will compare SPM, SPM_e, and DFN on the same duty cycle and quantify the accuracy-speed tradeoff. Finally, because every Part of this companion must train reproduction skill, we will reproduce a published SPM_e error-decomposition figure based on Marquis et al. (2019) and discuss exactly what "close enough" means when a modern software implementation is not numerically identical to the authors' original post-processing pipeline.

If you do this chapter carefully, PyBaMM stops being an intimidating package name and becomes a research instrument. That is the real milestone.

3.2 Prerequisites Check

- Required software: the `sib-research` environment from Lab Chapter 1; Python 3.11; JupyterLab 4.4+
- Additional Python package for this chapter: `pybamm==25.12.1`
- Recommended plotting stack already installed from earlier chapters: `numpy==2.3.4`, `scipy==1.16.0`, `pandas==3.0.2`, `matplotlib==3.10.8`
- Required textbook chapters: Textbook Chapter 8 is essential; Textbook Chapter 3 should be close at hand; Textbook Chapter 10 is helpful context
- Required prior lab chapters: Lab Chapter 1 and Lab Chapter 2
- Estimated time: 10 to 14 hours if your install is smooth; 14 to 16 hours if this is your first real PyBaMM workflow

If Lab Chapter 2 still feels shaky, especially with plotting and numerical thinking, revisit Sections 2.2 through 2.6 before going deeper here. If your Jupyter kernel setup from Chapter 1 was never fully clean, fix it now. The most common PyBaMM beginner problem is not a bad model. It is running the right code in the wrong environment.

3.3 Environment Setup

PyBaMM installs cleanly for most readers if the environment is already healthy. The key phrase there is “if the environment is already healthy.” We will not assume that. We will verify each step.

3.3.1 Activate the Chapter 1 environment

If you used conda in Lab Chapter 1:

```
conda activate sib-research
```

If you used venv on Linux or macOS:

```
source .venv/bin/activate
```

If you used venv on Windows PowerShell:

```
.venv\Scripts\Activate.ps1
```

3.3.2 Install PyBaMM with a pinned version

For this manual we will pin to `pybamm==25.12.1`. The pin is not because older or newer versions are necessarily wrong. It is because beginner pain rises sharply when examples are written against one API and run against another.

Install PyBaMM with:

```
python -m pip install pybamm==25.12.1
```

If you prefer to make the version pin explicit in a requirements file for the repository, create `requirements-chapter-3.txt`:

```
pybamm==25.12.1
```

Then install it with:

```
python -m pip install -r requirements-chapter-3.txt
```

If you are using conda, do not overcomplicate this. A clean conda environment plus `pip install pybamm==25.12.1` is a perfectly normal workflow for PyBaMM.

3.3.3 Verify the install with a minimal “hello battery”

Run this exact snippet in a terminal or a fresh notebook cell:

```
import pybamm
import numpy as np
```

```
# Build the default DFN with a bundled teaching parameter set.
```

```
model = pybamm.lithium_ion.DFN()
```

```
parameter_values = pybamm.ParameterValues("Chen2020")
```

```
simulation = pybamm.Simulation(model, parameter_values=parameter_values)
```

```
# Ask PyBaMM for output on a clean reporting grid.
```

```
evaluation_times_s = np.linspace(0, 600, 61)
```

```
solution = simulation.solve(evaluation_times_s)
```

```
# Evaluate the processed variable on the same time grid.
```

```
terminal_voltage_v = solution["Terminal voltage [V]"](evaluation_times_s)
```

```
print("PyBaMM version:", pybamm.__version__)
print("Initial voltage [V]:", round(float(terminal_voltage_v[0]), 4))
print("Voltage at 600 s [V]:", round(float(terminal_voltage_v[-1]), 4))
```

Expected output on the pinned version is:

```
PyBaMM version: 25.12.1
Initial voltage [V]: 4.0377
Voltage at 600 s [V]: 3.8155
```

Do not obsess if the last digit differs by a few units in the fourth decimal place on a different machine. Do stop immediately if you get an import error, a solver failure, or a voltage that is nowhere near these values.

3.3.4 Confirm Jupyter sees the correct environment

Launch JupyterLab:

```
jupyter lab
```

Create a notebook named `notebooks/chapter_3_first_pybamm_simulation.ipynb`, then run:

```
import sys
import pybamm

print(sys.executable)
print(pybamm.__version__)
```

The interpreter path should point into your intended environment, and the printed version should be 25.12.1.

3.3.5 Common install failures and fixes

1. ModuleNotFoundError: No module named 'pybamm'

Symptom: PyBaMM imports in the terminal fail, or they work in the terminal but not in Jupyter.

Fix: if both terminal and notebook fail, PyBaMM is not installed in the active environment. Re-activate the environment and reinstall. If the terminal works but Jupyter fails, re-register the kernel from the active environment with `python -m ipykernel install --user --name sib-research --display-name "Python (sib-research)"`.

2. Installation succeeds but import fails with a dependency error mentioning casadi, scipy, or a compiled wheel

Symptom: `pip` finishes, but `import pybamm` crashes immediately.

Fix: create a fresh environment instead of patching the broken one. Most PyBaMM import failures are environment-consistency problems, not package bugs.

3. Solver errors on the first run

Symptom: the package imports but the hello-world simulation fails.

Fix: first confirm you copied the code exactly. If you did, remove any local modifications to your environment and retry in a fresh environment. Beginner PyBaMM code should not fail on the built-in DFN plus Chen2020.

4. A notebook seems to “hang” while solving

Symptom: the cell runs longer than expected with no visible output.

Fix: wait a little longer first; the first import and solve can take longer on some systems. If it remains stalled, restart the kernel and run the small 600-second example before attempting anything larger.

3.4 Conceptual Bridge: From Textbook DFN Equations to PyBaMM Objects

In Textbook Chapter 8, the Doyle-Fuller-Newman model appeared as a coupled system of equations for lithium concentration in the solid particles, lithium-ion concentration in the electrolyte, charge conservation in the solid phase, charge conservation in the electrolyte, interfacial reaction kinetics, and the constitutive laws that tie those fields together. In that presentation, the intellectual burden is on understanding the model. In PyBaMM, the burden shifts slightly: you still need to understand the model, but now you also need to understand how the software stores, processes, discretizes, and solves it.

At the theory level, the DFN model can be summarized very compactly. In the active-material particles we solve diffusion:

$$\frac{\partial c_{s,k}}{\partial t} = \frac{1}{r^2} \frac{\partial}{\partial r} \left(D_{s,k} r^2 \frac{\partial c_{s,k}}{\partial r} \right), \quad k \in \{n, p\}, \quad (1)$$

while in the electrolyte we solve salt conservation:

$$\varepsilon_k \frac{\partial c_{e,k}}{\partial t} = -\nabla \cdot N_{e,k} + \frac{1-t_+^0}{F} a_k j_k, \quad (2)$$

and in the solid and electrolyte phases we enforce charge conservation:

$$\nabla \cdot i_{s,k} = -a_k j_k, \quad \nabla \cdot i_{e,k} = a_k j_k. \quad (3)$$

The interfacial current density j_k is usually closed through Butler-Volmer kinetics, and the measured terminal voltage emerges from the difference between positive and negative current-collector potentials, corrected by the various losses and concentration effects. None of that changes in PyBaMM. What changes is the representation.

PyBaMM does not begin by “running a solver.” It begins by building a symbolic model graph. When you write:

```
model = pybamm.lithium_ion.DFN()
```

you are not yet solving anything. You are constructing a software object that contains the model equations, variables, submodels, events, boundary conditions, and meta-data. You can think of `model` as the theory-textbook object translated into symbolic Python form.

The next critical object is the parameter set:

```
parameter_values = pybamm.ParameterValues("Chen2020")
```

This is one of the most important conceptual habits to internalize early. In PyBaMM, the model and the parameter values are intentionally separate. That separation matters because it lets you ask a research question cleanly. Are you changing the governing equations, or are you changing only the chemistry and cell properties fed into those equations? If you blur those two ideas together, you will later make it much harder to report what you actually did.

After parameters, PyBaMM still has more work to do before anything becomes numerical. The geometry object defines where each field lives. The mesh object decides where those domains are sampled. The discretization object converts the symbolic PDE system into an algebraic-differential system the solver can handle. The solver then advances that discretized system in time. Finally, the Simulation object wraps the whole pipeline so that in normal use you do not need to manually call every processing step.

That object hierarchy is worth making explicit:

PyBaMM object	What it represents conceptually	What you usually do with it
<code>model</code>	The governing equations and variables	Choose DFN, SPM, SPMe, or another model
<code>ParameterValues</code>	Cell chemistry and numerical constants	Select or modify a parameter set
<code>geometry</code>	Domains and coordinates	Usually accept the model default at first
<code>mesh</code>	Spatial sampling of those domains	Adjust resolution when accuracy or speed matters
<code>discretisation</code>	Numerical translation of equations to algebraic form	Usually left implicit until you need control
<code>solver</code>	Time integration and nonlinear solves	Swap only when a problem demands it
<code>simulation</code>	A convenience wrapper over the full workflow	The main entry point for most users

This is the software version of the bridge from theory to tool. Textbook Chapter 8 gave you the physics, assumptions, and derivation. PyBaMM gives you an organized representation of those same ingredients. If you ever feel lost in the library API, ask: which part of the mathematical model does this object correspond to?

There is a second conceptual bridge that matters just as much. In theory class, it is easy to talk about “the voltage” as if it were one simple number produced by the model. In PyBaMM, voltage is a composition of interpretable contributors. You can ask for terminal voltage, open-circuit voltage, reaction overpotentials, electrolyte ohmic losses, concentration overpotentials, and internal concentrations separately. This is not software trivia. It is the practical expression of the decomposition you learned in the textbook. When a simulation result looks odd, the first serious question is not “did the code work?” It is “which physical contribution is driving this shape?”

One more point is worth stating explicitly before we touch code. A `Simulation` object is convenient, but it can hide the layers underneath. That convenience is good for productivity, but bad for learning if you let it stay magical. In this chapter we will use `Simulation` heavily because that is the right tool for most work. But we will also open the box once so you see the manual pipeline. Later chapters on parameters, experiments, and reproducibility will make much more sense if you know what `Simulation` is saving you from having to do by hand.

So the mental map for the chapter is this: Textbook Chapter 8 tells you what the DFN is. PyBaMM tells you how to hold that DFN in software. The rest of the chapter is about learning to use that software representation without losing contact with the physics.

3.5 Guided Walkthrough 1: Meet the PyBaMM Object Hierarchy

Learning objective: Instantiate a DFN model, inspect the objects underneath `Simulation`, and understand what PyBaMM is building before it solves anything.

We will start by making the object hierarchy concrete. Most PyBaMM tutorials quite reasonably jump straight to `Simulation`. For a first working notebook that is fine, but for a methods book it is too opaque. We will briefly look at both the high-level and low-level workflows so that later chapters do not feel magical.

3.5.1 Walkthrough 1 code

```
import numpy as np
import pybamm

# Create the symbolic model and attach a real parameterization.
model = pybamm.lithium_ion.DFN()
parameter_values = pybamm.ParameterValues("Chen2020")

print("PyBaMM version:", pybamm.__version__)
print("Model name:", model.name)
print("Default geometry domains:", list(model.default_geometry.keys()))
print("Default solver:", type(model.default_solver).__name__)
print("Nominal capacity [A.h]:", parameter_values["Nominal cell capacity"]
      ↪ [A.h]])
print("Lower voltage cut-off [V]:", parameter_values["Lower voltage cut-off"]
      ↪ [V]])
print("Upper voltage cut-off [V]:", parameter_values["Upper voltage cut-off"]
      ↪ [V]])

geometry = model.default_geometry
parameter_values.process_model(model)
parameter_values.process_geometry(geometry)

# Keep the first teaching mesh intentionally modest.
var_pts = {
```

```

    "x_n": 10,
    "x_s": 10,
    "x_p": 10,
    "r_n": 10,
    "r_p": 10,
}

mesh = pybamm.Mesh(geometry, model.default_submesh_types, var_pts)
discretisation = pybamm.Discretisation(mesh, model.default_spatial_methods)
discretisation.process_model(model)

# Solve the discretized model directly, without the Simulation wrapper.
solver = model.default_solver
evaluation_times_s = np.linspace(0, 600, 51)
manual_solution = solver.solve(model, evaluation_times_s)

manual_voltage_v = manual_solution["Terminal voltage [V]"](evaluation_times_s)

print("Voltage at 600 s from the manual pipeline [V]:",
      ↪ round(float(manual_voltage_v[-1]), 4))

```

3.5.2 Walkthrough 1 explanation

The first two lines create the core objects of the workflow. `pybamm.lithium_ion.DFN()` gives us the symbolic DFN model itself. `pybamm.ParameterValues("Chen2020")` selects a built-in lithium-ion parameter set associated with the LG M50 cell described in Chen et al. (2020). This is standard practice in teaching and benchmarking because it gives us a parameter set that is already internally consistent.

The next group of print statements is not throwaway code. We are making the object hierarchy visible. `model.name` confirms which model we instantiated. `model.default_geometry.keys()` shows the physical domains that the DFN spans: negative electrode, separator, positive electrode, negative particle, positive particle, and current collector. `model.default_solver` tells us which solver PyBaMM would use by default if we delegated the solve to `Simulation`. On the pinned version that solver is `IDAKLUSolver`, which is a good default for stiff battery-model systems.

The `process_model` and `process_geometry` calls are where parameters stop being abstract values in a dictionary and start being substituted into the symbolic model and geometry. This is the first point where the mathematical object becomes chemistry-specific.

The `var_pts` dictionary controls the mesh resolution. The meanings are worth committing to memory now: `x_n`, `x_s`, and `x_p` are through-thickness points in the negative electrode, separator, and positive electrode, while `r_n` and `r_p` are radial points in the negative and positive particles. If you reduce these values, the simulation becomes faster but less accurate. If you increase them, the reverse happens. Later chapters will reuse this exact pattern when we talk about mesh sensitivity and computational cost.

`pybamm.Mesh(...)` builds the mesh from the geometry and the default submesh types. `pybamm.Discretisation(...)` constructs the numerical machinery for turning sym-

bolic PDEs into algebraic form. `discretisation.process_model(model)` is the point where the model becomes numerically solvable.

Only after those steps do we get to the solver. `solver.solve(model, evaluation_times_s)` numerically advances the discretized model over the requested time grid. The final two lines extract terminal voltage from the resulting solution and evaluate it at the grid we asked for.

3.5.3 Walkthrough 1 expected output

On the pinned version you should see output of this form:

```
PyBaMM version: 25.12.1
Model name: Doyle-Fuller-Newman model
Default geometry domains: ['negative electrode', 'separator', 'positive
↪ electrode', 'negative particle', 'positive particle', 'current collector']
Default solver: IDAKLUSolver
Nominal capacity [A.h]: 5.0
Lower voltage cut-off [V]: 2.5
Upper voltage cut-off [V]: 4.2
Voltage at 600 s from the manual pipeline [V]: 3.8171
```

The important thing is not memorizing the fourth decimal place. It is recognizing what the output proves. You have instantiated a DFN, attached a real parameter set, processed the geometry, created a mesh, discretized the model, and solved it directly through the lower-level API. That is the full theory-to-numerics chain in one short block of code.

3.5.4 Walkthrough 1 troubleshooting

1. `KeyError` when accessing a parameter name
Symptom: one of the string keys in `parameter_values[...]` fails.
Fix: parameter names in PyBaMM are exact string keys. Copy them carefully. Later chapters will show safer ways to inspect and modify parameter sets.
2. A mesh-related error during `pybamm.Mesh(...)`
Symptom: PyBaMM complains about missing or incompatible variable-point definitions.
Fix: make sure the keys in `var_pts` are exactly `x_n`, `x_s`, `x_p`, `r_n`, and `r_p` for this model.
3. A solver error on the manual pipeline but not on the high-level `Simulation` workflow
Symptom: the low-level path feels brittle.
Fix: that is exactly why `Simulation` exists. The manual pipeline is for understanding and for specialized control, not because you should build every notebook this way.

3.5.5 Walkthrough 1 reflection

This exercise teaches the first nontrivial PyBaMM habit: the library is not a black box that takes current in and voltage out. It is a structured numerical representation of the theory you learned in Textbook Chapter 8. That mindset will matter every time you later adjust parameters, experiments, meshes, or model fidelity.

3.6 Guided Walkthrough 2: Run a Default DFN Simulation and Interpret the Big Curves

Learning objective: Solve a built-in DFN model with Chen2020, extract the major outputs, and interpret terminal voltage, discharge capacity, and approximate SOC correctly.

Now we will switch to the high-level workflow you will use most of the time. This is where PyBaMM becomes productive. The goal is not only to run the simulation, but to read it properly. A correct plot is not useful unless you know what it means physically.

3.6.1 Walkthrough 2 code

```
import numpy as np
import matplotlib.pyplot as plt
import pybamm

# The high-level Simulation object wraps the full PyBaMM pipeline.
model = pybamm.lithium_ion.DFN()
parameter_values = pybamm.ParameterValues("Chen2020")
simulation = pybamm.Simulation(model, parameter_values=parameter_values)

# Report on a clean one-hour grid.
evaluation_times_s = np.linspace(0, 3600, 361)
solution = simulation.solve(evaluation_times_s)

time_s = evaluation_times_s
time_min = time_s / 60.0

# Pull out the main variables we want to interpret.
terminal_voltage_v = solution["Terminal voltage [V]"](time_s)
open_circuit_voltage_v = solution["Battery open-circuit voltage [V]"](time_s)
discharge_capacity_ah = solution["Discharge capacity [A.h]"](time_s)
nominal_capacity_ah = parameter_values["Nominal cell capacity [A.h]"]
approximate_soc = 1.0 - discharge_capacity_ah / nominal_capacity_ah

print("Initial terminal voltage [V]:", round(float(terminal_voltage_v[0]), 4))
print("Terminal voltage at 3600 s [V]:", round(float(terminal_voltage_v[-1]),
↪ 4))
print("Discharge capacity after 3600 s [A.h]:",
↪ round(float(discharge_capacity_ah[-1]), 4))
print("Approximate SOC after 3600 s [-]:", round(float(approximate_soc[-1]),
↪ 4))

fig, axes = plt.subplots(3, 1, figsize=(9, 10), sharex=True,
↪ constrained_layout=True)

axes[0].plot(time_min, terminal_voltage_v, linewidth=2.2, label="Terminal
↪ voltage")
axes[0].plot(time_min, open_circuit_voltage_v, linewidth=1.8, linestyle="--",
↪ label="Battery OCV")
```



```

axes[0].set_ylabel("Voltage [V]")
axes[0].set_title("DFN discharge with the Chen2020 parameter set")
axes[0].legend()
axes[0].grid(True, alpha=0.3)

axes[1].plot(time_min, discharge_capacity_ah, color="tab:green",
             linewidth=2.2)
axes[1].set_ylabel("Discharge Capacity [A.h]")
axes[1].grid(True, alpha=0.3)

axes[2].plot(time_min, approximate_soc, color="tab:red", linewidth=2.2)
axes[2].set_ylabel("Approximate SOC [-]")
axes[2].set_xlabel("Time [min]")
axes[2].grid(True, alpha=0.3)

plt.show()

```

3.6.2 Walkthrough 2 explanation

The first three lines are now familiar: instantiate the DFN, select a parameter set, and wrap the workflow in Simulation. This is the pattern you should expect to reuse constantly.

`evaluation_times_s = np.linspace(0, 3600, 361)` gives us a clean one-second-like reporting grid at ten-second spacing. PyBaMM may still use adaptive internal time stepping underneath, but this line tells the processed variables where we want the solution evaluated for plotting and downstream analysis.

`solution["Terminal voltage [V]"](time_s)` is the first PyBaMM pattern worth committing to muscle memory. A solution variable is not just a stored array. It is a processed variable that can be evaluated on the time grid you want, and later on spatial coordinates as well. This is one of the reasons PyBaMM feels more expressive than a hand-built ODE script.

We also extract Battery open-circuit voltage [V]. This is a very useful comparison curve because it separates equilibrium thermodynamics from dynamic losses. The gap between OCV and terminal voltage is not random. It is the accumulated effect of kinetic, ohmic, and concentration losses.

The Discharge capacity [A.h] variable is exactly what its name suggests: how much capacity has been drawn since the simulation began. We then compute an approximate SOC using

$$\text{SOC}_{\text{approx}}(t) = 1 - \frac{Q_{\text{discharged}}(t)}{Q_{\text{nominal}}}. \quad (4)$$

That approximation is fine for a first DFN walkthrough, but it is still an approximation. In later chapters and in real BMS work, SOC estimation becomes more subtle than discharged-capacity bookkeeping. I am flagging that now so you do not quietly internalize Equation (4) as a universal truth.

The plotting section deliberately puts the variables in separate axes rather than overloading one panel. For a research notebook, clarity beats compactness.

3.6.3 Walkthrough 2 expected output

The printed values on the pinned version should be approximately:

```
Initial terminal voltage [V]: 4.0377
Terminal voltage at 3600 s [V]: 2.9830
Discharge capacity after 3600 s [A.h]: 4.9381
Approximate SOC after 3600 s [-]: 0.0124
```

The first plot should show terminal voltage starting a little above 4.03 V and declining steadily toward about 2.98 V over the hour. The dashed OCV curve should sit above the terminal-voltage curve almost everywhere. That gap is physically meaningful: it is the sum of losses that appear only under load.

The second plot should be almost a straight line rising from 0 to just under 5 A.h, because the default Chen2020 setup is effectively a constant-current discharge over this interval.

The third plot should fall nearly linearly from 1 toward zero, ending just above zero rather than exactly at zero because the one-hour window is very close to, but not identical with, a complete nominal-capacity discharge for this particular model trajectory.

If your terminal-voltage curve is jagged, increases during discharge, or crosses above the OCV curve, something is wrong.

3.6.4 Walkthrough 2 troubleshooting

1. `KeyError` for a variable name

Symptom: `solution["Battery open-circuit voltage [V]"]` or a similar lookup fails.

Fix: variable names are exact. Use the strings shown here exactly, including units.

2. Confusing terminal voltage with OCV

Symptom: you interpret the dashed curve as “what the battery is doing” and ignore the loaded terminal voltage.

Fix: remember Textbook Chapter 3. OCV is an equilibrium quantity. Terminal voltage is what a pack controller or experiment actually sees under load.

3. Misreading the approximate SOC

Symptom: you assume it is a model-provided ground truth SOC state.

Fix: in this walkthrough it is simply discharged capacity normalized by nominal capacity. That is fine for teaching, but it is not the last word on SOC.

3.6.5 Walkthrough 2 reflection

This exercise teaches the basic rhythm of PyBaMM work: instantiate, solve, extract processed variables, and interpret them in physical language rather than just visually. We will reuse this pattern in every later PyBaMM chapter.

3.7 Guided Walkthrough 3: Open the Cell and Inspect Internal Variables

Learning objective: Extract internal concentrations, overpotentials, and spatial profiles from the DFN so that the model stops being “a voltage generator” and becomes a physically interpretable virtual cell.

This is the most important conceptual move in the chapter. Terminal voltage is the public face of the model. Internal variables are where the research value lives. If you only ever read voltage, you are using a physics-based model like a fancier equivalent circuit.

3.7.1 Walkthrough 3 code

```
import numpy as np
import matplotlib.pyplot as plt
import pybamm

model = pybamm.lithium_ion.DFN()
parameter_values = pybamm.ParameterValues("Chen2020")
parameter_values.update({"Current function [A]": 5.0})

simulation = pybamm.Simulation(model, parameter_values=parameter_values)

# Stop halfway through the nominal discharge so internal states stay easy to
↪ read.
evaluation_times_s = np.linspace(0, 1800, 301)
solution = simulation.solve(evaluation_times_s)

time_min = evaluation_times_s / 60.0

terminal_voltage_v = solution["Terminal voltage [V]"](evaluation_times_s)
battery_ocv_v = solution["Battery open-circuit voltage
↪ [V]"](evaluation_times_s)
xavg_electrolyte_concentration = solution["X-averaged electrolyte
↪ concentration [mol.m-3]"](evaluation_times_s)
xavg_negative_surface_concentration = solution[
    "X-averaged negative particle surface concentration [mol.m-3]"
](evaluation_times_s)
xavg_positive_surface_concentration = solution[
    "X-averaged positive particle surface concentration [mol.m-3]"
](evaluation_times_s)
xavg_reaction_overpotential_v = solution["X-averaged battery reaction
↪ overpotential [V]"](evaluation_times_s)
xavg_concentration_overpotential_v = solution["X-averaged battery
↪ concentration overpotential [V]"](evaluation_times_s)

# Use the parameter-set geometry so our spatial coordinates are valid.
negative_electrode_thickness_m = parameter_values["Negative electrode
↪ thickness [m]"]
separator_thickness_m = parameter_values["Separator thickness [m]"]
```

```

positive_electrode_thickness_m = parameter_values["Positive electrode
↪ thickness [m]"]
negative_particle_radius_m = parameter_values["Negative particle radius [m]"]

cell_length_m = (
    negative_electrode_thickness_m
    + separator_thickness_m
    + positive_electrode_thickness_m
)

x_positions_m = np.linspace(0.0, cell_length_m, 200)
r_positions_m = np.linspace(0.0, negative_particle_radius_m, 150)

electrolyte_profile_start = solution["Electrolyte concentration
↪ [mol.m-3]"](x=x_positions_m, t=0.0)
electrolyte_profile_end = solution["Electrolyte concentration
↪ [mol.m-3]"](x=x_positions_m, t=1800.0)

# Radial concentration inside an x-averaged negative particle.
negative_particle_profile_start = solution[
    "X-averaged negative particle concentration [mol.m-3]"
](r=r_positions_m, t=0.0)
negative_particle_profile_end = solution[
    "X-averaged negative particle concentration [mol.m-3]"
](r=r_positions_m, t=1800.0)

fig, axes = plt.subplots(2, 2, figsize=(12, 9), constrained_layout=True)

axes[0, 0].plot(time_min, terminal_voltage_v, linewidth=2.2, label="Terminal
↪ voltage")
axes[0, 0].plot(time_min, battery_ocv_v, linewidth=1.8, linestyle="--",
↪ label="Battery OCV")
axes[0, 0].set_title("Voltage-level variables")
axes[0, 0].set_xlabel("Time [min]")
axes[0, 0].set_ylabel("Voltage [V]")
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3)

axes[0, 1].plot(
    time_min,
    xavg_electrolyte_concentration,
    linewidth=2.2,
    label="X-averaged electrolyte concentration",
)
axes[0, 1].plot(
    time_min,
    xavg_negative_surface_concentration,
    linewidth=2.0,
    label="Negative particle surface concentration",
)
axes[0, 1].plot(
    time_min,

```

```

    xavg_positive_surface_concentration,
    linewidth=2.0,
    label="Positive particle surface concentration",
)
axes[0, 1].set_title("Concentration-level variables")
axes[0, 1].set_xlabel("Time [min]")
axes[0, 1].set_ylabel("Concentration [mol.m$^{-3}$]")
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

axes[1, 0].plot(
    time_min,
    xavg_reaction_overpotential_v * 1000.0,
    linewidth=2.2,
    label="Reaction overpotential",
)
axes[1, 0].plot(
    time_min,
    xavg_concentration_overpotential_v * 1000.0,
    linewidth=2.2,
    label="Concentration overpotential",
)
axes[1, 0].set_title("Overpotential contributions")
axes[1, 0].set_xlabel("Time [min]")
axes[1, 0].set_ylabel("Overpotential [mV]")
axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3)

axes[1, 1].plot(
    x_positions_m * 1e6,
    electrolyte_profile_start,
    linewidth=2.0,
    label="Electrolyte concentration at 0 min",
)
axes[1, 1].plot(
    x_positions_m * 1e6,
    electrolyte_profile_end,
    linewidth=2.0,
    label="Electrolyte concentration at 30 min",
)
axes[1, 1].set_title("Through-thickness electrolyte profile")
axes[1, 1].set_xlabel("Position through cell [ $\mu\text{m}$ ]")
axes[1, 1].set_ylabel("Electrolyte Concentration [mol.m$^{-3}$]")
axes[1, 1].legend()
axes[1, 1].grid(True, alpha=0.3)

plt.show()

fig, ax = plt.subplots(figsize=(8, 5), constrained_layout=True)
ax.plot(
    r_positions_m * 1e6,
    negative_particle_profile_start,

```

```

        linewidth=2.0,
        label="Negative particle concentration at 0 min",
    )
ax.plot(
    r_positions_m * 1e6,
    negative_particle_profile_end,
    linewidth=2.0,
    label="Negative particle concentration at 30 min",
)
ax.set_title("X-averaged negative-particle radial concentration profile")
ax.set_xlabel("Particle radius coordinate [ $\mu\text{m}$ ]")
ax.set_ylabel("Concentration [ $\text{mol.m}^{-3}$ ]")
ax.legend()
ax.grid(True, alpha=0.3)
plt.show()

```

3.7.2 Walkthrough 3 explanation

We intentionally shorten the simulation window to 1800 s, which is a half-hour at approximately 1C for the nominal 5 Ah cell. That gives us a mid-discharge view where the model is still far from terminal cutoffs and the internal-state interpretation is cleaner.

The first group of variable extractions uses X-averaged quantities. This is an important PyBaMM naming pattern. When a quantity varies spatially, PyBaMM often provides both the full field and spatially averaged versions. For a first inspection, the averages are easier to interpret. Later, when you need to diagnose localization or gradients, you move to the full fields.

The overpotential variables are especially important. X-averaged battery reaction overpotential [V] captures kinetic losses associated with interfacial charge transfer. X-averaged battery concentration overpotential [V] captures the contribution of concentration nonuniformity to voltage loss. When the terminal voltage drops away from the OCV curve, these are two of the places to look.

The next block extracts actual spatial profiles. This is where PyBaMM begins to feel like a genuine virtual cell. We compute the total cell thickness from the negative electrode, separator, and positive electrode thicknesses. We then define a coordinate array `x_positions_m` spanning that thickness and evaluate Electrolyte concentration [mol.m^{-3}] at the start and at thirty minutes. Because we pass both `x=` and `t=`, PyBaMM returns the field values on exactly the spatial slice we asked for.

The negative-particle radial profile is even closer to the theory textbook. X-averaged negative particle concentration [mol.m^{-3}] is the radial diffusion field averaged across the negative electrode thickness. Evaluating it over `r_positions_m` at two times shows whether the particle interior and surface are diverging as diffusion struggles to keep up with intercalation or deintercalation.

3.7.3 Walkthrough 3 expected output

The first figure should have four panels.

The top-left panel should show terminal voltage below the battery OCV, with the gap widening modestly as the discharge proceeds. If the two curves lie exactly on top of each other, something is wrong physically: a loaded cell should not have zero losses.

The top-right panel should show the x-averaged electrolyte concentration drifting away from its initial value of roughly 1000 mol.m^{-3} , while the negative and positive particle surface concentrations move in opposite directions. Under discharge, lithium leaves the negative electrode and arrives at the positive, so the surface concentrations should reflect that directionality.

The bottom-left panel should show overpotentials in millivolts. The reaction overpotential should generally be the larger of the two over the first half hour, with the concentration overpotential present but smaller. The exact magnitudes depend on the parameter set and current, but they should be smooth, physically plausible curves rather than noisy oscillations.

The bottom-right panel should show the electrolyte concentration profile through the cell thickness. At $t = 0$, it should be nearly flat. At $t = 30 \text{ min}$, it should have developed a gradient: higher in one region, lower in another, reflecting ionic transport and source terms through the layered structure.

The second figure should show the negative-particle radial concentration profile. At the start of discharge it should be fairly flat. After thirty minutes it should be steeper, with a visible gradient between the particle center and surface. That is the DFN's solid-phase diffusion physics made visible.

3.7.4 Walkthrough 3 troubleshooting

1. nan values when evaluating a spatial profile

Symptom: the profile arrays are full of nan.

Fix: this usually means the coordinate values are outside the physical domain. Use the parameter-set geometry values, as we did here, instead of guessing coordinate ranges.

2. Confusing full fields with averaged fields

Symptom: you interpret an X-averaged quantity as though it described a local point.

Fix: PyBaMM variable names are intentionally descriptive. Read them literally. X-averaged means averaged through the cell thickness.

3. Overpotential units look too small or too large

Symptom: a curve appears to be only 0.01 units tall and feels meaningless.

Fix: remember the raw units are volts. For interpretability we multiplied them by 1000 to plot in millivolts.

3.7.5 Walkthrough 3 reflection

This exercise teaches the real payoff of physics-based models. You are no longer limited to asking, “what voltage does the model predict?” You can ask which physical sub-process is responsible for the voltage behavior and inspect the corresponding internal state directly.

3.8 Guided Walkthrough 4: Compare SPM, SPMe, and DFN on the Same Duty Cycle

Learning objective: Quantify the runtime-versus-fidelity tradeoff between SPM, SPMe, and DFN on a common current profile, and learn when each model is a defensible choice.

This is the modeling-judgment exercise of the chapter. Battery researchers often talk loosely about reduced-order models being “faster” or full-order models being “more accurate.” Those statements are true but not useful until you quantify them on the same task. We will do that carefully.

To avoid event-trigger effects near voltage cutoffs, we will compare all three models over a fixed 1800 s discharge at 5 A, then evaluate every model on the same reporting grid. That produces a cleaner instructional comparison than allowing each model to stop at its own terminal-voltage event.

3.8.1 Walkthrough 4 code

```
import time
import numpy as np
import matplotlib.pyplot as plt
import pybamm

def run_model_on_common_grid(model_factory, current_a, evaluation_times_s):
    parameter_values = pybamm.ParameterValues("Chen2020")
    parameter_values.update({"Current function [A]": current_a})

    simulation = pybamm.Simulation(model_factory(),
    ↪ parameter_values=parameter_values)

    # Measure runtime in the same way for every model.
    start_time = time.perf_counter()
    solution = simulation.solve(evaluation_times_s)
    runtime_s = time.perf_counter() - start_time

    # Evaluate all models on the same reporting grid before comparison.
    terminal_voltage_v = solution["Terminal voltage [V]"](evaluation_times_s)

    return {
        "runtime_s": runtime_s,
        "terminal_voltage_v": terminal_voltage_v,
    }

evaluation_times_s = np.linspace(0.0, 1800.0, 301)
current_a = 5.0

model_factories = {
    "SPM": pybamm.lithium_ion.SPM,
    "SPMe": pybamm.lithium_ion.SPMe,
    "DFN": pybamm.lithium_ion.DFN,
```



```

}

results = {}
for model_name, model_factory in model_factories.items():
    results[model_name] = run_model_on_common_grid(
        model_factory=model_factory,
        current_a=current_a,
        evaluation_times_s=evaluation_times_s,
    )

dfn_voltage_v = results["DFN"]["terminal_voltage_v"]

# Compute every reduced model's error against the DFN baseline.
for model_name in ["SPM", "SPMe"]:
    rms_error_v = np.sqrt(
        np.mean((results[model_name]["terminal_voltage_v"] - dfn_voltage_v) **
        ↪ 2)
    )
    results[model_name]["rms_error_mV_vs_DFN"] = 1000.0 * rms_error_v

results["DFN"]["rms_error_mV_vs_DFN"] = 0.0

for model_name, result in results.items():
    print(
        f"{model_name:4s} | runtime = {result['runtime_s']:.4f} s"
        f" | RMS error vs DFN = {result['rms_error_mV_vs_DFN']:.3f} mV"
    )

time_min = evaluation_times_s / 60.0

fig, axes = plt.subplots(1, 2, figsize=(12, 5), constrained_layout=True)

for model_name, result in results.items():
    axes[0].plot(
        time_min,
        result["terminal_voltage_v"],
        linewidth=2.2,
        label=model_name,
    )

axes[0].set_title("Voltage prediction by model fidelity")
axes[0].set_xlabel("Time [min]")
axes[0].set_ylabel("Terminal Voltage [V]")
axes[0].legend()
axes[0].grid(True, alpha=0.3)

bar_positions = np.arange(len(results))
runtime_values = [results[name]["runtime_s"] for name in ["SPM", "SPMe",
↪ "DFN"]]
error_values = [results[name]["rms_error_mV_vs_DFN"] for name in ["SPM",
↪ "SPMe", "DFN"]]

```

```

axes[1].bar(bar_positions - 0.18, runtime_values, width=0.36, label="Runtime
↪ [s]")
axes[1].bar(bar_positions + 0.18, error_values, width=0.36, label="RMS error
↪ [mV]")
axes[1].set_xticks(bar_positions)
axes[1].set_xticklabels(["SPM", "SPMe", "DFN"])
axes[1].set_title("Runtime versus DFN-relative error")
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.show()

```

3.8.2 Walkthrough 4 explanation

The helper function `run_model_on_common_grid` exists for one reason: it forces the comparison to be methodologically clean. Every model receives the same current and the same evaluation grid. The only thing that changes is the model fidelity.

Inside that helper, we update the `Chen2020` parameter set with a fixed current of 5 A. That update pattern is the first one you should internalize for later parameter-study work. We are not editing `PyBaMM` source code. We are modifying a parameter object. `time.perf_counter()` is a practical runtime-measurement tool. The absolute numbers you see will depend on your machine, operating system, and whether this is the first solve in a fresh kernel. The relative ordering is the part that matters scientifically. After running all three models, we compute an RMS voltage error against the DFN:

$$\text{RMS}(m) = \sqrt{\frac{1}{N} \sum_{i=1}^N (V_m(t_i) - V_{\text{DFN}}(t_i))^2}. \quad (5)$$

This is not the only possible accuracy metric, but it is a transparent and defensible first one. We express it in millivolts because that is the scale on which battery-model voltage differences become intuitively interpretable.

The two-panel figure then makes the tradeoff visible: the left plot shows voltage predictions themselves, while the right plot puts runtime and error side by side.

3.8.3 Walkthrough 4 expected output

On the pinned version, a representative run is:

```

SPM   | runtime = 0.2527 s | RMS error vs DFN = 54.177 mV
SPMe  | runtime = 0.5038 s | RMS error vs DFN = 3.826 mV
DFN   | runtime = 1.3298 s | RMS error vs DFN = 0.000 mV

```

Your runtimes will vary, sometimes substantially, but the ordering should remain the same: SPM fastest, DFN slowest, SPMe in between. The accuracy trend should also be stable: SPMe should sit much closer to the DFN than the plain SPM on this moderate-rate discharge.

In the left plot, the SPMe voltage curve should be visually almost on top of the DFN curve, while the SPM should show a visibly larger deviation, especially as the discharge proceeds. In the right plot, the bars should make the core lesson obvious: the SPMe

buys a large error reduction at a modest runtime penalty relative to SPM, while still remaining faster than the DFN.

3.8.4 Walkthrough 4 interpreting the modeling choice

This is where we connect the software output back to research judgment.

Use DFN when internal-state fidelity matters, when you need mechanistic interpretation, when you are testing parameter sensitivity in a way that depends on electrolyte dynamics, or when you need a high-fidelity virtual cell to generate synthetic data for another workflow.

Use SPMe when you still care about electrolyte effects but need cheaper repeated solves. This is often the sweet spot for parameter studies, control-oriented simulation, and mid-fidelity research questions.

Use SPM when you need speed more than detailed fidelity and when the operating conditions are gentle enough that electrolyte limitations are not the dominant story. That last phrase matters. SPM is not “the beginner model.” It is a deliberate approximation with a domain of validity.

3.8.5 Walkthrough 4 troubleshooting

1. Treating runtime as a universal benchmark

Symptom: you compare your exact runtime numbers to a colleague’s machine or to this chapter and overinterpret the difference.

Fix: use runtime comparatively within one environment, not as a universal hardware-independent truth.

2. Comparing models on different time grids

Symptom: you accidentally compare arrays with different sampling or different terminal events.

Fix: evaluate every model on a common reporting grid before computing an error metric.

3. Declaring the SPMe “always better” without qualification

Symptom: you learn the ranking from this example and generalize too far.

Fix: model choice depends on operating regime, chemistry, question, and acceptable error. This walkthrough is a benchmark, not a universal law.

3.8.6 Walkthrough 4 reflection

This exercise teaches a habit that publishable work depends on: model fidelity should be chosen by evidence and task fit, not by prestige. The “best” battery model is the one that is accurate enough for the question while still practical enough for the workflow.

3.9 Dataset Integration: Parsing a Public Battery Dataset Before Chapter 4

This chapter is primarily about your first PyBaMM simulation, not about full experiment scripting. That deeper workflow belongs in Chapter 4. But battery-modeling

competence also requires comfort with public experimental data, so we will do one light but real dataset integration here.

We will use the **Oxford Energy Trading Battery Degradation Dataset**:

- Dataset landing page: <https://ora.ox.ac.uk/objects/uuid:9aae61af-2949-49f1-8ad5-6aea448979e5>
- DOI: 10.5287/bodleian:gJPdDzvP4
- Example file for this chapter: SPM_cell1_profileData.csv
- File size: approximately 2.9 MB
- Format: CSV without a header row
- License: ODC Open Database License (ODbL)

The Oxford Readme.txt explains that SPM_cell1_profileData.csv contains five columns:

1. profile time in seconds
2. current in amperes, with **negative meaning charging**
3. cell voltage in volts
4. cell surface temperature in degrees Celsius, where 0 means missing
5. ambient temperature in degrees Celsius, where 0 means missing

That sign convention is the first pitfall to notice. In many BMS and ECM contexts, discharge current is treated as positive. In this Oxford dataset, charging is negative. You must always decide whether to keep the native sign convention or map it to the one used in your model. For this chapter we will keep the dataset unchanged and only compute helper columns.

3.9.1 Dataset parsing code

```
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

csv_path = Path("data/raw/SPM_cell1_profileData.csv")

# The Oxford file has no header row, so we assign names manually.
profile_df = pd.read_csv(
    csv_path,
    header=None,
    names=[
        "profile_time_s",
        "current_a",
        "voltage_v",
        "surface_temperature_c",
        "ambient_temperature_c",
    ],
)

# Convert missing temperatures encoded as zero into proper NaN values.
```

```

profile_df["surface_temperature_c"] =
    ↪ profile_df["surface_temperature_c"].replace(0, np.nan)
profile_df["ambient_temperature_c"] =
    ↪ profile_df["ambient_temperature_c"].replace(0, np.nan)
profile_df["profile_time_h"] = profile_df["profile_time_s"] / 3600.0
profile_df["discharge_current_a"] = -profile_df["current_a"]

print(profile_df.head())
print(profile_df.describe())

window_df = profile_df.iloc[:2000].copy()

fig, axes = plt.subplots(3, 1, figsize=(10, 9), sharex=True,
    ↪ constrained_layout=True)

axes[0].plot(window_df["profile_time_h"], window_df["current_a"],
    ↪ linewidth=1.5)
axes[0].set_ylabel("Current [A]")
axes[0].set_title("Oxford SPM cell profile data: first window")
axes[0].grid(True, alpha=0.3)

axes[1].plot(window_df["profile_time_h"], window_df["voltage_v"],
    ↪ linewidth=1.5, color="tab:orange")
axes[1].set_ylabel("Voltage [V]")
axes[1].grid(True, alpha=0.3)

axes[2].plot(
    window_df["profile_time_h"],
    window_df["surface_temperature_c"],
    linewidth=1.5,
    label="Surface temperature",
)
axes[2].plot(
    window_df["profile_time_h"],
    window_df["ambient_temperature_c"],
    linewidth=1.5,
    label="Ambient temperature",
)
axes[2].set_ylabel("Temperature [°C]")
axes[2].set_xlabel("Profile Time [h]")
axes[2].legend()
axes[2].grid(True, alpha=0.3)

plt.show()

```

3.9.2 How to read the parsed data

The `header=None` argument matters because the file does not ship with column names. If you forget that, Pandas will quietly treat the first data row as a header and your whole dataset will shift.

Replacing zeros in the temperature columns with NaN is a small but important data-cleaning move. The Oxford readme explicitly states that 0 indicates a missing mea-

surement, not a real 0°C environment. If you skip that replacement, later averages and plots become physically misleading.

`discharge_current_a = -current_a` does not alter the raw data. It creates a convenience column in the sign convention many battery models use. That is a good research habit. Preserve the raw field, add a translated field, and document the choice.

3.9.3 Dataset parsing expected output

The `head()` output should show five raw numeric columns plus the helper columns we added. The plotted window should show a stepped current trace, a corresponding voltage trace constrained within the Kokam cell’s operating window, and temperature curves that move much more slowly than current or voltage.

The important conceptual lesson is simple: real battery files arrive with conventions and quirks. Even before we drive a PyBaMM simulation with public profiles in Chapter 4, we need to be able to read the file, label the units, handle missing values correctly, and state the sign convention without hesitation.

3.10 Guided Walkthrough 5: Reproduce a Published Figure from Marquis et al. (2019)

Learning objective: Reproduce the qualitative structure of a published SPMe-versus-DFN error-decomposition figure using PyBaMM’s `Marquis2019` parameter set, while documenting the modeling choices and ambiguities honestly.

This is the highest-value exercise in the chapter. The paper is:

Scott G. Marquis, Valentin Sulzer, Robert Timms, Colin P. Please, and S. Jon Chapman, “An asymptotic derivation of a single particle model with electrolyte,” *Journal of The Electrochemical Society*, 166(15), A3693-A3706, 2019. DOI: 10.1149/2.0341915jes.

The target is **Figure 3** in the accepted manuscript hosted by the Oxford University Research Archive. In the paper, that figure shows component-wise voltage errors between the SPMe and the DFN during a 3C discharge.

We need to be transparent about three facts before we begin.

First, PyBaMM now ships a built-in `Marquis2019` parameter set and a modern SPMe implementation, which is exactly what makes this exercise feasible on a laptop.

Second, the paper’s notation for the voltage decomposition does not map one-to-one onto PyBaMM variable names in a way the authors spelled out as a modern notebook-ready script. So we must make a principled mapping.

Third, the paper terminated the comparison at 3.2 V, while the bundled `Marquis2019` lower-voltage cutoff inside PyBaMM differs. We will therefore impose the paper’s 3.2 V cutoff explicitly in the experiment.

That is what honest reproduction looks like. We do not hide the ambiguities. We surface them and document the choices.

3.10.1 The variable mapping we will use

This is an informed mapping from the paper's plotted components to PyBaMM variables:

Paper notation	PyBaMM variable used here	Why this is a defensible mapping
$\bar{U}_{eq}$	Battery open-circuit voltage [V]	Represents the equilibrium open-circuit contribution
$\bar{\eta}_r$	X-averaged battery reaction overpotential [V]	Represents kinetic charge-transfer losses
$\bar{\Delta\Phi}_{Elec}$	X-averaged battery electrolyte ohmic losses [V]	Represents electrolyte-phase voltage drop
$\bar{\eta}_c$	X-averaged battery concentration overpotential [V]	Represents concentration-driven voltage loss

This mapping is an inference from the paper and PyBaMM's documented variable names. It is not a claim that we are using the authors' original private post-processing code line for line.

3.10.2 Walkthrough 5 code

```
import numpy as np
import matplotlib.pyplot as plt
import pybamm

parameter_values = pybamm.ParameterValues("Marquis2019")
electrode_area_m2 = (
    parameter_values["Electrode height [m]"] * parameter_values["Electrode
↪ width [m]"]
)

# Match the paper's comparison regime explicitly.
experiment = pybamm.Experiment(["Discharge at 3C until 3.2 V"])

models = {
    "SPMe": pybamm.lithium_ion.SPMe(),
    "DFN": pybamm.lithium_ion.DFN(),
}

component_variables = {
    "Battery OCV error": "Battery open-circuit voltage [V]",
    "Reaction overpotential error": "X-averaged battery reaction overpotential
↪ [V]",
    "Electrolyte ohmic error": "X-averaged battery electrolyte ohmic losses
↪ [V]",
}
```

```

        "Concentration overpotential error": "X-averaged battery concentration
        ↪ overpotential [V]",
    }

results = {}

for model_name, model in models.items():
    simulation = pybamm.Simulation(
        model,
        parameter_values=pybamm.ParameterValues("Marquis2019"),
        experiment=experiment,
    )
    solution = simulation.solve()

    result = {
        "time_s": solution["Time [s]"].entries,
        "discharge_capacity_ah_per_m2": (
            solution["Discharge capacity [A.h]"].entries / electrode_area_m2
        ),
    }

    for label, variable_name in component_variables.items():
        result[label] = solution[variable_name].entries

    results[model_name] = result

# Align the comparison on a common time axis before subtraction.
common_time_s = np.linspace(
    0.0,
    min(results["SPMe"]["time_s"][-1], results["DFN"]["time_s"][-1]),
    400,
)

capacity_axis_ah_per_m2 = np.interp(
    common_time_s,
    results["DFN"]["time_s"],
    results["DFN"]["discharge_capacity_ah_per_m2"],
)

component_errors_v = {}

for label in component_variables:
    spme_component = np.interp(
        common_time_s,
        results["SPMe"]["time_s"],
        results["SPMe"][label],
    )
    dfn_component = np.interp(
        common_time_s,
        results["DFN"]["time_s"],
        results["DFN"][label],
    )

```



```

component_errors_v[label] = spme_component - dfn_component
print(
    label,
    "maximum absolute error [mV] =",
    round(float(np.max(np.abs(component_errors_v[label])) * 1000.0), 3),
)

fig, ax = plt.subplots(figsize=(9, 5.5), constrained_layout=True)

for label, error_v in component_errors_v.items():
    ax.plot(
        capacity_axis_ah_per_m2,
        error_v * 1000.0,
        linewidth=2.0,
        label=label,
    )

ax.axhline(0.0, color="black", linewidth=0.8, alpha=0.5)
ax.set_xlabel("Discharge Capacity [A.h m$^{-2}$]")
ax.set_ylabel("SPMe - DFN component error [mV]")
ax.set_title("Reproduction of Marquis et al. (2019), Figure 3 structure")
ax.legend()
ax.grid(True, alpha=0.3)

plt.show()

```

3.10.3 Walkthrough 5 explanation

We begin with `ParameterValues("Marquis2019")`, because PyBaMM includes a parameter set specifically tied to the Marquis SPMe work. That is exactly the kind of alignment you want when reproducing a paper: use the paper-linked parameterization when the tool makes it available.

The electrode area is important because the paper's x-axis is expressed in discharge capacity per unit area. PyBaMM gives us discharge capacity in ampere-hours, so we divide by the electrode area to match the paper's scale:

$$Q_{\text{area}} = \frac{Q_{\text{discharged}}}{A_{\text{electrode}}}. \quad (6)$$

We then define an explicit experiment: Discharge at 3C until 3.2 V. The reason to specify the voltage cutoff directly instead of relying on defaults is methodological honesty. The paper used 3.2 V, and we want to follow that published choice.

For each model, we solve once and store two types of data: the discharge-capacity axis and the four voltage components we mapped from the paper notation.

Because the SPMe and DFN do not stop at exactly identical times, we create `common_time_s` and interpolate both solutions onto that shared interval. This matters. Comparing the raw arrays directly would be sloppy because the models do not report on identical internal time grids.

Finally, we subtract the DFN component from the SPMe component. The resulting curves are not the total voltage error; they are the component-wise differences underlying that total error.

3.10.4 Walkthrough 5 expected output

On the pinned version, representative maximum absolute component errors are:

Battery OCV error maximum absolute error [mV] = 8.29

Reaction overpotential error maximum absolute error [mV] = 8.666

Electrolyte ohmic error maximum absolute error [mV] = 5.212

Concentration overpotential error maximum absolute error [mV] = 4.0

The plot should show four error curves living close to zero over most of the discharge, then separating more noticeably as the discharge approaches its end. The exact magnitudes and line ordering may not match the paper pixel for pixel, but the qualitative story should: the SPMe tracks the DFN well through most of the discharge, and the component-level error grows as the system approaches the harder, end-of-discharge regime.

3.10.5 Walkthrough 5 reproduction tolerance

This is the part many reproduction exercises fail to state clearly.

For this figure, “close enough” does **not** mean that every curve must match the published accepted-manuscript figure at the pixel level. We are using a modern PyBaMM implementation, modern defaults, and an inferred mapping between paper notation and exposed software variables. It is entirely reasonable for the exact magnitudes to differ.

For this figure, “close enough” means:

1. the same experiment definition is used
2. the same model pair is compared
3. the x-axis scaling is aligned with the paper
4. the qualitative trend is recovered: small component errors for most of the discharge and stronger divergence near the end
5. the methodological differences are written down, not hidden

That is a rigorous reproduction standard for a laptop-based methods chapter.

3.10.6 Walkthrough 5 troubleshooting

1. You compare raw arrays without interpolation
Symptom: the subtraction fails or produces misleading results because the models stopped at different times.
Fix: always align the comparison axis first.
2. You use the Chen2020 parameter set by habit
Symptom: the result no longer meaningfully targets the Marquis paper.
Fix: reproduction work should use the paper-linked parameterization whenever possible.
3. You claim exact quantitative agreement where you only have qualitative agreement

Symptom: the write-up sounds more certain than the method deserves.

Fix: be explicit about any inferred mapping or version drift. This is part of good research practice, not an admission of failure.

3.10.7 Walkthrough 5 reflection

This exercise teaches reproduction as a research method rather than a classroom puzzle. You identified a target figure, matched the experiment definition, found a defensible parameter set, documented a variable-mapping choice, handled nonidentical solution grids correctly, and defined a reasonable success criterion. That is publishable-work behavior.

3.11 Open-Ended Exercises

These exercises ask you to modify the guided code rather than merely rerun it. Try them before reading the worked solutions at the end of the chapter.

3.11.1 Exercise 1: How does model ranking change with current?

Repeat Guided Walkthrough 4 for 2.5 A, 5.0 A, and 10.0 A. Which model fidelity changes its error most dramatically as current increases? Does runtime change much relative to error?

Hints: keep the same comparison grid structure; put the current values in a loop; store RMS error versus DFN for each model and current.

3.11.2 Exercise 2: How sensitive is the DFN to mesh resolution?

Using the low-level pipeline from Guided Walkthrough 1, compare a coarse mesh of 5 points per domain, the teaching mesh of 10 points per domain, and a finer mesh of 20 points per domain. Report the runtime and terminal voltage at 600 s.

Hints: the only thing you need to change is `var_pts`; keep the model, parameters, and time grid the same; compare coarse and fine solutions against the 20-point run.

3.11.3 Exercise 3: What happens if you compare Marquis2019 and Chen2020 under the same model?

Run a DFN with Chen2020 and a DFN with Marquis2019 on the same nominal 1C discharge for 1800 s. Compare terminal voltage and x-averaged electrolyte concentration. What differences are due to chemistry and parameterization rather than model fidelity?

Hints: use the same code pattern as Guided Walkthrough 3; do not change the model, only the parameter set; remember that the two cells have different nominal capacities and geometry.

3.12 Worked Solutions to the Open-Ended Exercises

3.12.1 Solution to Exercise 1

```

import time
import numpy as np
import pybamm

def compare_models_at_current(current_a):
    evaluation_times_s = np.linspace(0.0, 1800.0, 301)
    model_factories = {
        "SPM": pybamm.lithium_ion.SPM,
        "SPMe": pybamm.lithium_ion.SPMe,
        "DFN": pybamm.lithium_ion.DFN,
    }

    results = {}
    for model_name, model_factory in model_factories.items():
        parameter_values = pybamm.ParameterValues("Chen2020")
        parameter_values.update({"Current function [A]": current_a})
        simulation = pybamm.Simulation(model_factory(),
        ↪ parameter_values=parameter_values)

        start_time = time.perf_counter()
        solution = simulation.solve(evaluation_times_s)
        runtime_s = time.perf_counter() - start_time

        results[model_name] = {
            "runtime_s": runtime_s,
            "voltage_v": solution["Terminal voltage [V]"](evaluation_times_s),
        }

    dfn_voltage_v = results["DFN"]["voltage_v"]
    for model_name in ["SPM", "SPMe"]:
        rms_error_v = np.sqrt(np.mean((results[model_name]["voltage_v"] -
        ↪ dfn_voltage_v) ** 2))
        results[model_name]["rms_error_mV"] = 1000.0 * rms_error_v

    results["DFN"]["rms_error_mV"] = 0.0
    return results

for current_a in [2.5, 5.0, 10.0]:
    current_results = compare_models_at_current(current_a)
    print(f"\nCurrent = {current_a:.1f} A")
    for model_name in ["SPM", "SPMe", "DFN"]:
        print(
            f"{model_name:4s} | runtime =
            ↪ {current_results[model_name]['runtime_s']:.4f} s"
            f" | RMS error = {current_results[model_name]['rms_error_mV']:.3f}
            ↪ mV"
        )

```

What you should find is that the SPM error grows much faster with current than the SPMe error. That is the point. As electrolyte limitations become more important, the plain single-particle approximation loses accuracy faster than the electrolyte-aware reduced model does.

3.12.2 Solution to Exercise 2

```
import time
import numpy as np
import pybamm

def run_manual_dfn_with_mesh(points_per_domain):
    model = pybamm.lithium_ion.DFN()
    parameter_values = pybamm.ParameterValues("Chen2020")
    geometry = model.default_geometry

    parameter_values.process_model(model)
    parameter_values.process_geometry(geometry)

    var_pts = {
        "x_n": points_per_domain,
        "x_s": points_per_domain,
        "x_p": points_per_domain,
        "r_n": points_per_domain,
        "r_p": points_per_domain,
    }

    mesh = pybamm.Mesh(geometry, model.default_submesh_types, var_pts)
    discretisation = pybamm.Discretisation(mesh, model.default_spatial_methods)
    discretisation.process_model(model)

    solver = model.default_solver
    evaluation_times_s = np.linspace(0.0, 600.0, 51)

    start_time = time.perf_counter()
    solution = solver.solve(model, evaluation_times_s)
    runtime_s = time.perf_counter() - start_time

    terminal_voltage_v = solution["Terminal voltage [V]"](evaluation_times_s)
    return runtime_s, float(terminal_voltage_v[-1])

for points in [5, 10, 20]:
    runtime_s, final_voltage_v = run_manual_dfn_with_mesh(points)
    print(
        f"points per domain = {points:2d} | runtime = {runtime_s:.4f} s"
        f" | voltage at 600 s = {final_voltage_v:.4f} V"
    )
```

You should see the finer mesh take longer. The final voltage should converge rather than wander wildly. If it does wander wildly, that is a sign you either changed more than the mesh or ran into a setup mistake.

3.12.3 Solution to Exercise 3

```

import numpy as np
import matplotlib.pyplot as plt
import pybamm

evaluation_times_s = np.linspace(0.0, 1800.0, 301)
parameter_sets = ["Chen2020", "Marquis2019"]
results = {}

for parameter_set_name in parameter_sets:
    parameter_values = pybamm.ParameterValues(parameter_set_name)
    simulation = pybamm.Simulation(
        pybamm.lithium_ion.DFN(),
        parameter_values=parameter_values,
    )
    solution = simulation.solve(evaluation_times_s)

    results[parameter_set_name] = {
        "terminal_voltage_v": solution["Terminal voltage
↪ [V]"](evaluation_times_s),
        "xavg_electrolyte_concentration": solution[
            "X-averaged electrolyte concentration [mol.m-3]"
        ](evaluation_times_s),
    }

time_min = evaluation_times_s / 60.0

fig, axes = plt.subplots(2, 1, figsize=(9, 8), sharex=True,
↪ constrained_layout=True)

for parameter_set_name in parameter_sets:
    axes[0].plot(
        time_min,
        results[parameter_set_name]["terminal_voltage_v"],
        linewidth=2.0,
        label=parameter_set_name,
    )
    axes[1].plot(
        time_min,
        results[parameter_set_name]["xavg_electrolyte_concentration"],
        linewidth=2.0,
        label=parameter_set_name,
    )

axes[0].set_ylabel("Terminal Voltage [V]")
axes[0].set_title("Same DFN model, different parameter sets")
axes[0].legend()
axes[0].grid(True, alpha=0.3)

axes[1].set_ylabel("X-avg Electrolyte Concentration [mol.m$^{-3}$]")
axes[1].set_xlabel("Time [min]")

```

```
axes[1].legend()
axes[1].grid(True, alpha=0.3)
```

```
plt.show()
```

The main lesson is that “the model” is not the whole story. A DFN with one parameter set is not interchangeable with a DFN using another. Parameterization changes operating window, dynamic losses, concentration gradients, and therefore the interpretation of any later fitted or benchmarked result.

3.13 What Changes for Sodium-Ion?

Early in the companion, this section is intentionally brief, but it matters.

The PyBaMM workflow itself changes very little for sodium-ion. The model object, the parameter-value pattern, the solve-extract-plot rhythm, and the fidelity comparison logic all transfer directly. What changes is the chemistry-specific content that fills those objects.

First, sodium-ion parameter sets are much sparser than lithium-ion parameter sets in the public PyBaMM ecosystem. That means you will often need to build or adapt your own `ParameterValues` object from literature rather than relying on a bundled cell.

Second, sodium-ion open-circuit curves, especially for hard-carbon negative electrodes, can be flatter and more structured than typical lithium-ion graphite curves. That affects both the interpretation of terminal-voltage differences and the difficulty of parameter estimation and state estimation later in the manual.

Third, validation is harder because public sodium-ion datasets are far thinner. In practice, you will often validate one part of a workflow against lithium-ion public data to test the method, then port the same method to sodium-ion with literature parameter values and more limited experimental references.

Fourth, low-temperature behavior and transport-property choices become more central sooner in sodium-ion studies. That means the distinction between SPM, SPMe, and DFN may matter differently depending on the sodium-ion question. Do not assume a ranking observed on a lithium-ion teaching benchmark transfers unchanged.

For now, the key lesson is this: Chapter 3 teaches the transferable software method. Later chapters will show how to swap in sodium-ion-specific assumptions honestly.

3.14 Chapter Summary and Skill Checklist

- You installed and verified `pybamm==25.12.1` in a controlled research environment.
- You connected Textbook Chapter 8’s DFN equations to PyBaMM’s object hierarchy: model, parameters, geometry, mesh, discretization, solver, and simulation.
- You ran a default DFN simulation with the `Chen2020` parameter set and extracted terminal voltage, open-circuit voltage, discharge capacity, and approximate SOC.
- You inspected internal states including electrolyte concentration, particle concentration, and overpotential contributions.
- You compared SPM, SPMe, and DFN on a common current profile and quantified runtime versus DFN-relative voltage error.

- You parsed a public battery dataset and handled headerless columns, missing temperatures, and current-sign conventions correctly.
- You reproduced the structure of a published SPMe-versus-DFN figure from Marquis et al. (2019) while documenting the choices and ambiguities honestly.

The commands, functions, and patterns that should now be entering muscle memory are:

- `pybamm.lithium_ion.DFN()`
- `pybamm.lithium_ion.SPM()`
- `pybamm.lithium_ion.SPMe()`
- `pybamm.ParameterValues("Chen2020")`
- `parameter_values.update({...})`
- `pybamm.Simulation(...)`
- `solution["Variable name [units]"](time_grid)`
- spatial evaluation patterns such as `solution["Electrolyte concentration [mol.m-3]"](x=x_positions, t=1800.0)`
- clean benchmarking on a common reporting grid

You should now be able to answer “yes” to each of these:

- I can install PyBaMM in the environment built in Lab Chapter 1 and verify that it really works.
- I can explain the difference between a PyBaMM model object and a parameter set.
- I can run a DFN simulation without copying code blindly.
- I can extract internal variables rather than only terminal voltage.
- I can explain, with evidence, when SPM, SPMe, or DFN is the better choice for a given modeling task.
- I can reproduce a published figure honestly enough to document version drift and modeling choices instead of hiding them.

If any of those boxes are still unchecked, revisit the relevant walkthrough before moving into Chapter 4. Chapter 4 assumes this chapter feels natural.

3.15 Deliverable

The deliverable for this chapter is:

A notebook running all three model fidelities, comparing runtime and accuracy, with a written interpretation of when you’d choose which.

Approach it in three sections.

First, include a clean benchmark section that reproduces the SPM versus SPMe versus DFN comparison on a common time grid. Use the code pattern from Guided Walkthrough 4 and save both the voltage-comparison figure and the runtime-versus-error figure.

Second, include one internal-state section for the DFN, similar to Guided Walkthrough 3. At minimum, plot terminal voltage, x-averaged electrolyte concentration, and one particle-concentration profile. The point is to demonstrate that you understand what extra physical visibility the DFN buys you.

Third, write a short interpretation paragraph. It should not say only “DFN is most accurate.” It should state when the extra fidelity is worth the runtime and when SPMe or SPM would be the better research choice.

A strong partial deliverable looks like this:

- one notebook file with all code cells runnable top to bottom
- one saved figure comparing voltage traces for the three fidelities
- one saved figure comparing runtime and RMS error
- one paragraph explaining which model you would use for parameter studies, which for synthetic-data generation, and which for quick exploratory sweeps

3.16 Further Practice and Reading

1. PyBaMM documentation home: <https://docs.pybamm.org/en/stable/>
Bookmark this. It is the authoritative source for current PyBaMM APIs and notebooks.
2. PyBaMM parameter-set documentation: <https://docs.pybamm.org/en/stable/source/ap>
This is the page to revisit whenever you want to know what bundled parameter sets exist and how they are described.
3. Marquis et al. (2019), “An asymptotic derivation of a single particle model with electrolyte,” DOI 10.1149/2.0341915jes
This is the paper behind the SPMe reproduction exercise and one of the central references for reduced-order physics-based battery modeling.
4. Chen et al. (2020), “Development of Experimental Techniques for Parameterization of Multi-scale Lithium-ion Battery Models,” DOI 10.1149/1945-7111/ab9050
This is the reference behind the Chen2020 parameter set used heavily in teaching and benchmarking.
5. Oxford Energy Trading Battery Degradation Dataset, DOI 10.5287/bodleian:gJPdDzvP4
Keep this bookmarked. We only touched it lightly here, but Chapter 4 and later dataset-centric chapters will benefit from your early familiarity with its structure.

Chapter 4

Parameters, Experiments, and Drive Cycles

4.1 Chapter Opening

Chapter 3 taught you how to run PyBaMM. This chapter teaches you how to *control* it. That sounds like a small step, but it is the difference between replaying a canned example and building a research workflow. By the end of this chapter, you will know how to inspect and modify parameter sets without losing provenance, replace function-valued parameters such as OCV curves and diffusivity laws, encode multi-step experimental protocols with charge, discharge, rest, and CV holds, import a real drive-cycle current trace, and decide when an isothermal simulation is good enough and when thermal coupling is worth the cost.

This matters for publishable research because most battery-modeling papers do not fail at the level of the governing equations. They fail in the translation layer between theory and execution. A parameter value is copied from the wrong chemistry. A charge-rest-discharge protocol is described vaguely and implemented differently than stated. A drive cycle is used without documenting current sign conventions. A thermal result is reported without saying whether temperature was held fixed or solved dynamically. Reviewers notice those things because they are exactly the places where otherwise good battery papers become irreproducible.

Keep Textbook Chapter 8 open while you work. That chapter gave you the porous-electrode equations and the physical meaning of OCV, overpotential, diffusion limitation, and transport coupling. This lab operationalizes that chapter. When we swap an OCV function or slow a diffusivity law, we are not “tweaking software.” We are changing a constitutive relation inside the electrochemical model you already learned in theory. Keep Textbook Chapter 10 open as well. That chapter explained why test protocols, current excitation, and duty-cycle choice determine what a battery model can and cannot reveal. This lab turns those ideas into executable experiments.

The deeper point is that a battery model is never just “the DFN” or “the SPMe.” It is always the model *plus* a parameterization *plus* an operating protocol *plus* a solver configuration. Serious simulation work is the craft of managing all four honestly. If Chapter 3 taught you how to ask PyBaMM for a voltage curve, Chapter 4 teaches you how to justify the curve you asked for.

This chapter also matters directly for sodium-ion work. Early sodium-ion studies often reuse lithium-ion software workflows because the tooling is more mature on the lithium-ion side. That is sensible, but only if you understand which parts of the workflow are chemistry-agnostic and which are not. Parameter handling, experiment construction, drive-cycle import, and thermal-vs-isothermal judgment all transfer cleanly. OCV shape, transport functions, nominal voltage, and data availability do not. We will keep making that distinction explicit.

The chapter is structured as a gradual climb. We begin with the parameter set itself: how to inspect it, compare it, modify it, and save your changes in a way your future self can still trust. Next, we replace function-valued parameters, because this is where PyBaMM stops feeling like a library of built-ins and starts behaving like a research tool. Then we move to the Experiment class and build a full charge-rest-discharge workflow. After that, we bring in a public UDDS current profile and run it through a model under realistic dynamic forcing. Finally, we revisit temperature, first as a fixed input and then as a coupled state, and quantify what changes.

Part II already satisfied its formal reproduce-a-published-figure requirement in Chapter 3, but this chapter still includes a literature-backed reproduction exercise because it is too valuable to skip. We will recreate the six-step cycling structure reported by Pozzato, Allam, and Onori (2022) for an EV-style ageing protocol and document exactly where we are matching the paper, where we are approximating, and what “close enough” means.

If you work carefully through this chapter, you will finish with a genuinely reusable notebook pattern: parameter provenance at the top, protocol definition in the middle, public data ingestion on the left edge, and publication-quality interpretation at the end. That is the skeleton of a real battery paper.

4.2 Prerequisites Check

- Required software: the `sib-research` environment from Lab Chapter 1, Python 3.11+, JupyterLab 4.4+, Git
- Required Python packages for this chapter: `pybamm==25.12.1`, `numpy`, `pandas`, `scipy`, `matplotlib`
- Optional but recommended package for later file formats: `openpyxl`
- Required textbook chapters: Textbook Chapter 8 is essential; Textbook Chapter 10 is strongly recommended
- Required prior lab chapters: Lab Chapter 1, Lab Chapter 2, and Lab Chapter 3
- Estimated time: 12 to 16 hours if Chapter 3 felt comfortable; 16 to 20 hours if parameter handling and protocol design are new to you

If the object model from Chapter 3 still feels shaky, revisit Sections 3.3 through 3.6 before you begin. If Python plotting from Lab Chapter 2 still feels awkward, revisit Sections 2.4 and 2.5. This chapter assumes you can already read a NumPy array, build a clean Matplotlib figure, and recognize when a result is physically implausible.

4.3 Environment Setup

We will use the same pinned PyBaMM version as Chapter 3 so that parameter introspection, Experiment behavior, and drive-cycle examples all stay aligned.

4.3.1 Activate the research environment

If you are using conda:

```
conda activate sib-research
```

If you are using venv on Linux or macOS:

```
source .venv/bin/activate
```

If you are using venv on Windows PowerShell:

```
.venv\Scripts\Activate.ps1
```

4.3.2 Install the chapter dependencies

If you already completed Chapter 3 successfully, you may already have everything you need. If not, install the pinned stack explicitly:

```
python -m pip install pybamm==25.12.1 numpy pandas scipy matplotlib openpyxl
```

If your shell does not expose python, use:

```
python3 -m pip install pybamm==25.12.1 numpy pandas scipy matplotlib openpyxl
```

The openpyxl package is not needed for the core guided walkthroughs in this chapter, but it becomes useful the moment you start pulling .xlsx battery files from public repositories. Installing it now saves you a needless interruption later.

4.3.3 Verify the install with a minimal “hello protocol”

Run this snippet in a terminal-backed Python session or a fresh notebook cell:

```
import pybamm

params = pybamm.ParameterValues("Chen2020")

print("PyBaMM version:", pybamm.__version__)
print("Nominal capacity [A.h]:", params["Nominal cell capacity [A.h]"])
print("Thermal parameter count:", len(params.list_by_category("thermal")))
print("Drive-cycle helper available:", hasattr(pybamm, "DataLoader"))
```

Expected output on the pinned version is:

```
PyBaMM version: 25.12.1
Nominal capacity [A.h]: 5.0
Thermal parameter count: 13
Drive-cycle helper available: True
```

The exact order of the printed lines may differ if you add extra checks, but the version, nominal capacity, and thermal-parameter count should match.

4.3.4 Confirm Jupyter is using the correct interpreter

Launch JupyterLab:

```
jupyter lab
```

Then run:

```
import sys
import pybamm

print(sys.executable)
print(pybamm.__version__)
```

The interpreter path should point into your intended environment, and the version should again print 25.12.1.

4.3.5 Common install failures and their fixes

1. `AttributeError: 'ParameterValues' object has no attribute 'get_info'`
Symptom: code from this chapter works in the book but not in your notebook.
Fix: you are almost certainly on an older PyBaMM version. Reinstall with the exact pin `pybamm==25.12.1`.
2. `ModuleNotFoundError: No module named 'pybamm'` in Jupyter but not in the terminal
Symptom: the import works in one place and fails in another.
Fix: your notebook kernel is bound to the wrong environment. Re-register the active environment kernel and switch notebooks to it.
3. A drive-cycle file download fails when calling `pybamm.DataLoader()`
Symptom: PyBaMM imports correctly, but the first call to `get_data("UDDS.csv")` errors.
Fix: this is usually a transient network or firewall issue. Retry once. If it persists, open the PyBaMM data release URL in a browser and download the file manually to a local folder, then point `pandas.read_csv` at that path.
4. Solver warnings appear on your first drive-cycle solve
Symptom: PyBaMM complains about evaluating outside the drive-cycle time range or about step-size rejection.
Fix: read the warning before panicking. On dynamic current profiles, some rejected trial steps are normal. If the solve completes and the result looks physical, the warning is often benign.

4.4 Conceptual Bridge: What the Tool Thinks a “Parameter” and an “Experiment” Are

In Textbook Chapter 8, a porous-electrode model looked like a set of governing equations plus constitutive laws. In practice, that means a battery simulation is assembled from at least three logically distinct pieces:

1. the conservation laws and algebraic constraints,
2. the parameter functions and constants that close those equations,
3. the operating condition imposed on the cell.

PyBaMM mirrors that separation very deliberately, and if you internalize that separation now, later chapters become much easier.

The model itself is the equation structure. In Chapter 3, we instantiated DFN, SPMe, and SPM objects. Those objects determine what states exist, what PDEs or ODEs are solved,

and which constitutive relations the model expects. The parameter set is not the same thing. A `ParameterValues` object supplies numerical values and callable functions for names such as `Negative electrode OCP [V]`, `Positive particle diffusivity [m2.s-1]`, and `Current function [A]`. The experiment is not the same thing either. An experiment is a time program for the external boundary condition: discharge here, rest there, hold at this voltage until that current, then apply an oscillatory current trace from data.

That separation maps very naturally onto the terminal-voltage decomposition you learned in theory. Under load, the measured voltage is not just an OCV lookup. It is the result of equilibrium potentials, kinetic losses, transport losses, and ohmic losses combining under a particular applied current history:

$$V(t) = U_p(\theta_p(t)) - U_n(\theta_n(t)) - \eta_{\text{rxn}}(t) - \eta_{\text{ohm}}(t) - \eta_{\text{conc}}(t). \quad (1)$$

Equation (1) is why this chapter has the shape it does. If you change an OCV function, you are changing the equilibrium term. If you slow a diffusivity function, you are changing the transport-limited response and therefore the concentration overpotential. If you move from a mild CC discharge to a stop-start drive cycle, you are not changing the chemistry at all; you are changing the forcing that excites those terms. If you change from isothermal to lumped thermal coupling, you are allowing temperature to move from “fixed parameter” to “solved state,” which then feeds back into kinetics, conductivity, diffusivity, and entropic heating.

This is also why a battery-model result must always be reported with more than just the model name. “We used a DFN model” is not enough. A DFN with one OCV curve, one diffusivity law, one current profile, and fixed temperature is a different scientific object from a DFN with a different OCV curve, a pulse protocol, and thermal coupling. The governing equations are the same; the simulation is not.

PyBaMM encodes that distinction in software form. `ParameterValues` is dictionary-like because many parameters are indeed simple constants, but it is more than a dictionary because some parameters are functions and because the class knows how to process those functions into the model. Newer PyBaMM releases also expose parameter introspection methods such as `get_info`, `list_by_category`, and `diff`. Those are not niceties. They are reproducibility tools. They let you answer questions like:

- What exactly did I modify?
- Which parameters in this set belong to the thermal domain?
- How does one published parameter set differ from another?

The `Experiment` class does something equally important on the operating-condition side. It turns a prose protocol into an executable boundary program. The string

“Charge at 1 C until 4.2 V”

is not just a friendly label. It compiles into a specific current-controlled step with a voltage termination event. Likewise,

```
pybamm.step.current(drive_cycle_array)
```

means: at each time in the provided array, impose this current value and interpolate between the data points. That is why drive-cycle data must start at $t = 0$, have consistent time units, and use a current sign convention you can explain.

Temperature is the last conceptual bridge worth pausing on. In an isothermal run, temperature is treated as an imposed quantity, effectively a parameter. In a lumped thermal run, temperature becomes a state governed by an energy balance:

$$\rho_{\text{eff}} \frac{dT}{dt} = \bar{Q} - \frac{hA}{V} (T - T_{\infty}). \quad (2)$$

Here, $\bar{Q}$ is the average heat-generation rate, h is the total heat-transfer coefficient, A is the cooling area, and V is the cell volume. The important modeling lesson is not the algebra itself, but the change in role: temperature moves from a fixed input to a coupled state variable. Once that happens, every temperature-dependent parameter becomes dynamically active.

So the mental model for the rest of the chapter is this. The textbook gave you equations and physical meaning. PyBaMM gives you a disciplined way to encode constants, functions, and forcing histories separately. Good simulation practice is learning when to change each of those layers, how to document the change, and how to interpret the consequences without confusing one layer for another.

4.5 Guided Walkthrough 1: Inspect, Compare, Modify, and Save a Parameter Set

Learning objective: Treat a PyBaMM parameter set as a research object rather than a hidden bundle of defaults.

Before we modify anything, we need to build the habit of *looking* at a parameter set carefully. The most common beginner mistake in physics-based battery modeling is to treat a named parameter set as if it were a chemistry truth. It is not. It is a concrete, inspectable, editable package of assumptions. In this walkthrough, we will inspect metadata, compare two bundled sets, make a controlled set of overrides, and save those overrides so the chapter's later simulations remain traceable.

4.5.1 Walkthrough 1 code

```
import json
from pathlib import Path

import pybamm

# Load two built-in parameter sets so we can inspect and compare them.
chen_params = pybamm.ParameterValues("Chen2020")
marquis_params = pybamm.ParameterValues("Marquis2019")

# Ask PyBaMM for metadata about one specific parameter.
capacity_info = chen_params.get_info("Nominal cell capacity [A.h]")

# Pull a thematic subset to understand the structure of the parameter set.
thermal_parameters = chen_params.list_by_category("thermal")

# Compare two published parameter sets.
```

```

parameter_diff = chen_params.diff(marquis_params)

print("PyBaMM version:", pybamm.__version__)
print("Nominal capacity value:", capacity_info.value)
print("Nominal capacity units:", capacity_info.units)
print("Nominal capacity category:", capacity_info.category)
print("Thermal parameter count:", len(thermal_parameters))
print("First five thermal parameters:", thermal_parameters[:5])
print("Changed keys versus Marquis2019:", len(parameter_diff.changed))
print("Sample changed keys:", list(parameter_diff.changed.keys())[:5])

# Make a research copy before editing anything.
research_params = chen_params.copy()
research_overrides = {
    "Current function [A]": 7.5,
    "Ambient temperature [K]": 303.15,
    "Lower voltage cut-off [V]": 2.8,
}
research_params.update(research_overrides)

# Save both a small override file and a full snapshot for reproducibility.
output_dir = Path("artifacts/chapter_4")
output_dir.mkdir(parents=True, exist_ok=True)

override_path = output_dir / "chen2020_research_overrides.json"
snapshot_path = output_dir / "chen2020_research_snapshot.json"

override_path.write_text(json.dumps(research_overrides, indent=2))
research_params.to_json(snapshot_path)

# Reload the snapshot to prove it round-trips cleanly.
reloaded_params = pybamm.ParameterValues.from_json(snapshot_path)

print("Override file saved to:", override_path.resolve())
print("Snapshot file saved to:", snapshot_path.resolve())
print("Reloaded current [A]:", reloaded_params["Current function [A]"])
print("Reloaded ambient temperature [K]:", reloaded_params["Ambient
↪ temperature [K]"])
print("Reloaded lower cut-off [V]:", reloaded_params["Lower voltage cut-off
↪ [V]"])

```

4.5.2 Walkthrough 1 explanation

The first two lines load Chen2020 and Marquis2019 as full `ParameterValues` objects. We are not yet solving a model. This is purely parameter work, and that is an important conceptual distinction. You can and should study parameter sets before you ever launch a solver.

`chen_params.get_info("Nominal cell capacity [A.h]")` returns a metadata object rather than just the raw value. The returned object includes the numerical value,

the units string, and the category. That matters because real parameter sets get large very quickly, and metadata is what keeps them navigable.

`list_by_category("thermal")` is our first act of structured browsing. In research practice, this is exactly the kind of question you ask when you switch from an isothermal notebook to a coupled thermal notebook: which parameters in this set are actually thermal?

`diff(marquis_params)` is one of the most useful habits in the whole chapter. When you say “I reran the analysis with a different parameter set,” you should be able to answer the follow-up question: different *how*? The `diff` object gives you a principled starting point.

The next block is about safe editing. `copy()` is not optional here. If you modify the original `chen_params` in place and later reuse it assuming it is still pristine, you have created the kind of hidden state that ruins notebooks and confuses coauthors.

Our three overrides are deliberately modest. We increase the current to 7.5 A, increase ambient temperature to 303.15 K, and raise the lower voltage cutoff to 2.8 V. None of these changes alters the structure of the electrochemical model; they alter the operating assumptions attached to it.

The saving pattern deserves attention. We write a small human-readable overrides file ourselves using `json.dumps`, and we also ask PyBaMM to serialize the full parameter set with `to_json`. Both are useful. The override file is what a human wants to review in Git. The full snapshot is what makes a future rerun exact.

Finally, we reload the saved snapshot with `from_json` and print a few key fields. That last step is not ceremony. It is a verification that the saved artifact is genuinely usable rather than just decorative.

4.5.3 Walkthrough 1 expected output

You should see PyBaMM version: 25.12.1, a nominal capacity of 5.0 A.h, and a thermal parameter count of 13. The exact set of changed keys reported by `diff()` will be longer than what prints on screen, because we intentionally print only the first handful.

You should also see two file paths printed inside `artifacts/chapter_4`. The reloaded current, ambient temperature, and lower-cutoff voltage should match the override values you wrote. If they do not, stop here and fix the save-reload path before moving on. Hidden parameter drift is a real research problem.

4.5.4 Walkthrough 2 troubleshooting

1. You edit `chen_params` directly and later forget you changed it.
 Symptom: later notebooks behave strangely even though the code “looks unchanged.”
 Fix: always `copy()` before mutating a published parameter set.
2. `get_info` or `diff` is missing.
 Symptom: an `AttributeError` appears on one of the introspection calls.
 Fix: your PyBaMM version is too old. Reinstall the pinned version for this manual.

3. The snapshot JSON saves, but reload values do not match your edits.
 Symptom: the printed values are different after `from_json`.
 Fix: inspect whether you accidentally wrote the override file instead of the full snapshot file path, or overwrote the wrong file from an older run.

4.5.5 Reflection

This exercise teaches a quiet but essential research skill: parameters are not decorations around a model. They are first-class artifacts. If you can inspect them, compare them, and save them cleanly, you are already working at a more publishable level than many battery notebooks ever reach.

4.6 Guided Walkthrough 2: Replace an OCV Curve and a Diffusivity Function

Learning objective: Replace function-valued parameters safely and understand what parts of the simulated behavior each change should influence.

Textbook Chapter 8 treated OCV functions and transport coefficients as constitutive ingredients in the model. In PyBaMM, those ingredients are exposed directly enough that you can swap them. That is powerful, but it is also where many readers first collide with symbolic programming. The subtle point is that your custom function must be compatible with PyBaMM's expression tree. If you write a function that only works on ordinary NumPy arrays, it may fail once PyBaMM passes it a symbolic variable. We will handle that carefully.

4.6.1 Walkthrough 2 code

```
import numpy as np
import matplotlib.pyplot as plt
import pybamm
```

```
model = pybamm.lithium_ion.DFN()
```

```
# Start from a clean baseline and make the current slightly higher so transport
↪ effects show up.
```

```
baseline_params = pybamm.ParameterValues("Chen2020")
baseline_params.update({"Current function [A]": 7.5})
```

```
modified_params = baseline_params.copy()
```

```
# Grab the original functions so we can wrap rather than replace blindly.
base_negative_ocp = baseline_params["Negative electrode OCP [V]"]
base_positive_diffusivity = baseline_params["Positive particle diffusivity"]
↪ [m2.s-1]
```

```
# Build a table-based OCV perturbation. This mimics what you would do with
↪ measured OCV data.
```

```

sto_grid = np.linspace(0.01, 0.99, 200)
shifted_negative_ocp_values = base_negative_ocp(sto_grid) - 0.02 * (1.0 -
↪ sto_grid)

def chapter4_negative_ocp(sto):
    return pybamm.Interpolant(
        sto_grid,
        shifted_negative_ocp_values,
        sto,
        name="chapter4_negative_ocp_table",
    )

# Make the positive-particle diffusivity uniformly slower.
def slower_positive_diffusivity(sto, temperature):
    return 0.6 * base_positive_diffusivity(sto, temperature)

modified_params.update(
    {
        "Negative electrode OCP [V]": chapter4_negative_ocp,
        "Positive particle diffusivity [m2.s-1]": slower_positive_diffusivity,
    }
)

t_eval = np.linspace(0, 2400, 241)
solutions = {}

for label, params in {
    "Baseline": baseline_params,
    "Modified functions": modified_params,
}.items():
    simulation = pybamm.Simulation(model, parameter_values=params)
    solutions[label] = simulation.solve(t_eval)

time_min = t_eval / 60.0
baseline_voltage_v = solutions["Baseline"]["Terminal voltage [V]"](t_eval)
modified_voltage_v = solutions["Modified functions"]["Terminal voltage
↪ [V]"](t_eval)

baseline_conc_overpotential_v = solutions["Baseline"][
    "X-averaged battery concentration overpotential [V]"
](t_eval)
modified_conc_overpotential_v = solutions["Modified functions"][
    "X-averaged battery concentration overpotential [V]"
](t_eval)

print(
    "Voltage shift at t = 0 [mV]:",
    round(float((modified_voltage_v[0] - baseline_voltage_v[0]) * 1000), 2),

```

```

)
print(
    "Voltage shift at t = 2400 s [mV]:",
    round(float((modified_voltage_v[-1] - baseline_voltage_v[-1]) * 1000), 2),
)
print(
    "Max added concentration overpotential [mV]:",
    round(
        float(
            np.max(
                np.abs(
                    modified_conc_overpotential_v - baseline_conc_overpotential_v
                )
            )
        ) * 1000
    ),
    2,
),
)

```

```

fig, axes = plt.subplots(2, 1, figsize=(9, 8), sharex=True,
    ↪ constrained_layout=True)

```

```

axes[0].plot(time_min, baseline_voltage_v, linewidth=2.2, label="Baseline")
axes[0].plot(time_min, modified_voltage_v, linewidth=2.2, label="Modified
    ↪ functions")
axes[0].set_ylabel("Terminal Voltage [V]")
axes[0].set_title("Replacing an OCV curve and a diffusivity law")
axes[0].legend()
axes[0].grid(True, alpha=0.3)

```

```

axes[1].plot(
    time_min,
    baseline_conc_overpotential_v * 1000,
    linewidth=2.2,
    label="Baseline",
)
axes[1].plot(
    time_min,
    modified_conc_overpotential_v * 1000,
    linewidth=2.2,
    label="Modified functions",
)
axes[1].set_xlabel("Time [min]")
axes[1].set_ylabel("Concentration Overpotential [mV]")
axes[1].legend()
axes[1].grid(True, alpha=0.3)

```

```

plt.show()

```

4.6.2 Walkthrough 2 explanation

We begin with a DFN model and a clean Chen2020 parameter set. We then raise the current to 7.5 A so that concentration effects become large enough to read clearly over a modest simulation window. This is standard teaching practice: amplify the mechanism you want to observe, but say out loud that you are doing it.

The next two lines pull the original negative-electrode OCP function and positive-particle diffusivity function out of the parameter set. This is a much safer way to customize behavior than replacing functions from scratch without a reference.

The OCV replacement is intentionally table-based. We evaluate the original OCP on a stoichiometry grid and then subtract a small stoichiometry-dependent offset. That creates a synthetic measured-like table. The custom function `chapter4_negative_ocp(sto)` returns a `pybamm.Interpolant`, not a NumPy interpolation. That distinction is critical. `pybamm.Interpolant` can live inside the symbolic model; `np.interp` cannot.

The diffusivity modification is simpler. We define `slower_positive_diffusivity(sto, temperature)` with the same input signature as the original function and scale the baseline value by 0.6. This is a good example of a change that mostly affects dynamic transport behavior rather than the equilibrium voltage baseline.

After updating the modified parameter set, we solve the baseline and modified models on the same time grid. Using the same `t_eval` is essential because it makes the resulting differences pointwise comparable.

We then extract two variables. Terminal voltage [V] tells us what a user or cyler would observe directly. X-averaged battery concentration overpotential [V] is the internal mechanism we expect the diffusivity change to affect most strongly. This pairing is deliberate: an external signal plus an internal explanation.

4.6.3 Walkthrough 2 expected output and interpretation

You should see the modified-voltage curve sit slightly below the baseline from the very beginning, because we shifted the negative-electrode OCP downward. As the discharge progresses, the gap should widen somewhat, and the concentration-overpotential curve for the modified case should rise more aggressively than the baseline because we intentionally slowed the positive-particle diffusivity.

The printed values will be in the tens of millivolts, not volts. That scale is important. We are not building a cartoonishly different battery; we are making realistic-looking constitutive perturbations and watching how they propagate.

If both voltage curves are essentially identical and the concentration-overpotential curves sit on top of one another, something is wrong with your function replacement.

4.6.4 Walkthrough 3 troubleshooting

1. You use `np.interp` or other pure-NumPy logic in the custom OCV function.
Symptom: PyBaMM fails once the function is processed into the model.
Fix: use `pybamm.Interpolant` for tabulated functions that must accept symbolic inputs.

2. Your custom function signature does not match the expected inputs.
Symptom: the solver fails during parameter processing with a function-argument error.
Fix: use `model.print_parameter_info()` if needed to check which inputs a given function parameter expects.
3. You mutate the baseline parameter set and then “compare” it to itself.
Symptom: baseline and modified curves match suspiciously well.
Fix: make the modified set from `baseline_params.copy()` before updating.

4.6.5 Walkthrough 2 reflection

This exercise teaches one of the most transferable skills in the companion: how to replace the physics-bearing pieces of a PyBaMM model without losing symbolic compatibility. That is exactly the workflow you will need later for sodium-ion OCV curves, fitted transport laws, and literature-derived constitutive updates.

4.7 Guided Walkthrough 3: Encode a Full Charge-Rest-Discharge Protocol with Experiment

Learning objective: Build a multi-step protocol that includes CC, CV, and rest periods, then interpret the resulting current and voltage trajectory correctly.

Battery papers rarely use one clean constant-current segment and stop. Real workflows include transitions, cutoffs, and recovery periods. That is why PyBaMM’s `Experiment` class matters. In this walkthrough, we will encode a full protocol that begins with discharge, pauses for relaxation, recharges with a CC-CV sequence, and rests again. The objective is not just to run it, but to read the sign conventions and step transitions like an experimentalist.

4.7.1 Walkthrough 3 code

```
import numpy as np
import matplotlib.pyplot as plt
import pybamm

parameter_values = pybamm.ParameterValues("Chen2020")

experiment = pybamm.Experiment(
    [
        "Discharge at C/3 until 3.0 V",
        "Rest for 30 minutes",
        "Charge at 1 C until 4.2 V",
        "Hold at 4.2 V until C/20",
        "Rest for 30 minutes",
    ],
    period="10 seconds",
)

simulation = pybamm.Simulation(
```

```

pybamm.lithium_ion.DFN(),
parameter_values=parameter_values,
experiment=experiment,
solver=pybamm.CasadiSolver(mode="fast"),
)
solution = simulation.solve()

time_s = solution["Time [s]"].entries
time_h = time_s / 3600.0
current_a = solution["Current [A]"].entries
voltage_v = solution["Terminal voltage [V]"].entries
discharge_capacity_ah = solution["Discharge capacity [A.h]"].entries

discharge_energy_wh = np.trapezoid(
    np.clip(current_a, 0.0, None) * voltage_v,
    time_s,
) / 3600.0
charge_energy_wh = -np.trapezoid(
    np.clip(current_a, None, 0.0) * voltage_v,
    time_s,
) / 3600.0
throughput_ah = np.trapezoid(np.abs(current_a), time_s) / 3600.0

print("Total throughput [A.h]:", round(float(throughput_ah), 4))
print("Discharge energy [W.h]:", round(float(discharge_energy_wh), 4))
print("Charge energy [W.h]:", round(float(charge_energy_wh), 4))
print("Final terminal voltage [V]:", round(float(voltage_v[-1]), 4))

fig, axes = plt.subplots(3, 1, figsize=(10, 10), sharex=True,
    ↪ constrained_layout=True)

axes[0].plot(time_h, current_a, linewidth=2.1, color="tab:blue")
axes[0].set_ylabel("Current [A]")
axes[0].set_title("CC-rest-CCCV-rest protocol")
axes[0].grid(True, alpha=0.3)

axes[1].plot(time_h, voltage_v, linewidth=2.1, color="tab:orange")
axes[1].set_ylabel("Terminal Voltage [V]")
axes[1].grid(True, alpha=0.3)

axes[2].plot(time_h, discharge_capacity_ah, linewidth=2.1, color="tab:green")
axes[2].set_ylabel("Discharge Capacity [A.h]")
axes[2].set_xlabel("Time [h]")
axes[2].grid(True, alpha=0.3)

plt.show()

```

4.7.2 Walkthrough 3 explanation

The Experiment constructor is the star here. Each string is one operating step. PyBaMM parses these into structured step objects with durations, directions, and termination events.

The first step, "Discharge at C/3 until 3.0 V", means what it says: apply a positive discharge current equal to one-third of the nominal capacity in amperes until the lower voltage threshold is reached. The second step is a rest. In PyBaMM, that means zero current, not a magical equilibrium reset.

The third and fourth steps together form a standard CCCV charge. First we charge at a fixed current until the upper cutoff voltage 4.2 V is reached. Then we hold voltage constant at 4.2 V and allow the current to decay until it reaches C/20. This is standard practice in the field. If you publish charge simulations without being clear about whether your charge was CC-only or CCCV, you are leaving out an important part of the method.

We request a reporting period="10 seconds" because that is dense enough to show the CV taper smoothly without creating an unnecessarily large solution.

The energy calculations are worth reading carefully. Discharge current is positive in PyBaMM, so discharge energy is computed from the positive-current portion of the signal. Charge current is negative, so we take the negative-current portion, multiply by voltage, and apply a minus sign to report charge energy as a positive quantity. This sign bookkeeping is simple once you understand it, but it is one of the most common sources of silent mistakes in battery data analysis.

4.7.3 Walkthrough 3 expected output and interpretation

The current plot should have five visually distinct regions. First, a positive constant-current discharge segment. Second, a flat zero-current rest. Third, a negative constant-current charge segment. Fourth, a negative current that gradually decays in magnitude during the CV hold. Fifth, another flat zero-current rest.

The voltage plot should fall during discharge, rebound upward during the first rest, rise during charge, flatten near 4.2 V during the CV hold, and relax slightly during the final rest. The discharge-capacity curve should rise during discharge and then remain flat during the rest and charge portions because it measures discharged capacity since the beginning of the simulation, not signed charge throughput.

If your current sign is reversed in your mental model, this whole figure will feel backwards. That is exactly why this walkthrough exists.

4.7.4 Walkthrough 4 troubleshooting

1. You place the charge step first while keeping the default fully charged initial state.
Symptom: the first step is skipped or declared infeasible.
Fix: either start from discharge, as we do here, or explicitly set the initial state before charging first.
2. You confuse discharge capacity with net charge throughput.
Symptom: you expect the capacity curve to decrease during charge.

Fix: Discharge capacity [A.h] is cumulative discharged capacity, not signed coulomb counting.

3. The CV hold never appears.

Symptom: the current plot jumps straight from constant-current charge to rest.

Fix: the current may already have dropped below the termination threshold by the moment 4.2 V is reached, or your version/parser may differ. Re-check the string exactly.

4.7.5 Walkthrough 3 reflection

This exercise teaches you to think of an experiment as executable boundary data. That is a graduate-level modeling habit. You are no longer “running a battery model.” You are applying a specific test protocol to a virtual cell and reading the result with the same sign discipline you would need on a real cycler.

4.8 Dataset Integration: Public UDDS Drive-Cycle Data

This chapter’s real-data section uses a public UDDS current trace distributed in PyBaMM’s data release and derived from the standard Urban Dynamometer Driving Schedule workflow. We use the packaged file because it is small, directly machine-readable, and immediately compatible with PyBaMM. If you prefer to go to the regulatory source, keep the EPA Dynamometer Drive Schedules page bookmarked as the authoritative reference for the underlying standard.

Item	Value
Public current file used in code	UDDS.csv from the PyBaMM public data release
Programmatic access	<code>pybamm.DataLoader().get_data("UDDS.csv")</code>
Direct release pattern	https://github.com/pybamm-team/pybamm-data/releases/download/v1.0.x/UDDS.csv
Underlying standard	EPA Urban Dynamometer Driving Schedule (UDDS)
EPA reference page	https://www.epa.gov/vehicle-and-fuel-emissions-testing/dynamometer-drive-schedules
Reported EPA file size for the raw schedule page entry	12.32 KB for the tab-delimited UDDS text file
Format used in the PyBaMM convenience file	two columns: time in seconds, current in amperes
License and reuse note	cite the source you actually use; the EPA page does not expose a separate dataset license on the download listing, while the PyBaMM release is publicly distributed for model examples

For research notebooks, the packaged `UDDS.csv` is the right level of convenience. For a paper, cite the underlying standard trace or the data package you actually downloaded. Do not pretend a convenience wrapper is the primary source if it is not.

The parsing workflow in the next walkthrough will make four checks that should become muscle memory:

1. strip comment lines and null rows,
2. ensure the time column is monotonically increasing,
3. confirm the time axis starts at zero,
4. state the current sign convention explicitly.

Those checks feel boring right up until the day one of them saves your figure.

4.9 Guided Walkthrough 4: Load a Real UDDS Current Profile and Run It in PyBaMM

Learning objective: Parse a public drive-cycle file cleanly, validate its columns and sign convention, and run it through a physics-based model.

This is the point where the chapter becomes recognizably research-like. We are no longer using a hand-written one-line current function. We are taking a public current trace, validating it, and using it as a dynamic input. For this walkthrough we use SPMe rather than DFN. That is a standard practice in the field when you want electrolyte dynamics but also want fast iteration under long or jagged duty cycles.

4.9.1 Walkthrough 4 code

```
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import pybamm

def load_drive_cycle(filename):
    data_loader = pybamm.DataLoader()
    file_path = Path(data_loader.get_data(filename))

    drive_cycle = pd.read_csv(
        file_path,
        comment="#",
        header=None,
        names=["time_s", "current_a"],
    )

    drive_cycle = (
        drive_cycle.dropna()
        .drop_duplicates(subset="time_s", keep="first")
        .sort_values("time_s")
        .reset_index(drop=True)
    )
```

```

    # Normalize the time axis in case a downloaded file starts at a nonzero
    ↪ timestamp.
    drive_cycle["time_s"] = drive_cycle["time_s"] -
    ↪ drive_cycle["time_s"].iloc[0]

    return file_path, drive_cycle

udds_path, udds = load_drive_cycle("UDDS.csv")

udds["delta_t_s"] = udds["time_s"].diff().fillna(0.0)
udds["signed_capacity_increment_ah"] = (
    udds["current_a"] * udds["delta_t_s"] / 3600.0
)
udds["cumulative_signed_capacity_ah"] =
    ↪ udds["signed_capacity_increment_ah"].cumsum()

print("Loaded file:", udds_path)
print("Row count:", len(udds))
print("Duration [s]:", round(float(udds["time_s"].iloc[-1]), 1))
print("Minimum current [A]:", round(float(udds["current_a"].min()), 4))
print("Maximum current [A]:", round(float(udds["current_a"].max()), 4))
print("Time starts at zero:", bool(np.isclose(udds["time_s"].iloc[0], 0.0)))

fig, axes = plt.subplots(2, 1, figsize=(10, 7), sharex=True,
    ↪ constrained_layout=True)

axes[0].plot(udds["time_s"] / 60.0, udds["current_a"], linewidth=1.7,
    ↪ color="tab:blue")
axes[0].set_ylabel("Current [A]")
axes[0].set_title("Public UDDS current profile")
axes[0].grid(True, alpha=0.3)

axes[1].plot(
    udds["time_s"] / 60.0,
    udds["cumulative_signed_capacity_ah"],
    linewidth=1.7,
    color="tab:purple",
)
axes[1].set_xlabel("Time [min]")
axes[1].set_ylabel("Cumulative Signed Capacity [A.h]")
axes[1].grid(True, alpha=0.3)

plt.show()

drive_cycle_array = udds[["time_s", "current_a"]].to_numpy()

experiment = pybamm.Experiment(
    [pybamm.step.current(drive_cycle_array)],
    period="1 second",

```

```

)

parameter_values = pybamm.ParameterValues("Chen2020")
simulation = pybamm.Simulation(
    pybamm.lithium_ion.SPMe(),
    parameter_values=parameter_values,
    experiment=experiment,
    solver=pybamm.CasadiSolver(mode="fast"),
)
solution = simulation.solve()

time_s = solution["Time [s]"].entries
time_min = time_s / 60.0
current_a = solution["Current [A]"].entries
voltage_v = solution["Terminal voltage [V]"].entries
discharge_capacity_ah = solution["Discharge capacity [A.h]"].entries
approx_soc = 1.0 - discharge_capacity_ah / parameter_values["Nominal cell
↪ capacity [A.h]"]
electrolyte_concentration = solution["X-averaged electrolyte concentration
↪ [mol.m-3]"].entries

print("Final terminal voltage [V]:", round(float(voltage_v[-1]), 4))
print("Final approximate SOC [-]:", round(float(approx_soc[-1]), 4))
print(
    "Electrolyte concentration range [mol.m-3]:",
    round(float(electrolyte_concentration.min()), 2),
    "to",
    round(float(electrolyte_concentration.max()), 2),
)

fig, axes = plt.subplots(4, 1, figsize=(10, 12), sharex=True,
↪ constrained_layout=True)

axes[0].plot(time_min, current_a, linewidth=1.7, color="tab:blue")
axes[0].set_ylabel("Current [A]")
axes[0].set_title("UDDS-driven SPMe simulation")
axes[0].grid(True, alpha=0.3)

axes[1].plot(time_min, voltage_v, linewidth=1.9, color="tab:orange")
axes[1].set_ylabel("Voltage [V]")
axes[1].grid(True, alpha=0.3)

axes[2].plot(time_min, approx_soc, linewidth=1.9, color="tab:red")
axes[2].set_ylabel("Approx. SOC [-]")
axes[2].grid(True, alpha=0.3)

axes[3].plot(
    time_min,
    electrolyte_concentration,
    linewidth=1.9,
    color="tab:green",

```

```
)
axes[3].set_ylabel("X-avg Electrolyte c [mol.m$^{-3}$]")
axes[3].set_xlabel("Time [min]")
axes[3].grid(True, alpha=0.3)

plt.show()
```

4.9.2 Walkthrough 4 explanation

We begin with a reusable helper, `load_drive_cycle(filename)`. This does more than read a CSV. It performs light but meaningful data hygiene: remove nulls, remove repeated timestamps, sort by time, and normalize the time axis so the trace begins at zero. Those four operations are so common in battery-data work that they are worth wrapping early.

The cumulative signed-capacity calculation is not something PyBaMM needs in order to solve the model. It is something *you* need in order to inspect the dataset. A current profile can look plausible while still having an unexpected net bias or sign mistake. Integrating it once is a quick way to see whether the trace is mostly discharge, mostly charge, or approximately energy-neutral.

The `drive_cycle_array` passed into `pybamm.step.current` must be a two-column NumPy array. The first column is time in seconds. The second is current in amperes. PyBaMM treats positive current as discharge and negative current as charge or regenerative current. State that explicitly whenever you work with a dynamic trace, because public datasets do not all use the same sign convention.

We then build a one-step experiment containing the whole drive cycle. The `period="1 second"` argument tells PyBaMM how densely to record the outputs. Since UDDS is conventionally used at a one-second cadence, this is the natural choice.

We use SPMe instead of DFN for a reason. This is standard practice when you want realistic electrolyte dynamics and internal concentration behavior but also want a model that remains pleasant to rerun many times in a notebook. Later, if a reviewer-level result depends critically on this case, you can rerun the final comparison with the DFN.

Finally, we plot four panels: current, voltage, approximate SOC, and x-averaged electrolyte concentration. That panel selection is intentional. It ties together the external input, external response, state depletion, and one internal transport variable.

4.9.3 Walkthrough 4 expected output and interpretation

The first figure should show a jagged current trace with frequent changes and many sections near zero. That is exactly what you expect from an urban stop-and-go profile. The cumulative signed-capacity curve should drift rather than remain perfectly flat, indicating a net discharge bias over the cycle.

The simulation figure should show the terminal voltage sagging during higher discharge-current sections and recovering during low-current or regenerative sections. The approximate SOC should decline gradually rather than linearly, because the current is not constant. The x-averaged electrolyte concentration should vary more subtly than voltage, but it should still respond coherently to the current bursts.

If the current looks upside down relative to the voltage response, the first thing to check is sign convention, not solver correctness.

4.9.4 Walkthrough 5 troubleshooting

1. The first timestamp is not zero.
Symptom: PyBaMM raises an error or the drive cycle starts strangely.
Fix: normalize the time axis exactly as we do in the helper function.
2. Your current sign convention is reversed relative to PyBaMM.
Symptom: voltage rises when you think the battery is discharging.
Fix: inspect the input current, not just the output voltage. Positive current should mean discharge in this chapter's code.
3. `DataLoader` cannot download the file on a restricted network.
Symptom: the code fails before `pandas.read_csv` ever runs.
Fix: manually download the CSV once and point `read_csv` at the local path.
4. The solver prints a warning about evaluating outside the data range.
Symptom: a warning appears before the solve completes.
Fix: on dynamic traces, some rejected trial steps are normal. If the final result is smooth and the solve completes, this warning is usually harmless.

4.9.5 Walkthrough 4 reflection

This exercise teaches the real workflow boundary between public data and simulation. A drive cycle is not magic. It is just a time series. But once you learn how to validate, parse, and impose it cleanly, your battery model stops living in a toy world of constant-current segments.

4.10 Guided Walkthrough 5: Temperature as an Input Versus Temperature as a Coupled State

Learning objective: Compare an isothermal drive-cycle simulation to a lumped-thermal simulation and quantify what thermal coupling changes.

Chapter 4 is the right place to learn this distinction because temperature often sneaks into modeling work without being declared clearly. In an isothermal model, temperature is an imposed condition. In a thermally coupled model, temperature is part of the solution. We will use the same dynamic current trace in both cases so the comparison remains honest.

To make the thermal effect visible over a single short drive cycle, we will double the UDDS current amplitude. This is a teaching shortcut, not a claim about a real vehicle pack. I am flagging that explicitly because it is exactly the kind of modeling choice you should label clearly in a paper.

4.10.1 Walkthrough 5 code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```

import pybamm

data_loader = pybamm.DataLoader()
udds = pd.read_csv(
    data_loader.get_data("UDDS.csv"),
    comment="#",
    header=None,
    names=["time_s", "current_a"],
).dropna()

udds["time_s"] = udds["time_s"] - udds["time_s"].iloc[0]

# Teaching shortcut: amplify the current so the temperature response is visible
↪ in one cycle.
scaled_udds = udds.copy()
scaled_udds["current_a"] = 2.0 * scaled_udds["current_a"]
drive_cycle_array = scaled_udds[["time_s", "current_a"]].to_numpy()

experiment = pybamm.Experiment(
    [pybamm.step.current(drive_cycle_array)],
    period="1 second",
)

models = {
    "Isothermal SPM": pybamm.lithium_ion.SPM(),
    "Lumped-thermal SPM": pybamm.lithium_ion.SPM({"thermal": "lumped"}),
}

solutions = {}
for label, model in models.items():
    params = pybamm.ParameterValues("Chen2020")
    params.update(
        {
            "Ambient temperature [K]": 298.15,
            "Initial temperature [K]": 298.15,
            "Total heat transfer coefficient [W.m-2.K-1]": 10.0,
        }
    )

    simulation = pybamm.Simulation(
        model,
        parameter_values=params,
        experiment=experiment,
        solver=pybamm.CasadiSolver(mode="fast"),
    )
    solutions[label] = simulation.solve()

time_s = solutions["Isothermal SPM"]["Time [s]"].entries
time_min = time_s / 60.0

```

```

isothermal_voltage_v = solutions["Isothermal SPMe"]["Terminal voltage
↪ [V]"].entries
lumped_voltage_v = solutions["Lumped-thermal SPMe"]["Terminal voltage
↪ [V]"].entries

isothermal_temperature_k = solutions["Isothermal SPMe"][
    "Volume-averaged cell temperature [K]"
].entries
lumped_temperature_k = solutions["Lumped-thermal SPMe"][
    "Volume-averaged cell temperature [K]"
].entries
lumped_total_heating = solutions["Lumped-thermal SPMe"][
    "Volume-averaged total heating [W.m-3]"
].entries
current_a = solutions["Lumped-thermal SPMe"]["Current [A]"].entries

print(
    "Isothermal max temperature rise [K]:",
    round(float(isothermal_temperature_k.max() - 298.15), 4),
)
print(
    "Lumped max temperature rise [K]:",
    round(float(lumped_temperature_k.max() - 298.15), 4),
)
print(
    "Max voltage difference [mV]:",
    round(float(np.max(np.abs(lumped_voltage_v - isothermal_voltage_v)) *
↪ 1000), 2),
)

fig, axes = plt.subplots(3, 1, figsize=(10, 10), sharex=True,
↪ constrained_layout=True)

axes[0].plot(time_min, current_a, linewidth=1.6, color="tab:blue")
axes[0].set_ylabel("Current [A]")
axes[0].set_title("Isothermal versus lumped-thermal drive-cycle simulation")
axes[0].grid(True, alpha=0.3)

axes[1].plot(time_min, isothermal_voltage_v, linewidth=2.0,
↪ label="Isothermal")
axes[1].plot(time_min, lumped_voltage_v, linewidth=2.0, label="Lumped
↪ thermal")
axes[1].set_ylabel("Voltage [V]")
axes[1].legend()
axes[1].grid(True, alpha=0.3)

axes[2].plot(
    time_min,
    isothermal_temperature_k,
    linewidth=2.0,
    label="Isothermal",
)

```



```

axes[2].plot(
    time_min,
    lumped_temperature_k,
    linewidth=2.0,
    label="Lumped thermal",
)
axes[2].plot(
    time_min,
    298.15 + 0.000001 * lumped_total_heating,
    linewidth=1.0,
    linestyle="--",
    alpha=0.6,
    label="Heating trend (scaled for shape only)",
)
axes[2].set_ylabel("Temperature [K]")
axes[2].set_xlabel("Time [min]")
axes[2].legend()
axes[2].grid(True, alpha=0.3)

plt.show()

```

4.10.2 Walkthrough 5 explanation

The first part of the code reloads the UDDS profile and doubles the current amplitude. I am saying this twice because it matters pedagogically: this is a teaching move to make thermal coupling visible over one short trace. In real work, you would justify current amplitude from the cell or pack you are studying.

We then create two models with identical electrochemical structure but different thermal treatment. `pybamm.lithium_ion.SPMe()` is isothermal by default. `pybamm.lithium_ion.SPMe({"thermal": "lumped"})` adds the lumped thermal ODE from Equation (2).

The parameter updates add three thermal quantities: ambient temperature, initial temperature, and total heat-transfer coefficient. These are the minimum thermal assumptions that keep the comparison explicit.

We extract three outputs: voltage, volume-averaged temperature, and volume-averaged total heating. The heating variable is particularly useful because it reminds you that a temperature rise is not an abstract solver artifact. It is being driven by interpretable heat generation inside the model.

The final plot overlays temperature from both cases. In the isothermal case, the temperature should remain flat. In the lumped case, it should rise. That is the whole conceptual distinction, rendered as a figure.

4.10.3 Walkthrough 5 expected output and interpretation

The printed maximum temperature rise for the isothermal case should be essentially zero, because the model is not allowed to heat up. The lumped case should show a positive temperature rise. The exact magnitude depends on the current amplification and heat-transfer coefficient, but it should be visibly nonzero.

In the figure, the isothermal temperature line should be flat at 298.15 K. The lumped-thermal line should drift upward over the cycle, with the heating trend roughly following higher-current sections. The voltage traces may differ only by a few millivolts, but that is exactly the lesson: sometimes thermal coupling changes the answer a little, and sometimes it changes it a lot. You need to quantify rather than guess.

4.10.4 What could go wrong

1. You forget to enable the thermal option.

Symptom: both temperature traces are flat.

Fix: make sure the second model is instantiated with `{"thermal": "lumped"}`.

2. The thermal effect is too small to see.

Symptom: the lumped-temperature line is technically rising but visually flat.

Fix: that can happen on mild traces. Increase current amplitude modestly or reduce the heat-transfer coefficient, and label the change honestly as a teaching adjustment.

3. You choose unrealistic thermal parameters and get absurd heating.

Symptom: the temperature rise is many tens of kelvin over one short cycle.

Fix: revisit the heat-transfer coefficient and the current scaling before trusting anything else.

4.10.5 Walkthrough 5 reflection

This exercise teaches a modeling judgment rather than a syntax pattern. Temperature can be a fixed assumption or a solved state. The code difference is one option dictionary. The scientific difference is much larger, because it changes which mechanisms are actually active during the simulation.

4.11 Reproduction Exercise: Recreate the Six-Step EV Ageing Protocol from Pozzato, Allam, and Onori (2022)

The open-access dataset paper by Pozzato, Allam, and Onori describes an EV-oriented ageing workflow built from CC-CV charge steps, a shallow CC discharge, and a UDDS discharge segment. The paper's Figure 1 shows the first three cycling profiles for one cell and zooms into the six-step cycle structure. The figure caption and data-description text are unusually helpful: the authors explicitly state that positive current denotes discharge and negative current denotes charge, and they describe the six steps in prose.

That makes this a strong reproduction target for Chapter 4. We are not trying to reproduce every exact voltage value of the authors' experimental cell. We are reproducing the *protocol structure* in PyBaMM and making our approximation choices explicit.

The paper is:

G. Pozzato, A. Allam, and S. Onori, "Lithium-ion battery aging dataset based on electric vehicle real-driving profiles," *Data in Brief*, 41, 107995, 2022, DOI 10.1016/j.dib.2022.107995.

4.11.1 What we can match exactly

We can match the six-step sequence described in the paper:

1. charge at a specified C-rate until 4.0 V,
2. hold at 4.0 V until current falls below 50 mA,
3. charge at C/4 until 4.2 V,
4. hold at 4.2 V until current falls below 50 mA,
5. shallow discharge at C/4,
6. UDDS discharge.

4.11.2 What we cannot match exactly from the paper alone

The exact cell is an INR21700-M50T with nominal capacity 4.85 Ah, while our bundled teaching parameter set is Chen2020. The paper's Step 6 uses a UDDS profile normalized to the authors' cell capacity, and the exact stop criterion for the concatenated UDDS section depends on the experimental capacity state at that point in life. Those details matter.

4.11.3 Our declared approximations

We will use the Chen2020 parameter set but reset its nominal capacity to 4.85 Ah so the C-rates are closer to the paper's cell. We will explicitly initialize the cell at 20% SOC so the opening charge steps are feasible. We will approximate the shallow C/4 discharge from 100% to 80% SOC as 48 minutes at C/4, because 20% of a full 4 h C/4 discharge is 0.8 h. We will include one public UDDS trace rather than concatenating traces until a specific terminal SOC event.

That means our reproduction target is *structural fidelity*, not exact voltage matching. Close enough here means the current trace has the correct sign pattern, the step ordering matches the paper, and the voltage evolution is physically consistent with that sequence.

4.11.4 Reproduction code

```
import pandas as pd
import matplotlib.pyplot as plt
import pybamm

data_loader = pybamm.DataLoader()
udds = pd.read_csv(
    data_loader.get_data("UDDS.csv"),
    comment="#",
    header=None,
    names=["time_s", "current_a"],
).dropna()
udds["time_s"] = udds["time_s"] - udds["time_s"].iloc[0]
udds_array = udds[["time_s", "current_a"]].to_numpy()

parameter_values = pybamm.ParameterValues("Chen2020")
parameter_values.update({"Nominal cell capacity [A.h]": 4.85})
```

```

parameter_values = parameter_values.set_initial_state(0.2, inplace=False)

protocol = pybamm.Experiment(
    [
        "Charge at C/2 until 4.0 V",
        "Hold at 4.0 V until 50 mA",
        "Charge at C/4 until 4.2 V",
        "Hold at 4.2 V until 50 mA",
        "Rest for 30 minutes",
        "Discharge at C/4 for 48 minutes",
        pybamm.step.current(udds_array),
    ],
    period="1 second",
)

simulation = pybamm.Simulation(
    pybamm.lithium_ion.SPMe({"thermal": "lumped"}),
    parameter_values=parameter_values,
    experiment=protocol,
    solver=pybamm.CasadiSolver(mode="fast"),
)
solution = simulation.solve()

time_h = solution["Time [s]"].entries / 3600.0
current_a = solution["Current [A]"].entries
voltage_v = solution["Terminal voltage [V]"].entries

fig, axes = plt.subplots(2, 1, figsize=(10, 8), sharex=True,
    ↪ constrained_layout=True)

axes[0].plot(time_h, current_a, linewidth=1.8, color="tab:blue")
axes[0].set_ylabel("Current [A]")
axes[0].set_title("Protocol-structure reproduction of Pozzato et al. (2022),
    ↪ Fig. 1")
axes[0].grid(True, alpha=0.3)

axes[1].plot(time_h, voltage_v, linewidth=1.8, color="tab:orange")
axes[1].set_ylabel("Voltage [V]")
axes[1].set_xlabel("Time [h]")
axes[1].grid(True, alpha=0.3)

plt.show()

```

4.11.5 What the reproduced figure should look like

The current trace should begin negative, because the cycle starts with charge. You should see an initial negative constant-current segment, then a tapering negative CV segment near 4.0 V, then another smaller negative constant-current segment at C/4, then a second tapering CV hold near 4.2 V. After the rest, you should see a positive C/4 discharge plateau, followed by a jagged UDDS section with frequent current changes and occasional low-current intervals.

The voltage trace should rise during the two charge stages, flatten during the CV holds, relax during the rest, drop gently during the shallow C/4 discharge, and then oscillate during the UDDS segment. If that sequence appears in the correct order with the correct sign convention, your reproduction is doing the intended scientific job.

4.11.6 Where “close enough” ends

If your current signs are reversed, the step sequence is infeasible, or the CC-CV structure is not visible, the reproduction is not close enough. If the precise voltage values differ modestly from the experimental figure because we used a bundled teaching parameter set rather than the authors’ exact cell characterization, that is acceptable *as long as you say so*. Honest approximation is part of good reproduction practice.

4.12 Open-Ended Exercises

4.12.1 Exercise 1: Design a GITT protocol and extract quasi-OCV

Build a galvanostatic intermittent titration protocol with ten repeated pairs of 10 minutes of C/10 discharge and 50 minutes of rest. Use the end-of-rest voltages to estimate quasi-OCV as a function of discharged capacity.

Hints:

- Use a repeated tuple inside `pybamm.Experiment`.
- The end of each rest segment is where the current is still essentially zero, but the next sample is not.
- Plot quasi-OCV points against Discharge capacity [A.h].

4.12.2 Exercise 2: Swap UDDS for WLTC and compare transport stress

Load `WLTC.csv` with the same helper pattern used for `UDDS.csv`. Run the same SPMe workflow and compare:

- final approximate SOC,
- minimum terminal voltage,
- maximum x-averaged concentration overpotential.

Hints:

- PyBaMM’s data release includes `WLTC.csv`.
- Use the same plotting structure so the comparison stays fair.
- Do not compare two traces on mismatched time units.

4.12.3 Exercise 3: Quantify cooling sensitivity

Using the lumped-thermal setup from Walkthrough 5, run the same amplified UDDS trace with three heat-transfer coefficients: 5, 10, and 25 $\text{W}\cdot\text{m}^{-2}\cdot\text{K}^{-1}$. Report the maximum temperature rise and the maximum terminal-voltage deviation from the isothermal case.

Hints:

- Write a small helper function that accepts `h` and returns a solved simulation.

- Reuse the same current trace and model options each time.
- If you change two things at once, the comparison becomes much harder to interpret.

4.13 Worked Solutions

4.13.1 Solution to Exercise 1

```
import numpy as np
import matplotlib.pyplot as plt
import pybamm

experiment = pybamm.Experiment(
    [
        (
            "Discharge at C/10 for 10 minutes",
            "Rest for 50 minutes",
        )
    ]
    * 10,
    period="1 minute",
)

simulation = pybamm.Simulation(
    pybamm.lithium_ion.SPM(),
    parameter_values=pybamm.ParameterValues("Chen2020"),
    experiment=experiment,
    solver=pybamm.CasadiSolver(mode="fast"),
)
solution = simulation.solve()

time_s = solution["Time [s]"].entries
current_a = solution["Current [A]"].entries
voltage_v = solution["Terminal voltage [V]"].entries
discharge_capacity_ah = solution["Discharge capacity [A.h]"].entries

rest_mask = np.isclose(current_a, 0.0, atol=1e-6)
rest_end_indices = np.where(rest_mask & np.r_[~rest_mask[1:], True])[0]

quasi_ocv_v = voltage_v[rest_end_indices]
quasi_ocv_capacity_ah = discharge_capacity_ah[rest_end_indices]

fig, axes = plt.subplots(2, 1, figsize=(9, 8), constrained_layout=True)

axes[0].plot(time_s / 3600.0, current_a, linewidth=1.7)
axes[0].set_ylabel("Current [A]")
axes[0].set_title("GITT-like protocol")
axes[0].grid(True, alpha=0.3)

axes[1].plot(quasi_ocv_capacity_ah, quasi_ocv_v, marker="o", linewidth=1.7)
axes[1].set_xlabel("Discharge Capacity [A.h]")
axes[1].set_ylabel("Quasi-OCV [V]")
```

```
axes[1].grid(True, alpha=0.3)
```

```
plt.show()
```

The first plot should look like a staircase of short positive pulses separated by long zero-current rests. The second plot should show quasi-OCV decreasing smoothly with discharged capacity. The key lesson is methodological: GITT is not just a protocol string. It is a protocol plus a post-processing rule.

4.13.2 Solution to Exercise 2

```
from pathlib import Path
```

```
import numpy as np
import pandas as pd
import pybamm
```

```
def load_cycle(name):
    loader = pybamm.DataLoader()
    cycle = pd.read_csv(
        Path(loader.get_data(name)),
        comment="#",
        header=None,
        names=["time_s", "current_a"],
    ).dropna()
    cycle["time_s"] = cycle["time_s"] - cycle["time_s"].iloc[0]
    return cycle

def run_cycle(name):
    cycle = load_cycle(name)
    experiment = pybamm.Experiment(
        [pybamm.step.current(cycle[["time_s", "current_a"]].to_numpy())],
        period="1 second",
    )
    sim = pybamm.Simulation(
        pybamm.lithium_ion.SPMe(),
        parameter_values=pybamm.ParameterValues("Chen2020"),
        experiment=experiment,
        solver=pybamm.CasadiSolver(mode="fast"),
    )
    sol = sim.solve()
    return {
        "cycle": name,
        "final_soc": float(
            1.0
            - sol["Discharge capacity [A.h]"].entries[-1]
            / pybamm.ParameterValues("Chen2020")["Nominal cell capacity [A.h]"]
        ),
        "min_voltage": float(sol["Terminal voltage [V]"].entries.min()),
        "max_conc_overpotential": float(
            np.max(
                np.abs(
```

```

        sol["X-averaged battery concentration overpotential
↪ [V]"].entries
    )
    ),
}

```

```

for record in [run_cycle("UDDS.csv"), run_cycle("WLTC.csv")]:
    print(record)

```

The exact numbers depend on the packaged trace amplitudes, but the workflow is the point: same model, same solver, same metrics, different forcing.

4.13.3 Solution to Exercise 3

```

import numpy as np
import pandas as pd
import pybamm

```

```

loader = pybamm.DataLoader()
udds = pd.read_csv(
    loader.get_data("UDDS.csv"),
    comment="#",
    header=None,
    names=["time_s", "current_a"],
).dropna()
udds["time_s"] = udds["time_s"] - udds["time_s"].iloc[0]
udds["current_a"] = 2.0 * udds["current_a"]

experiment = pybamm.Experiment(
    [pybamm.step.current(udds[["time_s", "current_a"]].to_numpy())],
    period="1 second",
)

```

```

def solve_with_h(h_value):
    params = pybamm.ParameterValues("Chen2020")
    params.update(
        {
            "Ambient temperature [K]": 298.15,
            "Initial temperature [K]": 298.15,
            "Total heat transfer coefficient [W.m-2.K-1]": h_value,
        }
    )
    sim = pybamm.Simulation(
        pybamm.lithium_ion.SPMe({"thermal": "lumped"}),
        parameter_values=params,
        experiment=experiment,
        solver=pybamm.CasadiSolver(mode="fast"),
    )
    return sim.solve()

```



```

isothermal_solution = pybamm.Simulation(
    pybamm.lithium_ion.SPMe(),
    parameter_values=pybamm.ParameterValues("Chen2020"),
    experiment=experiment,
    solver=pybamm.CasadiSolver(mode="fast"),
).solve()

for h_value in [5.0, 10.0, 25.0]:
    sol = solve_with_h(h_value)
    max_temp_rise = float(
        sol["Volume-averaged cell temperature [K]"].entries.max() - 298.15
    )
    max_voltage_delta = float(
        np.max(
            np.abs(
                sol["Terminal voltage [V]"].entries
                - isothermal_solution["Terminal voltage [V]"].entries
            )
        )
    )
    print(
        {
            "h_W_m2_K": h_value,
            "max_temp_rise_K": round(max_temp_rise, 4),
            "max_voltage_delta_mV": round(max_voltage_delta * 1000, 2),
        }
    )

```

You should find that stronger cooling reduces peak temperature rise. Whether it materially changes voltage over one trace depends on the size of the thermal feedback under your chosen current amplitude. That is the answer the exercise is designed to make you quantify rather than guess.

4.14 What Changes for Sodium-Ion?

By Chapter 4, the sodium-ion adaptation is no longer a footnote. It starts affecting the workflow in visible ways.

First, parameter replacement becomes more central. PyBaMM now includes an official sodium-ion example model, but the public parameter ecosystem is still much sparser than for lithium-ion. In practice, that means you will spend more time doing exactly what Walkthroughs 1 and 2 taught: inspecting parameter values, replacing OCV functions, and documenting constitutive assumptions from papers.

Second, the OCV problem gets harder. Hard-carbon sodium-ion negative electrodes often exhibit flatter or more structured OCV regions than graphite lithium-ion cells. That makes voltage less informative over some SOC windows and increases the value of protocols like GITT for building quasi-equilibrium voltage maps.

Third, equal power does not mean equal current. Sodium-ion cells typically operate at lower nominal voltage than comparable lithium-ion cells, so a drive cycle expressed

in power terms often maps to higher current demand at the cell level. If you port a lithium-ion drive-cycle notebook to sodium-ion, current scaling deserves a deliberate second look.

Fourth, validation usually becomes less direct. With lithium-ion, you can often test a method against rich public datasets and then adapt it to your own chemistry. With sodium-ion, you may need to validate the workflow logic on lithium-ion public data first, then port it to sodium-ion parameter sets or digitized literature curves. That is not a weakness if you state it honestly. It is standard practice in a young data ecosystem.

Finally, low-temperature behavior matters sooner. One reason sodium-ion is attractive is its potential low-temperature robustness in some chemistries. That means the distinction between fixed-temperature and coupled-temperature modeling, which might be a small correction in one lithium-ion notebook, may become central in a sodium-ion study.

So the transferable lesson is this: Chapter 4's software patterns carry over cleanly, but sodium-ion raises the stakes on parameter provenance, OCV fidelity, and explicit validation strategy.

4.15 Chapter Summary and Skill Checklist

- You inspected parameter metadata with `get_info`, browsed thematic subsets with `list_by_category`, and compared published sets with `diff`.
- You made safe parameter edits by copying a baseline set, applying overrides, and saving both a human-readable override file and a full snapshot.
- You replaced function-valued parameters, including a table-based OCV function using `pybamm.Interpolant`.
- You encoded a full CC-rest-CCCV-rest experiment with `pybamm.Experiment`.
- You loaded a public UDDS drive-cycle file, validated its columns and timing, and ran it through SPMe.
- You compared isothermal and lumped-thermal dynamic simulations and quantified the consequences of thermal coupling.
- You reproduced the protocol structure of a published EV-style ageing cycle while documenting your approximations explicitly.

The commands, classes, and patterns that should now be entering muscle memory are:

- `pybamm.ParameterValues("Chen2020")`
- `params.get_info(...)`
- `params.list_by_category(...)`
- `params.diff(other_params)`
- `params.copy()`
- `params.update({...})`
- `params.to_json(...)`
- `pybamm.ParameterValues.from_json(...)`
- `pybamm.Experiment([...], period="...")`
- `pybamm.step.current(drive_cycle_array)`
- `pybamm.DataLoader().get_data("UDDS.csv")`

- `pybamm.Interpolant(x_data, y_data, symbolic_argument)`
- `pybamm.CasadiSolver(mode="fast")`

You should now be able to answer “yes” to each of these:

- I can inspect a PyBaMM parameter set without treating it as an opaque black box.
- I can explain the difference between changing a constant parameter, changing a function parameter, and changing the operating protocol.
- I can encode CC, CV, rest, and drive-cycle steps in a single reproducible experiment.
- I can import a public current trace and check its time base and sign convention before trusting the result.
- I can explain whether temperature was fixed or coupled in a given simulation and why that choice was acceptable.

If any of those boxes are still unchecked, revisit the relevant walkthrough before you move into Chapter 5. Parameter estimation is much harder when parameter provenance and protocol design are still fuzzy.

4.16 Deliverable

The deliverable for this chapter is:

A notebook that runs a realistic drive cycle on a commercial-like cell and produces publication-quality plots of voltage, current, SOC, and internal concentrations.

Approach it in four stages.

First, create a parameter-provenance section at the top of the notebook. Load a bundled parameter set, save an override file, and print the exact parameters you changed. The user-visible rule here is simple: if a reviewer asked what changed relative to the base chemistry, your notebook should answer in one screen.

Second, include one clean protocol section using `Experiment`. It can be the CC-rest-CCCV-rest example from Walkthrough 3 or a variant you design yourself.

Third, include one real-data section that loads `UDDS.csv` or another public drive cycle such as `WLTC.csv`, validates the file structure, and runs it through either `SPMe` or `DFN`. Save a figure with four aligned panels showing current, voltage, SOC, and one internal concentration variable.

Fourth, add a short interpretation paragraph. It should state which model fidelity you chose, which parameter edits matter scientifically, whether temperature was fixed or coupled, and why those choices are appropriate for a first research-grade notebook.

A strong partial deliverable includes:

- one top-to-bottom runnable notebook,
- one saved JSON override file,
- one saved protocol figure,
- one saved drive-cycle figure,
- one short written interpretation of the modeling choices.

4.17 Further Practice and Reading

1. PyBaMM Tutorial 4, “Setting parameter values”
https://docs.pybamm.org/en/stable/source/examples/notebooks/getting_started/tutorials/4-setting-parameter-values.html
2. PyBaMM Tutorial 5, “Run experiments”
https://docs.pybamm.org/en/v25.4.0/source/examples/notebooks/getting_started/tutorials/5-run-experiments.html
3. PyBaMM notebook, “Changing the input current when solving PyBaMM models”
<https://docs.pybamm.org/en/stable/source/examples/notebooks/parameterization/3-changing-input-current.html>
4. G. Pozzato, A. Allam, and S. Onori, “Lithium-ion battery aging dataset based on electric vehicle real-driving profiles,” *Data in Brief*, 41, 107995, 2022.
DOI 10.1016/j.dib.2022.107995
5. Y. Xiaolong Chen et al., “Development of Experimental Techniques for Parameterization of Multi-scale Lithium-ion Battery Models,” *Journal of The Electrochemical Society*, 167, 080534, 2020.
DOI 10.1149/1945-7111/ab9050
6. PyBaMM official sodium-ion example
<https://docs.pybamm.org/en/stable/source/examples/notebooks/models/sodium-ion.html>
7. EPA Dynamometer Drive Schedules page
<https://www.epa.gov/vehicle-and-fuel-emissions-testing/dynamometer-drive-schedules>
8. PyBaMM GitHub discussions for practical implementation questions
<https://github.com/pybamm-team/PyBaMM/discussions>